

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ GIAO THÔNG VẬN TẢI
KHOA CÔNG NGHỆ THÔNG TIN



ĐINH VIỆT LINH

**ĐỀ TÀI: XÂY DỰNG HỆ THỐNG QUẢN LÝ
TÀI CHÍNH NỘI BỘ DOANH NGHIỆP
TÍCH HỢP TRỢ LÝ AI AGENT.**

ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC CHÍNH QUY

Ngành: Hệ thống thông tin

HÀ NỘI – 2026

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ GIAO THÔNG VẬN TẢI
KHOA CÔNG NGHỆ THÔNG TIN**

ĐỒ ÁN TỐT NGHIỆP

**ĐỀ TÀI: XÂY DỰNG HỆ THỐNG QUẢN LÝ TÀI CHÍNH NỘI BỘ
DOANH NGHIỆP
TÍCH HỢP TRỢ LÝ AI AGENT.**

Ngành: Hệ thống thông tin

Giảng viên hướng dẫn: Lê Thị Chi

Hà Nội – 2026

NHẬN XÉT VÀ ĐÁNH GIÁ

LỜI NÓI ĐẦU

Trong thời đại chuyển đổi số phát triển mạnh mẽ như hiện nay, việc ứng dụng công nghệ thông tin vào hoạt động quản trị nội bộ đã trở thành yếu tố then chốt giúp doanh nghiệp tối ưu hóa nguồn lực và nâng cao năng lực cạnh tranh. Trong số đó, quản lý tài chính luôn là mạch máu của mọi tổ chức, đòi hỏi sự chính xác, minh bạch và kịp thời.

Các phương thức quản lý tài chính truyền thống dựa trên sổ sách hoặc các công cụ rời rạc hiện nay đã lộ rõ nhiều hạn chế như: dễ xảy ra sai sót dữ liệu, quy trình phê duyệt thủ công tốn thời gian, và đặc biệt là thiếu đi khả năng phân tích dự báo luồng tiền theo thời gian thực. Điều này đặt ra một nhu cầu cấp thiết về việc xây dựng một hệ thống quản lý tài chính nội bộ thông minh, tập trung và có khả năng tự động hóa cao.

Cùng với sự bứt phá của Trí tuệ nhân tạo (AI), đặc biệt là mô hình Trợ lý AI Agent, công nghệ giờ đây không chỉ dừng lại ở việc xử lý dữ liệu thô mà còn có khả năng hiểu bối cảnh, phân tích tài chính đa chiều và hỗ trợ ra quyết định. Việc tích hợp AI Agent vào hệ thống quản lý tài chính giúp tối ưu hóa quy trình kiểm toán nội bộ, tự động cảnh báo rủi ro dòng tiền và tối giản hóa thao tác của nhà quản trị thông qua tương tác ngôn ngữ tự nhiên.

Thông qua đề tài này, em mong muốn vận dụng những kiến thức đã tích lũy về kiến trúc hệ thống (Microservices), cơ sở dữ liệu và trí tuệ nhân tạo để xây dựng một giải pháp hoàn chỉnh, góp phần số hóa toàn diện quy trình tài chính và nâng cao hiệu quả quản trị cốt lõi của doanh nghiệp.

Trong quá trình thực hiện đề tài, mặc dù đã cố gắng hết sức, nhưng do hạn chế về thời gian, kinh nghiệm và kiến thức thực tế, đồ án khó tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý, chỉ bảo tận tình từ quý thầy cô trong Hội đồng để đề tài được hoàn thiện hơn.

Em xin chân thành cảm ơn!

LỜI CẢM ƠN

Trong quá trình học tập và nghiên cứu tại **Trường Đại học Công nghệ Giao thông Vận tải**, em xin gửi lời cảm ơn sâu sắc và chân thành đến tất cả những cá nhân, tập thể đã giúp đỡ, đóng góp ý kiến để em có thể hoàn thiện đồ án tốt nghiệp này.

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc đến cô **Lê Thị Chi** – giảng viên hướng dẫn.

Cô là người đã trực tiếp chỉ bảo, nhận xét và hỗ trợ em trong suốt quá trình thực hiện đề tài.

Sự tận tâm, kiên nhẫn cùng những hướng dẫn chi tiết và kiến thức chuyên môn sâu rộng của cô đã giúp em củng cố kiến thức và hoàn thiện các kỹ năng cần thiết để hoàn thành đồ án.

Tiếp theo, em xin chân thành cảm ơn Ban Giám hiệu nhà trường cùng toàn thể các thầy cô trong Khoa Công nghệ Thông tin đã tạo điều kiện thuận lợi, cung cấp nền tảng kiến thức vững chắc cho em trong suốt quá trình học tập và thực hiện đồ án tốt nghiệp.

Bên cạnh đó, em xin gửi lời cảm ơn đến tập thể lớp 73DCHT22 – những người bạn đã luôn đồng hành, động viên và hỗ trợ em trong quá trình học tập cũng như khi thực hiện đồ án.

Cuối cùng, em xin gửi lời cảm ơn sâu sắc đến gia đình – chỗ dựa tinh thần vững chắc, luôn động viên và tạo điều kiện tốt nhất về cả vật chất lẫn tinh thần để em có thể hoàn thành đồ án này.

Mặc dù đã rất cố gắng trong quá trình thực hiện, nhưng do hạn chế về thời gian, kiến thức và kinh nghiệm thực tế, đồ án không thể tránh khỏi những thiếu sót.

Em rất mong nhận được sự đánh giá, góp ý và chỉ bảo từ quý thầy cô và mọi người để em có thể hoàn thiện hơn.

Em xin chân thành cảm ơn!

LỜI CAM ĐOAN

Em xin cam đoan đồ án này là kết quả nghiên cứu của riêng em.

Các tài liệu trong đồ án được sử dụng trung thực, nguồn trích dẫn có ghi chú rõ ràng, minh bạch.

Các kết quả trong đồ án có tính độc lập riêng và chưa được công bố trong các công trình khác. Nếu không đúng như đã nêu, em xin hoàn toàn chịu trách nhiệm về đề tài của mình.

Hà Nội, ngày tháng năm

Sinh viên thực hiện

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU	1
1.1. Lý do chọn đề tài	1
1.2. Mục tiêu của đề tài	1
1.2.1. Mục tiêu tổng quát	1
1.2.2. Mục tiêu cụ thể	2
1.3. Giới hạn và phạm vi của đề tài	2
1.3.1. Phạm vi nghiên cứu	2
1.3.2. Phạm vi cụ thể bao gồm	2
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ ỨNG DỤNG	4
2.1. Kiến trúc hệ thống phân tán (Microservices Architecture)	4
2.1.1. Tổng quan về kiến trúc Microservices	4
2.1.2. Cấu trúc các thành phần trong EFMS	4
2.1.3. Chiến lược Multi-tenancy – Shared Database, Separate Schema	5
2.1.4. Giao tiếp giữa các dịch vụ và giải pháp N+1 Query	5
2.2. Công nghệ phát triển Front-end	6
2.2.1. React.js và Vite – Nền tảng giao diện hiện đại	6
2.2.2. React Router – Định tuyến ứng dụng trang đơn	6
2.2.3. Shaden UI và Tailwind CSS – Hệ thống thiết kế giao diện	6
2.2.4. Axios và Zustand – Quản lý dữ liệu và trạng thái	6
2.3. Công nghệ phát triển Back-end	7
2.3.1. Framework Java Spring Boot	7
2.3.2. Kiến trúc ba tầng (Layered Architecture)	7
2.3.3. Xác thực và Phân quyền bằng JSON Web Token (JWT)	8
2.4. Tích hợp Trí tuệ Nhân tạo qua Giao thức Model Context Protocol (MCP)	8
2.4.1. Tổng quan về Model Context Protocol	8
2.4.2. Ba trụ cột tương tác của MCP	8
2.4.3. Cơ chế xác thực OAuth 2.1 trong MCP Server	9
2.5. Hệ quản trị Cơ sở dữ liệu PostgreSQL	9
2.5.1. Lý do lựa chọn PostgreSQL	9
2.5.2. Cơ chế kiểm soát đồng thời đa phiên bản (MVCC)	9
2.5.3. Kiểu dữ liệu JSONB và Audit Logging	9
2.6. Tiêu chuẩn Thiết kế Hệ thống	10
2.6.1. Chuẩn phản hồi API thống nhất (ApiResponse<T>)	10
2.6.2. Hệ thống lưu vết thao tác (Audit Logging)	10

2.6.3. Quy chuẩn mã nguồn và kiểu dữ liệu	10
2.7. Kiểm thử Phần mềm (Software Testing)	11
2.7.1. Chiến lược kiểm thử tổng thể	11
2.7.2. Kiểm thử Hộp đen (Black-box Testing)	11
2.7.3. Kiểm thử Giao thức Mạng (API Testing)	12
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	13
3.1. Khảo sát hệ thống và đặc tả nghiệp vụ	13
3.1.1. Khảo sát hệ thống	13
3.1.2. Đặc tả nghiệp vụ	13
3.2. Phân tích hệ thống	13
3.2.1. Xác định tác nhân hệ thống	13
3.2.2. Biểu đồ Usecase	14
3.2.3. Biểu đồ Trình tự (Sequence Diagram)	44
3.2.4. Biểu đồ hoạt động (Activity Diagram)	53
3.2.5. Biểu đồ lớp (Class Diagram)	56
CHƯƠNG 4. XÂY DỰNG CHƯƠNG TRÌNH	72
4.1. Xây dựng hệ thống	72
4.1.1. Xây dựng hệ thống	72
4.1.2. Lý do chọn công nghệ	72
4.1.3. Khả năng mở rộng (Scalability)	72
4.1.4. Chi tiết nghiệp vụ và Vận hành hệ thống	73
4.2. Giao diện người dùng	74
4.3. Kiểm thử	82
4.3.1. Kiểm thử chức năng Đăng nhập & Xác thực (Identity Service)	82
4.3.2. Kiểm thử chức năng Quản lý Người dùng (Multi-tenancy & RBAC) ..	83
4.3.3. Kiểm thử luồng nghiệp vụ Hóa đơn mua hàng (AP Bill - Core Service)	83
KẾT LUẬN	85
1. Kết quả đạt được	85
2. Hạn chế	85
3. Hướng phát triển đồ án	85
TÀI LIỆU THAM KHẢO	87

CHƯƠNG 1. GIỚI THIỆU

1.1. Lý do chọn đề tài

Trong kỷ nguyên công nghiệp 4.0 và xu hướng chuyển đổi số đang diễn ra mạnh mẽ trên toàn cầu, việc ứng dụng công nghệ thông tin vào công tác quản trị doanh nghiệp không còn là một lựa chọn mà đã trở thành yếu tố sống còn để duy trì và nâng cao lợi thế cạnh tranh.

Đối với lĩnh vực quản trị tài chính - kế toán, sự chính xác, minh bạch, bảo mật và khả năng xử lý khối lượng lớn giao dịch theo thời gian thực (real-time) là những yêu cầu tối quan trọng.

Tuy nhiên, thực trạng hiện nay cho thấy nhiều doanh nghiệp vẫn đang vận hành trên các hệ thống kế toán truyền thống được xây dựng theo kiến trúc khối nguyên khối (Monolithic Architecture).

Các hệ thống này thường bộc lộ nhiều điểm hạn chế nghiêm trọng khi quy mô dữ liệu và số lượng người dùng tăng đột biến: khả năng mở rộng (scalability) kém, chi phí và thời gian bảo trì cao, hệ thống trở nên cồng kềnh khó kiểm soát, khó khăn trong việc tích hợp với các hệ sinh thái phần mềm bên ngoài, và đặc biệt là những rủi ro đi kèm nguy cơ gián đoạn toàn bộ dịch vụ (single point of failure) khi một phân hệ nhỏ gặp sự cố.

Bên cạnh đó, với sự phát triển của các mô hình kinh doanh theo hình thức tập đoàn, tổng công ty có nhiều công ty con và chi nhánh, nhu cầu cấp bách đặt ra là cần một nền tảng hỗ trợ đa công ty (Multi-tenant).

Hệ thống này phải cho phép ban lãnh đạo quản lý tập trung toàn diện, nhưng đồng thời vẫn đảm bảo tính độc lập tuyệt đối và sự riêng tư dữ liệu của từng đơn vị thành viên.

Mặt khác, các quy trình quản lý phê duyệt tài chính nội bộ truyền thống thường dựa trên giấy tờ hoặc các công cụ giao tiếp rời rạc, dẫn đến tình trạng ách tắc, thiếu minh bạch, rủi ro gian lận nhân sự và không đáp ứng được tính đối soát tự động nhanh chóng.

Xuất phát từ những vấn đề nhức nhối và thách thức kỹ thuật nêu trên, dự án "Xây dựng hệ thống quản lý tài chính doanh nghiệp EFMS (Enterprise Financial Management System)" đã được tiến hành nghiên cứu và phát triển.

EFMS lấy cốt lõi là kiến trúc Microservices linh hoạt tiên tiến, kết hợp cùng các nền tảng kỹ thuật hiện đại bậc nhất hiện nay như Java Spring Boot, công cụ thiết kế quy trình Camunda 8 cho bài toán tối ưu hóa quy trình (Business Process Automation).

Hệ thống không chỉ dừng lại ở việc giải quyết xuất sắc các bài toán kế toán truyền thống mà còn nhắm tới việc kiến tạo tính tự động hóa triệt để trong các luồng công việc phê duyệt tài chính, tối ưu hóa các quyết định nghiệp vụ kinh doanh

1.2. Mục tiêu của đề tài

1.2.1. Mục tiêu tổng quát

Xây dựng hệ thống quản trị tài chính doanh nghiệp EFMS (Enterprise Financial Management System) dựa trên kiến trúc Microservices hiện đại, hỗ trợ mô hình đa công ty (Multi-tenant) và tự động hóa quy trình nghiệp vụ (BPA), nhằm tối ưu hóa

công tác quản lý tài chính, đảm bảo tính minh bạch, bảo mật và khả năng mở rộng cao cho doanh nghiệp.

1.2.2. Mục tiêu cụ thể

- **Phân tích thiết kế:** Phân tích các nghiệp vụ kế toán chuyên sâu (Kế toán kép, Sổ cái, Công nợ) và quy trình phê duyệt tài chính trong doanh nghiệp. Thiết kế kiến trúc hệ thống theo mô hình Microservices và triển khai cơ sở dữ liệu hỗ trợ Multi-tenancy để cô lập dữ liệu khách hàng.
- **Về xây dựng hệ thống (Frontend & Backend):**
 - Xây dựng các dịch vụ nền tảng: Identity Service (Xác thực JWT, phân quyền RBAC) và API Gateway (Điều phối request, bảo mật lớp ngoài).
 - Hiện thực hóa dịch vụ lõi Core Service: Quản lý hệ thống tài khoản, hạch toán chứng từ và theo dõi báo cáo tuổi nợ.
 - Phát triển dịch vụ tiện ích Common Service: Quản lý tệp đính kèm và bình luận bằng mô hình liên kết đa hình (Polymorphic Association).
- **Về tự động hóa quy trình (Workflow Automation):**
 - Tích hợp Camunda Platform 8 để số hóa và vận hành quy trình phê duyệt hóa đơn đầu vào (AP Bill Approval).
 - Xây dựng các Service Task tự động xử lý dữ liệu và User Task dành cho các cấp lãnh đạo phê duyệt.

1.3. Giới hạn và phạm vi của đề tài

1.3.1. Phạm vi nghiên cứu

Hệ thống EFMS được thiết kế như một nền tảng quản trị tài chính toàn diện.

Trong khuôn khổ đề án, hệ thống tập trung hiện thực hóa các phân hệ xương sống dựa trên kiến trúc Microservices, đảm bảo tính hội tụ giữa nghiệp vụ kế toán chuẩn mực và công nghệ vận hành hiện đại.

1.3.2. Phạm vi cụ thể bao gồm

- **Cổng giao tiếp ứng dụng (EFMS API Gateway):**
 - Điểm kết nối duy nhất (Single Entry Point), bảo vệ và điều phối luồng dữ liệu toàn hệ thống.
 - Định tuyến động (Dynamic Routing): Tự động phân giải và chuyển tiếp yêu cầu từ phía người dùng đến các Microservices tương ứng.
 - Quản trị lưu lượng: Tích hợp cơ chế cân bằng tải (Load Balancing), kiểm soát tần suất truy cập (Rate Limiting) và xử lý chính xác cấu hình CORS.
- **Phân hệ Quản lý định danh & Phân quyền (EFMS Identity Service):**
 - Đóng vai trò là "người gác cổng" trung tâm, chịu trách nhiệm bảo mật và điều phối truy cập.
 - Xác thực & Bảo mật: Triển khai cơ chế xác thực tập trung, cấp phát và quản lý vòng đời JSON Web Tokens (JWT).
 - Kiến trúc Đa thuê (Multi-tenancy): Thiết lập lõi cơ sở dữ liệu hỗ trợ ngăn

cách dữ liệu tuyệt đối giữa các công ty/chi nhánh (Tenants).

- Kiểm soát truy cập (RBAC): Xây dựng ma trận quyền dựa trên vai trò (Role-based Access Control), kiểm soát chi tiết đến từng Endpoint API.
- **Phân hệ Quản lý định danh & Phân quyền (EFMS Core Service):**
 - "Trái tim" của hệ thống, nơi thực thi các quy tắc tài chính và hạch toán dữ liệu.
 - Kế toán chuẩn mực: Duy trì hệ thống tài khoản (Chart of Accounts), áp dụng nguyên tắc kế toán kép (Double-entry) và quản lý sổ cái chung (General Ledger).
 - Quản trị công nợ: Theo dõi chi tiết các khoản phải thu/phải trả, tự động kết xuất báo cáo tuổi nợ phục vụ quản trị dòng tiền.
- **Phân hệ tích tích dùng chung (EFMS Common Service):**
 - Cung cấp hạ tầng phục vụ ngang (Cross-cutting) cho toàn bộ các dịch vụ khác.
 - Quản lý dữ liệu đa hình: Áp dụng mô hình Polymorphic Association để gắn tệp đính kèm (Attachments) và bình luận (Comments) vào bất kỳ thực thể nào mà không làm thay đổi cấu trúc bảng.
 - Tối ưu hiệu năng: Triển khai cơ chế Batch Fetching để truy xuất dữ liệu liên dịch vụ, triệt tiêu vấn đề N+1 Query trong môi trường phân tán.
- **Phân hệ trợ lý thông minh (EFMS MCP Server):**
 - Lớp giao tiếp hiện đại, kết nối dữ liệu tài chính với sức mạnh của trí tuệ nhân tạo.
 - Giao thức chuẩn hóa: Triển khai Model Context Protocol (MCP), cho phép các mô hình ngôn ngữ lớn (LLM) truy cập an toàn vào ngữ cảnh dữ liệu của hệ thống.
 - Hỗ trợ ra quyết định: Cung cấp các công cụ tra cứu trạng thái quy trình và phân tích biến động tài chính thông qua giao diện hội thoại tự nhiên.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ ỨNG DỤNG

Để xây dựng một hệ thống Phần mềm Quản lý Tài chính Nội bộ Doanh nghiệp (Enterprise Financial Management System – EFMS) có khả năng vận hành thực tế, đảm bảo tính chính xác tuyệt đối về số liệu kế toán và đáp ứng yêu cầu về tính sẵn sàng cao (High Availability), việc xác lập một nền tảng lý thuyết vững chắc và lựa chọn công nghệ phù hợp là điều kiện tiên quyết. Hệ thống EFMS được định hình không chỉ nhằm giải quyết các bài toán nghiệp vụ hạch toán thông thường, mà còn phải xử lý triệt để các vấn đề về hiệu năng, bảo mật hạ tầng và khả năng mở rộng quy mô (Scalability) khi doanh nghiệp tăng trưởng theo thời gian.

Chương này trình bày có hệ thống các cơ sở lý thuyết và công nghệ được áp dụng, bao gồm sáu nội dung chính: (1) Kiến trúc hệ thống phân tán theo mô hình Microservices; (2) Hệ sinh thái công nghệ phát triển Front-end; (3) Nền tảng Back-end và cơ chế bảo mật; (4) Tích hợp Trí tuệ Nhân tạo thông qua giao thức Model Context Protocol; (5) Hệ quản trị cơ sở dữ liệu PostgreSQL; và (6) Các tiêu chuẩn thiết kế hệ thống và quy trình kiểm thử phần mềm.

2.1. Kiến trúc hệ thống phân tán (Microservices Architecture)

2.1.1. Tổng quan về kiến trúc Microservices

Kiến trúc Microservices là một phương pháp thiết kế phần mềm trong đó một ứng dụng phức tạp được phân rã thành tập hợp các dịch vụ nhỏ, độc lập, mỗi dịch vụ thực hiện một tập trách nhiệm nghiệp vụ được xác định rõ ràng. Khác với kiến trúc nguyên khối (Monolithic Architecture) truyền thống – nơi toàn bộ logic ứng dụng được đóng gói trong một đơn vị triển khai duy nhất – Microservices cho phép mỗi dịch vụ được phát triển, triển khai, mở rộng và bảo trì một cách độc lập.

Đối với hệ thống EFMS, lựa chọn kiến trúc Microservices xuất phát từ yêu cầu nghiêm ngặt về tính cô lập lỗi (Fault Isolation) và khả năng mở rộng chọn lọc. Trong môi trường tài chính doanh nghiệp, một lỗi xảy ra tại module xử lý hóa đơn không được phép lan sang module xác thực người dùng hay sổ cái kế toán. Kiến trúc phân tán giải quyết vấn đề này bằng cách thiết lập ranh giới rõ ràng giữa các miền nghiệp vụ.

2.1.2. Cấu trúc các thành phần trong EFMS

Hệ thống EFMS được phân rã thành bốn thành phần dịch vụ chính, mỗi thành phần hoạt động độc lập trên cổng riêng và chịu trách nhiệm về một miền nghiệp vụ cụ thể:

Thành phần	Vai trò và Đặc điểm kỹ thuật
API Gateway (Port 8080)	Điểm truy cập duy nhất (Single Entry Point) cho toàn bộ hệ thống. Thực hiện định tuyến động (Dynamic Routing) đến các dịch vụ nội bộ, kiểm soát và xác thực JWT, cấu hình CORS và áp dụng cơ chế giới hạn truy cập (Rate Limiting) nhằm ngăn chặn tấn công từ chối dịch vụ.
Identity Service (Port 8081)	Quản lý toàn bộ vòng đời xác thực và phân quyền: đăng ký tài khoản, cấp phát

	JWT, quản lý cấu trúc đa công ty (Multi-company), vai trò (Role), quyền hạn (Permission) theo mô hình RBAC và lưu vết thao tác.
Core Service (Port 8082)	Thực thi toàn bộ nghiệp vụ tài chính – kế toán cốt lõi: danh mục tài khoản (Chart of Accounts), quản lý đối tác (Partners), hóa đơn mua vào/bán ra (AP/AR Invoices), thanh toán (Payments), sổ cái bút toán kép (Journal Entries) và tài khoản ngân hàng (Bank Accounts).
Common Service (Port 8083)	Cung cấp các tính năng dùng chung cho toàn hệ thống theo mô hình đa hình (Polymorphic): quản lý tệp đính kèm (Attachments), bình luận và thảo luận (Comments), lưu vết thay đổi dữ liệu (Audit Logs) mà không ràng buộc khóa ngoại cứng với bất kỳ miền nghiệp vụ nào.

2.1.3. Chiến lược Multi-tenancy – Shared Database, Separate Schema

EFMS được thiết kế để phục vụ nhiều doanh nghiệp (Tenant) trên cùng một hạ tầng vật lý. Trong ba mô hình Multi-tenancy phổ biến – Separate Database, Shared Database/Separate Schema và Shared Database/Shared Schema – hệ thống lựa chọn mô hình Shared Database kết hợp Separate Schema nhằm cân bằng giữa chi phí hạ tầng và yêu cầu cô lập dữ liệu tuyệt đối.

Theo mô hình này, mỗi doanh nghiệp thành viên sở hữu một schema riêng biệt trong cùng một database server PostgreSQL. Cách tiếp cận này đảm bảo rằng dữ liệu kế toán của từng đơn vị được ngăn cách hoàn toàn ở tầng cơ sở dữ liệu, trong khi tài nguyên máy chủ vẫn được chia sẻ hiệu quả. Ở tầng ứng dụng, mọi truy vấn đều bắt buộc phải đính kèm điều kiện lọc theo `company_id`, được truyền xuống từ JWT token thông qua header `X-User-Company-Id` do API Gateway inject.

2.1.4. Giao tiếp giữa các dịch vụ và giải pháp N+1 Query

Trong môi trường Microservices, mỗi dịch vụ quản lý kho dữ liệu riêng, dẫn đến một thách thức phổ biến: khi cần hiển thị thông tin tổng hợp từ nhiều nguồn, hệ thống dễ rơi vào bẫy N+1 Query – thực hiện N lần gọi API riêng lẻ để lấy thông tin cho N bản ghi, thay vì một lần gọi duy nhất.

EFMS giải quyết vấn đề này bằng cơ chế Batch Fetching: thay vì gọi sang Identity Service để lấy tên người tạo cho từng chứng từ một, Core Service và Common Service thu thập toàn bộ danh sách ID người dùng liên quan, sau đó thực hiện một lần gọi đến endpoint nội bộ `POST /internal/users/batch` để lấy về tất cả thông tin cần thiết. Kết quả là độ phức tạp truy vấn được tối ưu từ $O(N)$ xuống $O(1)$ lần gọi dịch vụ chéo, cải thiện đáng kể thời gian phản hồi khi xử lý danh sách lớn.

2.2. Công nghệ phát triển Front-end

2.2.1. React.js và Vite – Nền tảng giao diện hiện đại

React.js là thư viện JavaScript mã nguồn mở do Meta phát triển, sử dụng mô hình lập trình dựa trên thành phần (Component-based Architecture) và cơ chế DOM ảo (Virtual DOM) để tối ưu hóa hiệu năng kết xuất giao diện. Thay vì cập nhật trực tiếp cây DOM thực mỗi khi dữ liệu thay đổi, React duy trì một bản sao DOM trong bộ nhớ, so sánh sự khác biệt (Diffing Algorithm) và chỉ áp dụng những thay đổi tối thiểu cần thiết. Đặc điểm này đặc biệt có giá trị trong giao diện ERP tài chính, nơi các lưới dữ liệu (Data Grids) có thể chứa hàng nghìn dòng bút toán cần được làm mới liên tục.

Vite là công cụ biên dịch (Build Tool) thế hệ mới thay thế Webpack truyền thống. Thay vì đóng gói toàn bộ mã nguồn trước khi khởi động, Vite tận dụng Native ES Modules (ESM) được hỗ trợ bởi các trình duyệt hiện đại để chỉ biên dịch các module thực sự được yêu cầu tại thời điểm đó. Kết quả là thời gian khởi động máy chủ phát triển giảm từ hàng chục giây xuống còn dưới một giây, trong khi tính năng Hot Module Replacement (HMR) cho phép thay đổi mã nguồn được phản ánh tức thì lên trình duyệt mà không cần tải lại trang.

2.2.2. React Router – Định tuyến ứng dụng trang đơn

EFMS được xây dựng theo mô hình Single Page Application (SPA), trong đó toàn bộ điều hướng giữa các phân hệ được xử lý hoàn toàn phía trình duyệt mà không cần tải lại trang từ máy chủ. React Router đóng vai trò quản lý hệ thống định tuyến lồng cấp (Nested Routing), ánh xạ các đường dẫn URL đến các thành phần giao diện tương ứng, đồng thời hỗ trợ bảo vệ các route yêu cầu xác thực (Protected Routes) bằng cách kiểm tra trạng thái đăng nhập trước khi cho phép truy cập.

2.2.3. Shadcn UI và Tailwind CSS – Hệ thống thiết kế giao diện

Tailwind CSS áp dụng triết lý Utility-first CSS, cho phép lập trình viên xây dựng giao diện bằng cách kết hợp các lớp tiện ích có mục đích đơn (single-purpose utility classes) trực tiếp trong cấu trúc HTML/JSX, thay vì viết các tệp CSS riêng biệt. Phương pháp này loại bỏ chi phí đặt tên lớp CSS, ngăn ngừa xung đột kiểu dáng và giữ cho tệp CSS cuối cùng ở kích thước tối thiểu nhờ cơ chế loại bỏ mã không dùng (Tree Shaking).

Shadcn UI là bộ thành phần giao diện (Component Library) theo triết lý "Unstyled Copy/Paste UI". Khác với các thư viện đóng gói (như Material UI hay Ant Design) – nơi các thành phần được phân phối dưới dạng gói npm và lập trình viên không có quyền kiểm soát mã nguồn bên trong – Shadcn UI cung cấp mã nguồn của từng thành phần trực tiếp vào dự án. Lập trình viên sở hữu và chỉnh sửa toàn bộ logic nội bộ, đảm bảo tính linh hoạt tối đa, dễ dàng tùy biến cho từng yêu cầu nghiệp vụ cụ thể và không bị ràng buộc bởi phiên bản thư viện bên ngoài.

2.2.4. Axios và Zustand – Quản lý dữ liệu và trạng thái

Axios là thư viện HTTP client dựa trên Promise, cung cấp API nhất quán để thực hiện các yêu cầu HTTP từ phía trình duyệt. Tính năng quan trọng nhất được khai thác trong EFMS là Axios Interceptors: một cơ chế cho phép can thiệp (intercept) vào các yêu cầu HTTP trước khi gửi đi và các phản hồi trước khi xử lý. Cụ thể, interceptor yêu cầu tự động đính kèm JWT token vào header Authorization của mọi yêu cầu API,

trong khi interceptor phản hồi xử lý luồng làm mới token (Token Refresh Flow) khi nhận được mã lỗi 401, đảm bảo phiên làm việc liên tục mà không yêu cầu người dùng đăng nhập lại.

Zustand là thư viện quản lý trạng thái (State Management) gọn nhẹ, không phụ thuộc vào các khái niệm phức tạp như Reducers hay Actions của Redux. Zustand cung cấp một kho trạng thái tập trung (Centralized Store) có thể được truy cập và cập nhật từ bất kỳ thành phần nào trong ứng dụng, đặc biệt phù hợp để điều phối dữ liệu tài chính chia sẻ giữa nhiều phân hệ giao diện.

2.3. Công nghệ phát triển Back-end

2.3.1. Framework Java Spring Boot

Spring Boot là framework Java dựa trên nền tảng Spring Framework, áp dụng triết lý "Convention over Configuration" (ưu tiên quy ước hơn cấu hình) để tối giản hóa quá trình thiết lập và khởi động ứng dụng. Thay vì phải cấu hình thủ công từng thành phần, Spring Boot cung cấp cơ chế Auto-configuration tự động phát hiện và cấu hình các dependency có mặt trong classpath.

Hai cơ chế cốt lõi của Spring – Inversion of Control (IoC) và Dependency Injection (DI) – cho phép quản lý vòng đời của các đối tượng (Bean) một cách tập trung. Thay vì các đối tượng tự khởi tạo các phụ thuộc của mình (tạo ra liên kết cứng – Tight Coupling), IoC Container của Spring chịu trách nhiệm tạo và tiêm các phụ thuộc cần thiết vào mỗi đối tượng. Đây là nền tảng cho khả năng kiểm thử đơn vị (Unit Testing) và khả năng thay thế các thành phần mà không ảnh hưởng đến phần còn lại của hệ thống.

Đối với hệ thống xử lý nghiệp vụ tài chính, Java được lựa chọn vì hai đặc tính sống còn: khả năng xử lý đa luồng (Multithreading) được kiểm soát chặt chẽ thông qua JVM, và hệ thống kiểu dữ liệu tĩnh, mạnh mẽ (Strongly-typed System) giúp phát hiện sai sót ngay tại thời điểm biên dịch thay vì trong quá trình vận hành thực tế.

2.3.2. Kiến trúc ba tầng (Layered Architecture)

Toàn bộ các dịch vụ backend của EFMS tuân thủ kiến trúc ba tầng tiêu chuẩn, thể hiện nguyên tắc Trách nhiệm Duy nhất (Single Responsibility Principle – SRP) của SOLID:

- **Controller Layer (Tầng trình bày):** Đây là lớp ngoài cùng tiếp nhận các yêu cầu HTTP từ API Gateway. Controller chịu trách nhiệm kiểm tra và xác thực định dạng dữ liệu đầu vào thông qua annotation `@Valid`, sau đó chuyển tiếp yêu cầu đến Service Layer. Controller giữ logic ở mức tối thiểu – không chứa bất kỳ quy tắc nghiệp vụ nào.
- **Service Layer (Tầng nghiệp vụ):** Đây là trung tâm xử lý của hệ thống, nơi tàng trữ và thực thi toàn bộ quy tắc nghiệp vụ kế toán, bao gồm các quy tắc định khoản, kiểm tra tính hợp lệ của giao dịch và điều phối luồng xử lý đa bước. Mọi quyết định nghiệp vụ đều được thực hiện tại tầng này.
- **Repository Layer (Tầng truy cập dữ liệu):** Tầng đáy chịu trách nhiệm duy nhất là tương tác với cơ sở dữ liệu PostgreSQL thông qua cơ chế ORM (Object-Relational

Mapping) của Spring Data JPA. Mọi truy vấn tại tầng này đều bắt buộc bao gồm điều kiện lọc theo companyId để đảm bảo cô lập dữ liệu đa thuê.

2.3.3. Xác thực và Phân quyền bằng JSON Web Token (JWT)

Hệ thống EFMS áp dụng mô hình xác thực Phi Trạng thái (Stateless Authentication) thông qua chuẩn JSON Web Token (JWT – RFC 7519), thay thế hoàn toàn cho cơ chế lưu trữ phiên phía máy chủ (Server-side Session) truyền thống vốn tạo ra điểm nghẽn cổ chai trong môi trường phân tán.

JWT bao gồm ba phần được mã hóa Base64URL và nối nhau bằng dấu chấm: Header (chứa thuật toán ký – HS256 hoặc RS256), Payload (chứa các claim: userId, email, companyId, roles/permissions) và Signature (chữ ký số xác minh tính toàn vẹn của token). Luồng xác thực được triển khai theo bốn giai đoạn: (1) Gateway xác minh chữ ký và hạn sử dụng token; (2) Gateway giải mã các claim và inject vào header HTTP nội bộ (X-User-Id, X-User-Company-Id, X-User-Permission); (3) Các dịch vụ downstream đọc header thông qua GatewayHeaderFilter và nạp vào SecurityContextHolder; (4) Kiểm soát truy cập chi tiết (RBAC) được thực thi bằng annotation `@PreAuthorize("hasAuthority('...')")` tại phương thức Controller hoặc Service.

2.4. Tích hợp Trí tuệ Nhân tạo qua Giao thức Model Context Protocol (MCP)

2.4.1. Tổng quan về Model Context Protocol

Model Context Protocol (MCP) là một giao thức mã nguồn mở do Anthropic phát triển và công bố vào cuối năm 2024, với mục tiêu chuẩn hóa cách thức các mô hình ngôn ngữ lớn (Large Language Models – LLM) kết nối và tương tác với các nguồn dữ liệu, công cụ bên ngoài. MCP định nghĩa một kiến trúc Client – Server rõ ràng: AI Model đóng vai trò Client gửi yêu cầu, trong khi MCP Server đóng vai trò trung gian, kiểm soát luồng thông tin giữa AI và hệ thống nội bộ doanh nghiệp.

Trong bối cảnh EFMS, MCP giải quyết một thách thức mang tính căn bản: làm thế nào để tận dụng năng lực phân tích của AI hiện đại mà không để dữ liệu tài chính nhạy cảm rời khỏi hạ tầng kiểm soát của doanh nghiệp. Thay vì gửi dữ liệu kế toán thô lên các dịch vụ AI công khai (Public AI APIs), MCP Server nằm ẩn trong hạ tầng nội bộ, chỉ cấp phát cho AI những thông tin cụ thể được yêu cầu thông qua các công cụ (Tools) đã được định nghĩa sẵn và phê duyệt.

2.4.2. Ba trụ cột tương tác của MCP

Giao thức MCP tổ chức khả năng tương tác của AI với hệ thống xung quanh ba loại nguyên thủy (Primitives) chính:

- **Resources (Tài nguyên):** Cấp quyền đọc thụ động các tài liệu cấu trúc hóa, chẳng hạn như hướng dẫn quy trình kế toán nội bộ, chính sách phê duyệt hóa đơn hoặc danh mục tài khoản. AI nhận thông tin như một người đọc tài liệu, không thể thực hiện hành động nào.
- **Tools (Công cụ):** Cấp quyền thực thi chủ động, cho phép AI gọi các hàm nghiệp vụ cụ thể như tra cứu số dư sổ cái, lấy danh sách hóa đơn chờ phê duyệt hay kiểm tra

trạng thái thanh toán. Mỗi tool được định nghĩa với schema đầu vào/đầu ra rõ ràng, đảm bảo AI chỉ truyền đúng tham số và nhận lại đúng cấu trúc dữ liệu mong đợi.

- Prompts (Kịch bản): Thiết lập các mẫu hội thoại (Prompt Templates) được cấu hình sẵn, ép buộc AI phản hồi theo đúng vai trò chuyên môn – ví dụ như chuyên gia kế toán phân tích báo cáo – thay vì trả lời theo phong cách trợ lý tổng quát.

2.4.3. Cơ chế xác thực OAuth 2.1 trong MCP Server

EFMS MCP Server được triển khai theo mô hình HTTP + SSE Transport kết hợp luồng xác thực OAuth 2.1 tiêu chuẩn. Theo cơ chế này, mỗi yêu cầu từ Claude Desktop gửi đến endpoint /mcp đều kèm theo header Authorization: Bearer <token>. MCP Server xác minh token bằng cách gọi sang EFMS Identity Service, sau đó khởi tạo một phiên làm việc riêng biệt gắn với companyId của người dùng đó. Mỗi phiên MCP do đó hoàn toàn cô lập về dữ liệu, đảm bảo nguyên tắc Multi-tenancy được duy trì nhất quán từ API xuống đến tầng AI.

2.5. Hệ quản trị Cơ sở dữ liệu PostgreSQL

2.5.1. Lý do lựa chọn PostgreSQL

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) mã nguồn mở, được phát triển và duy trì bởi cộng đồng toàn cầu từ năm 1986. Đây là một trong số ít hệ CSDL thương mại và mã nguồn mở đáp ứng đầy đủ chuẩn ACID (Atomicity, Consistency, Isolation, Durability) – yêu cầu tối thiểu không thể thỏa hiệp đối với bất kỳ nền tảng tài chính kế toán nào.

Bên cạnh sự tuân thủ ACID, PostgreSQL được lựa chọn cho EFMS vì ba lý do kỹ thuật cụ thể: hỗ trợ kiến trúc Multi-Schema phù hợp với mô hình Multi-tenancy đã trình bày; cơ chế kiểm soát đồng thời đa phiên bản (MVCC) đảm bảo hiệu năng cao trong môi trường nhiều người dùng; và kiểu dữ liệu JSONB linh hoạt phục vụ nhu cầu lưu trữ lịch sử thay đổi dữ liệu (Audit Trail).

2.5.2. Cơ chế kiểm soát đồng thời đa phiên bản (MVCC)

Multi-Version Concurrency Control (MVCC) là cơ chế cốt lõi của PostgreSQL cho phép các giao dịch đọc và ghi diễn ra đồng thời mà không cần phải chờ đợi nhau thông qua cơ chế khóa (Locking) truyền thống. Khi một giao dịch bắt đầu, PostgreSQL cung cấp cho nó một "ảnh chụp" (Snapshot) nhất quán của trạng thái dữ liệu tại thời điểm đó. Mọi thay đổi từ các giao dịch khác diễn ra song song sẽ không được nhìn thấy cho đến khi giao dịch hiện tại kết thúc.

Trong ngữ cảnh EFMS, điều này có nghĩa thực tế là: trong khi Kế toán A đang chỉnh sửa một chứng từ hóa đơn (giao dịch ghi), Kế toán B vẫn có thể xem báo cáo tổng hợp tài chính (giao dịch đọc) với dữ liệu nhất quán tại thời điểm truy vấn, không bị gián đoạn hay nhận dữ liệu không hoàn chỉnh. Cơ chế này đảm bảo hệ thống duy trì thông lượng cao kể cả khi có hàng nghìn giao dịch tài chính diễn ra đồng thời.

2.5.3. Kiểu dữ liệu JSONB và Audit Logging

PostgreSQL hỗ trợ kiểu dữ liệu JSONB (Binary JSON) cho phép lưu trữ và truy vấn dữ liệu có cấu trúc JSON với hiệu năng cao. Trong EFMS, JSONB được ứng dụng đặc biệt trong bảng audit_logs, nơi các trường old_data và new_data lưu trữ trạng thái

hoàn chỉnh của bản ghi trước và sau mỗi thao tác thay đổi. Nhờ đó, Kiểm toán viên có thể truy xuất toàn bộ lịch sử thay đổi của bất kỳ chứng từ tài chính nào, so sánh từng trường dữ liệu qua các phiên bản khác nhau, phục vụ yêu cầu kiểm tra và giải trình tài chính.

2.6. Tiêu chuẩn Thiết kế Hệ thống

2.6.1. Chuẩn phản hồi API thống nhất (ApiResponse<T>)

Để đảm bảo tính nhất quán trong giao tiếp giữa Front-end và tất cả các microservice, EFMS áp dụng một cấu trúc phản hồi API chuẩn hóa duy nhất ApiResponse<T>. Mọi endpoint trên tất cả các dịch vụ đều trả về đối tượng bao gồm ba trường bắt buộc: status (trạng thái xử lý), message (thông điệp mô tả kết quả) và data (payload dữ liệu có kiểu tham số T linh hoạt).

Chuẩn hóa phản hồi này được tăng cường bởi một cơ chế xử lý ngoại lệ toàn cục (Global Exception Handler) ở mỗi dịch vụ. Thay vì để các lỗi chưa được xử lý lan ra ngoài với thông tin kỹ thuật nhạy cảm (stack trace, tên lớp, câu truy vấn SQL), Global Exception Handler bắt mọi loại ngoại lệ, chuyển đổi chúng thành phản hồi ApiResponse chuẩn với mã HTTP phù hợp và thông điệp lỗi thân thiện với người dùng, đồng thời ghi log chi tiết vào hệ thống giám sát nội bộ.

2.6.2. Hệ thống lưu vết thao tác (Audit Logging)

Mọi thao tác tạo mới, sửa đổi hoặc xóa dữ liệu tài chính đều được ghi lại vào bảng audit_logs tương ứng của từng dịch vụ. Mỗi bản ghi audit bao gồm: định danh bảng dữ liệu bị tác động (table_name), khóa chính của bản ghi (record_id), loại hành động (action: INSERT/UPDATE/DELETE), thông tin người thực hiện (changed_by), thời điểm thay đổi (changed_at) và quan trọng nhất – trạng thái dữ liệu đầy đủ trước và sau thay đổi được lưu dưới dạng JSONB (old_data, new_data).

Hệ thống Audit Logging này phục vụ hai mục tiêu kép: về mặt kỹ thuật, nó là nền tảng để debug và khôi phục dữ liệu khi có sự cố; về mặt nghiệp vụ, nó cung cấp khả năng truy vết và giải trình đầy đủ cho Kiểm toán viên nội bộ và bên ngoài theo yêu cầu tuân thủ pháp lý.

2.6.3. Quy chuẩn mã nguồn và kiểu dữ liệu

Tiêu chuẩn	Phân tích và Lý do áp dụng
BigDecimal cho tiền tệ	Bắt buộc sử dụng kiểu BigDecimal của Java cho mọi trường lưu giá trị tiền tệ. Kiểu dữ liệu float và double sử dụng biểu diễn nhị phân nội tại, dẫn đến sai số làm tròn (Rounding Error) không thể tránh khỏi trong phép tính thập phân. Ví dụ, 0.1 + 0.2 trong biểu diễn float nhị phân không bằng chính xác 0.3. Với dữ liệu tài chính, sai số dù nhỏ cũng tích lũy qua hàng triệu giao dịch và gây ra mất cân bằng sổ cái.
UUID cho Primary Keys	Mọi khóa chính (Primary Key) và khóa ngoại (Foreign Key) tham chiếu đều sử dụng

	dụng UUID (Universally Unique Identifier). UUID loại bỏ hoàn toàn rủi ro xung đột khóa trong môi trường phân tán – điều không thể tránh khỏi với khóa tự tăng (Auto-increment Integer) khi có nhiều dịch vụ sinh dữ liệu độc lập.
Lombok	Thư viện xử lý annotation tại thời điểm biên dịch, tự động sinh mã mẫu lặp lại (Boilerplate Code) như getter, setter, constructor, equals/hashCode, giúp giảm đáng kể độ dài tệp mã nguồn và tập trung vào logic nghiệp vụ.
Constructor Injection	Cơ chế tiêm phụ thuộc thông qua hàm khởi tạo (Constructor Injection) được ưu tiên so với Field Injection vì đảm bảo tính bất biến (Immutability), dễ kiểm thử đơn vị và làm rõ ràng các phụ thuộc của từng lớp.
MapStruct	Công cụ sinh mã ánh xạ (Mapping Code Generation) tại thời điểm biên dịch, thực hiện chuyển đổi giữa Entity và DTO mà không cần Reflection tại runtime. Điều này đảm bảo hiệu năng cao hơn so với các thư viện như ModelMapper và phát hiện lỗi cấu hình sớm tại bước biên dịch.

2.7. Kiểm thử Phần mềm (Software Testing)

2.7.1. Chiến lược kiểm thử tổng thể

Công tác đảm bảo chất lượng (Quality Assurance – QA) được tích hợp xuyên suốt toàn bộ vòng đời phát triển phần mềm (Software Development Life Cycle – SDLC) thay vì được coi là một giai đoạn độc lập ở cuối chu trình. Quan điểm này – được gọi là "Shift Left Testing" – nhằm phát hiện và khắc phục lỗi càng sớm càng tốt trong quá trình phát triển, khi chi phí sửa chữa còn thấp.

2.7.2. Kiểm thử Hộp đen (Black-box Testing)

Kiểm thử Hộp đen tập trung vào tầng giao diện người dùng (Front-end), đánh giá hành vi của hệ thống từ góc nhìn người dùng cuối mà không xét đến cơ chế hoạt động bên trong. Phương pháp này xây dựng các bộ dữ liệu đầu vào (Test Cases) đại diện cho các kịch bản nghiệp vụ thực tế – bao gồm cả các tình huống biên (Edge Cases) và dữ liệu không hợp lệ – sau đó đối chiếu kết quả thực tế với kết quả mong đợi được định nghĩa theo đặc tả nghiệp vụ.

Trong EFMS, kiểm thử Hộp đen bao phủ các luồng nghiệp vụ hoàn chỉnh như: tạo hóa đơn mua vào từ bản nháp đến trạng thái phê duyệt, ghi nhận thanh toán và

phân bổ vào hóa đơn, kiểm tra tính chính xác của số dư trên giao diện báo cáo. Đây là lớp kiểm thử quan trọng nhất từ góc độ người dùng nghiệp vụ.

2.7.3. Kiểm thử Giao thức Mạng (API Testing)

Kiểm thử API đi sâu vào trực nội bộ Backend, đánh giá từng endpoint RESTful độc lập với giao diện người dùng. Phương pháp này sử dụng các công cụ kiểm thử chuyên biệt (như Postman, Newman, hoặc các framework kiểm thử tích hợp của Spring Boot) để gửi yêu cầu HTTP với các tham số và payload được kiểm soát chặt chẽ.

Các kịch bản kiểm thử API trong EFMS bao gồm: xác minh cơ chế xác thực JWT (token hợp lệ, token hết hạn, token bị giả mạo); kiểm tra phân quyền RBAC (người dùng thiếu quyền nhận mã 403 Forbidden); đo lường thời gian phản hồi (Response Time) dưới điều kiện tải bình thường; và kiểm thử chịu tải (Load Testing) bằng cách mô phỏng hàng nghìn yêu cầu đồng thời để xác định ngưỡng hiệu năng tối đa của hệ thống trước khi đưa vào vận hành thực tế.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Khảo sát hệ thống và đặc tả nghiệp vụ

3.1.1. Khảo sát hệ thống

Mục tiêu của việc khảo sát là nhằm thu thập, phân tích và đánh giá nhu cầu thực tế của người dùng đối với hệ thống EFMS (Enterprise Financial Management System), từ đó làm cơ sở cho việc thiết kế và phát triển hệ thống phù hợp. Cụ thể:

- Xác định nhu cầu quản lý tài chính, kế toán của doanh nghiệp trong môi trường đa công ty (multi-tenant).
- Khảo sát mức độ quan tâm của người dùng đối với các tính năng như: quản lý hóa đơn, thanh toán, bút toán, báo cáo tài chính và phân quyền theo vai trò.
- Xác định những khó khăn, hạn chế mà người dùng gặp phải khi sử dụng các hệ thống quản lý tài chính hiện có.
- Thu thập ý kiến người dùng nhằm đề xuất các chức năng cần thiết và cải tiến giao diện, trải nghiệm người dùng (UI/UX).

3.1.2. Đặc tả nghiệp vụ

Hệ thống EFMS được xây dựng theo kiến trúc microservices, bao gồm các dịch vụ chính: Identity Service (quản lý xác thực và phân quyền), Core Service (quản lý nghiệp vụ tài chính kế toán) và Common Service (quản lý tài liệu và bình luận dùng chung). Đồ án tập trung vào module Identity Service – nền tảng xác thực, phân quyền và quản lý tổ chức đa công ty cho toàn bộ hệ thống.

Identity Service chịu trách nhiệm xác thực người dùng thông qua JWT, quản lý cấu trúc đa công ty (multi-tenant), phân quyền dựa trên vai trò (RBAC) và quản lý hồ sơ người dùng. Toàn bộ yêu cầu từ client đều được định tuyến qua API Gateway (cổng 8080) trước khi chuyển tiếp đến các dịch vụ con tương ứng.

3.2. Phân tích hệ thống

3.2.1. Xác định tác nhân hệ thống

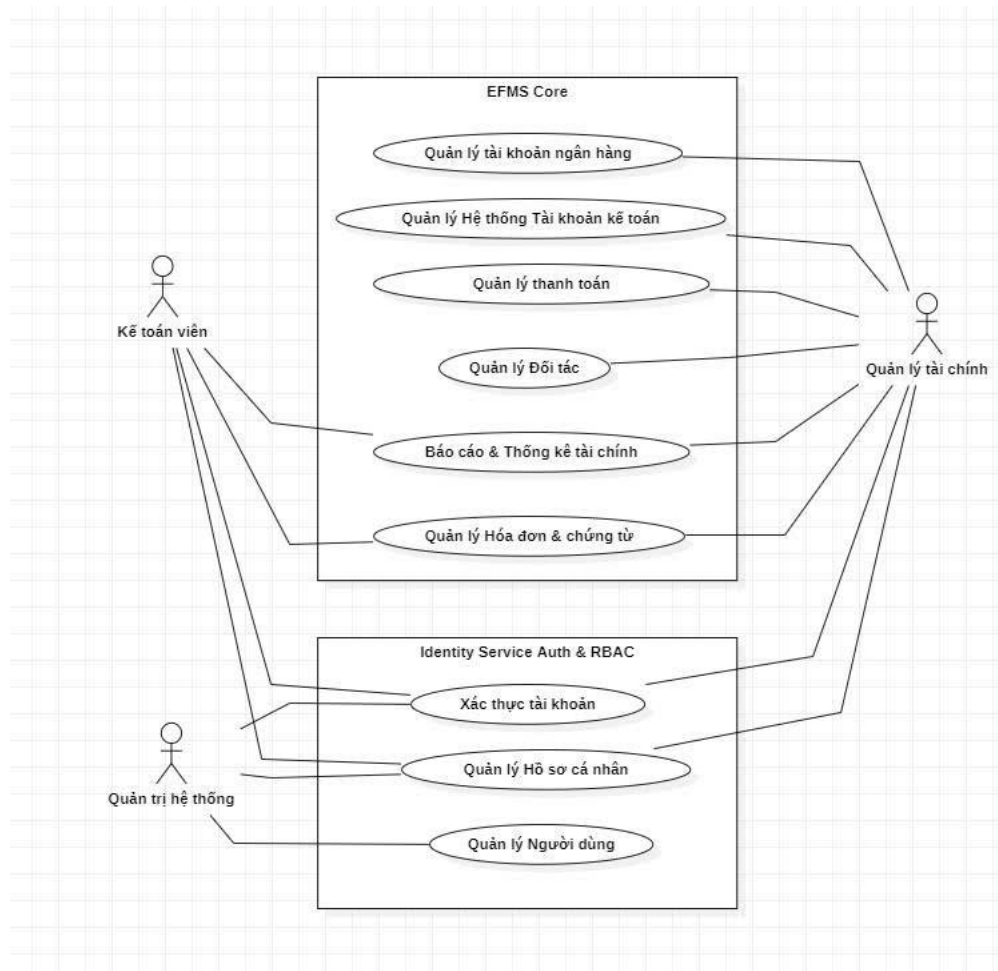
Hệ thống EFMS Identity Service bao gồm các tác nhân chính tham gia vào các nghiệp vụ:

STT	Tác nhân	Mô tả vai trò
2	Kế toán viên	Nhân viên kế toán thực hiện các nghiệp vụ tài chính: hóa đơn, thanh toán, bút toán, đối tác. Được cấp quyền theo vai trò Accountant.
3	Quản lý tài chính	Cấp quản lý có quyền phê duyệt, xem báo cáo tài chính tổng hợp, quản lý tài khoản

		ngân hàng và hệ thống tài khoản kế toán.
4	Quản trị hệ thống	Người dùng có quyền cao nhất: quản lý công ty, phân quyền vai trò, thêm người dùng, cấu hình hệ thống RBAC toàn hệ thống.

3.2.2. Biểu đồ Usecase

3.2.2.1. Usecase tổng quát



Biểu đồ usecase tổng quát thể hiện toàn bộ các chức năng của hệ thống EFMS được tổ chức thành ba module chính tương ứng với ba microservice độc lập:

- EFMS Common: Cung cấp các chức năng dùng chung cho toàn hệ thống, bao gồm Quản lý tệp đính kèm và Quản lý Bình luận. Cả Kế toán viên và Quản lý tài chính đều có quyền sử dụng các chức năng này để bổ sung tài liệu và trao đổi nội bộ trên các thực thể nghiệp vụ.
- EFMS Core: Xử lý toàn bộ nghiệp vụ tài chính kế toán, bao gồm: Quản lý tài khoản ngân hàng, Quản lý Hệ thống Tài khoản kế toán, Quản lý thanh toán, Quản lý Đối tác, Báo cáo & Thống kê tài chính, và Quản lý Hóa đơn & chứng

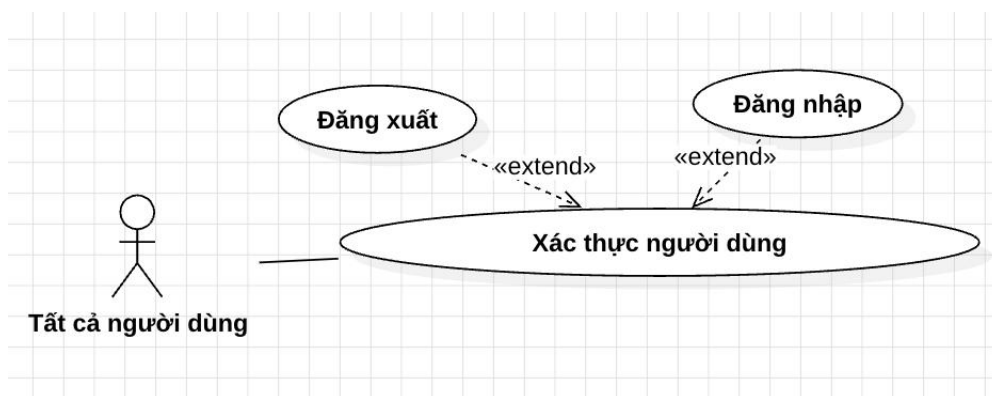
từ. Kế toán viên thực hiện các nghiệp vụ hàng ngày; Quản lý tài chính có quyền xem báo cáo và phê duyệt.

- Identity Service Auth & RBAC: Quản lý xác thực, phân quyền và cấu trúc tổ chức, bao gồm: Xác thực người dùng, Quản lý Hồ sơ người dùng, Phân quyền vai trò hệ thống và Quản lý Công ty. Tất cả người dùng đều tương tác với chức năng Xác thực người dùng; Quản trị hệ thống có toàn quyền trên module này.

Hệ thống có ba tác nhân chính: Kế toán viên (thực hiện nghiệp vụ tài chính và sử dụng common services), Quản lý tài chính (phê duyệt, xem báo cáo và quản lý tài khoản), và Quản trị hệ thống (quản lý người dùng, vai trò và cấu hình hệ thống). Mọi yêu cầu đều được định tuyến qua API Gateway và phải qua bước xác thực JWT trước khi đến các service tương ứng.

3.2.2.2. Đặc tả Usecase module Quản lý xác thực, phân quyền (EFMS Uidentity Service)

1. Usecase xác thực người dùng:



Biểu đồ usecase Xác thực người dùng (hình ảnh đính kèm) thể hiện tác nhân 'Tất cả người dùng' tương tác với usecase trung tâm 'Xác thực người dùng'. Hai usecase mở rộng là 'Đăng nhập' và 'Đăng xuất' với quan hệ <<extend>>.

Bảng 3.1. Đặc tả Usecase Xác thực người dùng – Đăng nhập

Đặc tả Usecase: UC-AUTH-01 – Đăng nhập	
Tên Usecase	Đăng nhập (Login)
Mã Usecase	UC-AUTH-01
Tác nhân	Tất cả người dùng
Mô tả	Người dùng nhập email và mật khẩu để xác thực danh tính. Hệ thống kiểm tra thông tin, cấp phát JWT token và cho phép truy cập vào các chức năng hệ thống theo phân quyền vai trò.

Đặc tả Usecase: UC-AUTH-01 – Đăng nhập	
Điều kiện tiên quyết	Người dùng chưa đăng nhập. Tài khoản đã được tạo và đang ở trạng thái hoạt động (is_active = true).
Điều kiện sau	Người dùng nhận được JWT access token hợp lệ. Phiên làm việc được khởi tạo. API Gateway lưu token để xác thực các yêu cầu tiếp theo.
Luồng sự kiện chính	1. Người dùng truy cập endpoint POST /api/identity/auth/login. 2. Người dùng nhập email và mật khẩu. 3. Hệ thống kiểm tra email tồn tại trong cơ sở dữ liệu. 4. Hệ thống so sánh mật khẩu sau khi mã hóa (BCrypt). 5. Hệ thống kiểm tra trạng thái tài khoản (is_active). 6. Hệ thống sinh JWT token với claims: userId, email, companyId, permissions. 7. Hệ thống trả về JWT token cho client.
Luồng thay thế	3a. Email không tồn tại → Trả về lỗi 'Tài khoản không tồn tại'. 4a. Mật khẩu không khớp → Trả về lỗi 'Sai mật khẩu'. 5a. Tài khoản bị khóa → Trả về lỗi 'Tài khoản đã bị vô hiệu hóa'.
Ngoại lệ	Lỗi kết nối cơ sở dữ liệu → Trả về HTTP 500. Dữ liệu đầu vào thiếu trường bắt buộc → Trả về HTTP 400 với thông báo lỗi validation.

Bảng 3.2. Đặc tả Usecase Xác thực người dùng – Đăng xuất

Đặc tả Usecase: UC-AUTH-02 – Đăng xuất	
Tên Usecase	Đăng xuất (Logout)
Mã Usecase	UC-AUTH-02
Tác nhân	Tất cả người dùng
Mô tả	Người dùng thực hiện đăng xuất khỏi hệ thống, kết thúc phiên làm việc hiện tại. JWT token phía client được xóa.
Điều kiện tiên quyết	Người dùng đã đăng nhập thành công và đang có phiên

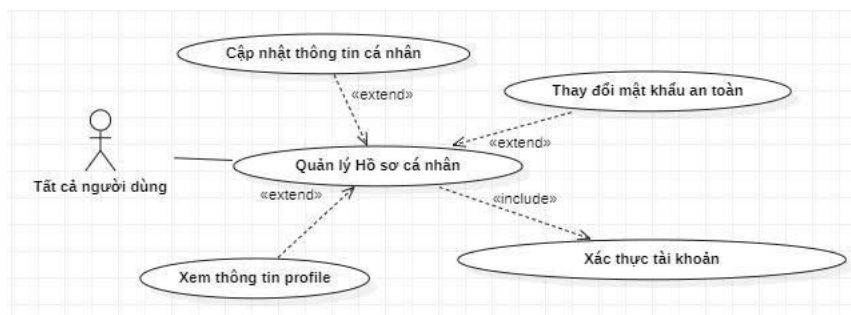
Đặc tả Usecase: UC-AUTH-02 – Đăng xuất	
	làm việc hợp lệ.
Điều kiện sau	Phiên làm việc kết thúc. JWT token bị vô hiệu hóa phía client. Người dùng được chuyển về trang đăng nhập.
Luồng sự kiện chính	1. Người dùng chọn chức năng Đăng xuất. 2. Client xóa JWT token khỏi bộ nhớ (localStorage/sessionStorage). 3. Hệ thống trả về HTTP 200 xác nhận đăng xuất thành công. 4. Người dùng được điều hướng về trang đăng nhập.
Luồng thay thế	2a. Token đã hết hạn trước khi người dùng đăng xuất → Hệ thống vẫn trả về thành công và xóa token phía client.
Ngoại lệ	Lỗi mạng khi gửi yêu cầu → Client vẫn xóa token cục bộ và chuyển về trang đăng nhập.

Bảng 3.3. Luồng xử lý xác thực qua API Gateway

Bước	Thành phần	Hành động	Kết quả
1	Client	Gửi POST /api/identity/auth/login kèm {email, password}	Yêu cầu đến API Gateway
2	API Gateway	Kiểm tra endpoint nằm trong danh sách whitelist (không cần JWT)	Cho phép đi thẳng đến Identity Service
3	Identity Service	Xác thực thông tin đăng nhập, tạo JWT với claims đầy đủ	JWT token được tạo
4	Identity Service	Trả về JWT token trong ApiResponse<T>	Client nhận token
5	Client	Gửi yêu cầu kèm JWT trong header Authorization:	API Gateway nhận và validate

Bước	Thành phần	Hành động	Kết quả
		Bearer {token}	
6	API Gateway	Giải mã JWT, inject headers: X-User-Id, X-User-Email, X-User-Company-Id, X-User-Permission	Downstream service nhận headers
7	Identity Service	GatewayHeaderFilter đọc headers, populate SecurityContextHolder	RBAC kiểm tra @PreAuthorize

2. Usecase Quản lý Hồ sơ cá nhân



Biểu đồ usecase Quản lý Hồ sơ người dùng (hình ảnh đính kèm) thể hiện tác nhân 'Tất cả người dùng' tương tác với usecase trung tâm 'Quản lý Hồ sơ người dùng'. Ba usecase mở rộng gồm: 'Cập nhật thông tin cá nhân', 'Thay đổi mật khẩu an toàn' và 'Xem thông tin profile' với quan hệ <<extend>>. Usecase 'Xác thực người dùng' là điều kiện bắt buộc (<<include>>).

Bảng 3.4. Đặc tả Usecase Cập nhật thông tin cá nhân

Đặc tả Usecase: UC-PROFILE-01 – Cập nhật thông tin cá nhân	
Tên Usecase	Cập nhật thông tin cá nhân
Mã Usecase	UC-PROFILE-01
Tác nhân	Tất cả người dùng (đã đăng nhập)
Mô tả	Người dùng cập nhật các thông tin hồ sơ cá nhân như tên hiển thị, thông tin liên hệ. Thông tin được lưu vào bảng

Đặc tả Usecase: UC-PROFILE-01 – Cập nhật thông tin cá nhân	
	users trong cơ sở dữ liệu Identity Service.
Điều kiện tiên quyết	Người dùng đã đăng nhập thành công. JWT token hợp lệ được đính kèm trong header Authorization.
Điều kiện sau	Thông tin cá nhân được cập nhật trong bảng users. Audit log ghi nhận thay đổi với old_data và new_data dạng JSONB.
Luồng sự kiện chính	1. Người dùng gửi PUT /api/identity/v1/users/{id} kèm dữ liệu cập nhật. 2. API Gateway validate JWT và inject headers người dùng. 3. Identity Service kiểm tra X-User-Id khớp với {id} trong path hoặc có quyền admin. 4. Hệ thống validate dữ liệu đầu vào (tên không rỗng, định dạng hợp lệ). 5. Hệ thống cập nhật bản ghi trong bảng users. 6. Hệ thống ghi audit log với thông tin thay đổi. 7. Hệ thống trả về ApiResponse với dữ liệu người dùng đã cập nhật.
Luồng thay thế	3a. Người dùng cố truy cập hồ sơ người khác mà không có quyền → HTTP 403 Forbidden. 4a. Dữ liệu không hợp lệ (tên rỗng) → HTTP 400 với thông báo lỗi cụ thể.
Ngoại lệ	Lỗi cơ sở dữ liệu → HTTP 500. JWT hết hạn → HTTP 401, yêu cầu đăng nhập lại.

Bảng 3.5. Đặc tả Usecase Thay đổi mật khẩu an toàn

Đặc tả Usecase: UC-PROFILE-02 – Thay đổi mật khẩu an toàn	
Tên Usecase	Thay đổi mật khẩu an toàn
Mã Usecase	UC-PROFILE-02
Tác nhân	Tất cả người dùng (đã đăng nhập)

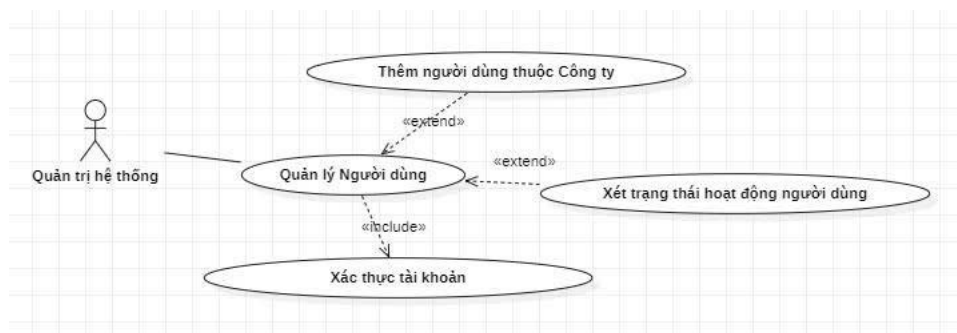
Đặc tả Usecase: UC-PROFILE-02 – Thay đổi mật khẩu an toàn	
Mô tả	Người dùng thay đổi mật khẩu của tài khoản. Hệ thống yêu cầu nhập mật khẩu cũ để xác nhận danh tính trước khi cho phép cập nhật mật khẩu mới.
Điều kiện tiên quyết	Người dùng đã đăng nhập. Người dùng biết mật khẩu hiện tại.
Điều kiện sau	Mật khẩu mới được mã hóa (BCrypt) và lưu vào cơ sở dữ liệu. Audit log ghi nhận sự kiện thay đổi mật khẩu.
Luồng sự kiện chính	1. Người dùng truy cập chức năng Đổi mật khẩu và gửi PUT /api/identity/v1/users/{id}/password. 2. Người dùng nhập: mật khẩu cũ (oldPassword), mật khẩu mới (newPassword), xác nhận mật khẩu mới. 3. Hệ thống kiểm tra oldPassword khớp với hash trong CSDL. 4. Hệ thống kiểm tra newPassword và confirmPassword trùng nhau. 5. Hệ thống kiểm tra newPassword đáp ứng chính sách bảo mật (tối thiểu 8 ký tự, có chữ hoa, số). 6. Hệ thống mã hóa newPassword bằng BCrypt và cập nhật vào CSDL. 7. Hệ thống ghi audit log. 8. Hệ thống trả về thông báo thành công.
Luồng thay thế	3a. oldPassword sai → HTTP 400 'Mật khẩu cũ không đúng'. 4a. Mật khẩu xác nhận không khớp → HTTP 400. 5a. Mật khẩu mới không đủ độ phức tạp → HTTP 400 với hướng dẫn cụ thể.
Ngoại lệ	Lỗi kết nối CSDL → HTTP 500. JWT hết hạn → HTTP 401.

Bảng 3.6. Đặc tả Usecase Xem thông tin profile

Đặc tả Usecase: UC-PROFILE-03 – Xem thông tin profile	
Tên Usecase	Xem thông tin profile
Mã Usecase	UC-PROFILE-03

Đặc tả Usecase: UC-PROFILE-03 – Xem thông tin profile	
Tác nhân	Tất cả người dùng (đã đăng nhập)
Mô tả	Người dùng xem thông tin hồ sơ cá nhân của bản thân bao gồm: tên, email, vai trò, công ty và trạng thái tài khoản.
Điều kiện tiên quyết	Người dùng đã đăng nhập với JWT hợp lệ.
Điều kiện sau	Thông tin hồ sơ được hiển thị (chỉ đọc). Không có thay đổi nào đối với CSDL.
Luồng sự kiện chính	1. Người dùng gửi GET /api/identity/v1/users/{id}. 2. API Gateway validate JWT, inject X-User-Id. 3. Identity Service kiểm tra quyền truy cập. 4. Hệ thống truy vấn thông tin người dùng kèm thông tin vai trò và công ty. 5. Hệ thống trả về ApiResponse với dữ liệu hồ sơ đầy đủ.
Luồng thay thế	3a. Người dùng xem hồ sơ người khác mà không có quyền → HTTP 403.
Ngoại lệ	Người dùng không tồn tại → HTTP 404. JWT không hợp lệ → HTTP 401.

3. Usecase Quản lý người dùng



Biểu đồ usecase Quản lý Người dùng (hình ảnh đính kèm) thể hiện tác nhân 'Quản trị hệ thống' tương tác với usecase 'Quản lý Người dùng'. Hai usecase mở rộng là 'Thêm người dùng thuộc Công ty' và 'Xét trạng thái hoạt động người dùng' là điều kiện bắt buộc (<<include>>).

Bảng 3.12. Đặc tả Usecase Thêm người dùng thuộc Công ty

Đặc tả Usecase: UC-COMPANY-02 – Thêm người dùng thuộc Công ty	
Tên Usecase	Thêm người dùng thuộc Công ty
Mã Usecase	UC-COMPANY-02
Tác nhân	Quản trị hệ thống
Mô tả	Quản trị viên tạo tài khoản người dùng mới và liên kết với một công ty cụ thể. Người dùng được cấp vai trò ngay khi tạo. Đây là luồng đăng ký tài khoản qua API /auth/register.
Điều kiện tiên quyết	Công ty mục tiêu đã tồn tại và đang hoạt động. Vai trò cần gán đã được tạo.
Điều kiện sau	Tài khoản người dùng mới được tạo trong bảng users với company_id và role_id tương ứng. Mật khẩu được mã hóa BCrypt. Audit log ghi nhận việc tạo tài khoản.
Luồng sự kiện chính	1. Người dùng/quản trị viên gửi POST /api/identity/auth/register với {name, email, password, companyId, roleId}. 2. Hệ thống kiểm tra email chưa được đăng ký. 3. Hệ thống kiểm tra companyId và roleId tồn tại và hợp lệ. 4. Hệ thống mã hóa mật khẩu bằng BCrypt. 5. Hệ thống tạo bản ghi người dùng mới. 6. Hệ thống ghi audit log. 7. Hệ thống trả về thông tin tài khoản đã tạo.
Luồng thay thế	2a. Email đã tồn tại → HTTP 409. 3a. companyId không hợp lệ → HTTP 404. 3b. roleId không hợp lệ → HTTP 404.
Ngoại lệ	Lỗi mã hóa mật khẩu → HTTP 500.

Bảng 3.13. Đặc tả Usecase Xét trạng thái hoạt động người dùng

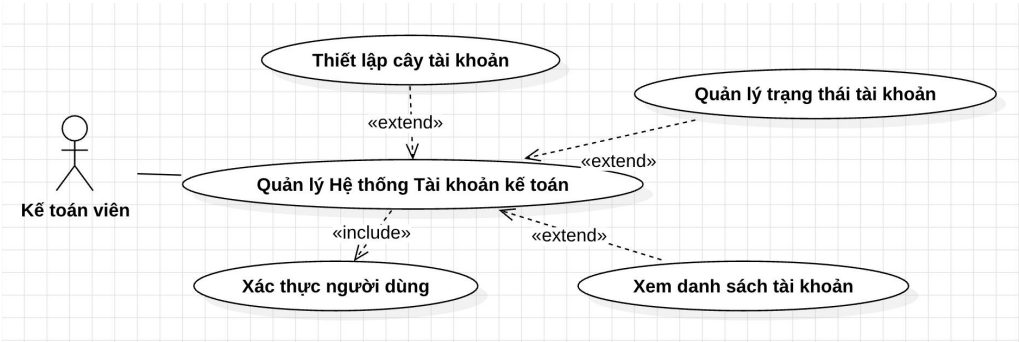
Đặc tả Usecase: UC-COMPANY-03 – Tạo mới Công ty Chi nhánh	
Tên Usecase	Xét trạng thái hoạt động người dùng
Mã Usecase	UC-COMPANY-03

Đặc tả Usecase: UC-COMPANY-03 – Tạo mới Công ty Chi nhánh	
Tác nhân	Quản trị hệ thống
Mô tả	Quản trị viên có thể sửa cập nhật trạng thái hoạt động của người dùng trong hệ thống công ty.
Điều kiện tiên quyết	Quản trị viên có quyền users:update. id người dùng, thông tin trạng thái.
Điều kiện sau	Người dùng đã có bản ghi trong database.
Luồng sự kiện chính	1. Quản trị viên gửi POST /api/identity/v1/users với thông tin người dùng. 2. Hệ thống kiểm tra xem có người dùng tồn tại không. 3. Hệ thống cập nhật lại trạng thái hoạt động của người dùng. 4. Hệ thống ghi audit log. 5. Hệ thống trả về trạng thái mới của người dùng.
Luồng thay thế	2a. Người dùng không tồn tại → HTTP 404.
Ngoại lệ	Không có quyền → HTTP 403. Lỗi CSDL → HTTP 500.

3.2.2.3. Đặc tả Usecase module tài chính kế toán (EFMS Core Service)

EFMS Core Service là dịch vụ xử lý toàn bộ nghiệp vụ tài chính kế toán của hệ thống. Module này bao gồm các nhóm chức năng: Quản lý Hệ thống Tài khoản kế toán, Quản lý thanh toán, Quản lý Hóa đơn & chứng từ, Quản lý Đối tác, Báo cáo & Thống kê tài chính, và Quản lý Tài khoản ngân hàng. Mọi dữ liệu đều được cô lập theo company_id, đảm bảo tính đa người thuê (multi-tenant).

1. Usecase Quản lý Hệ thống Tài khoản kế toán



Biểu đồ usecase Quản lý Hệ thống Tài khoản kế toán thể hiện tác nhân 'Kế toán viên' tương tác với usecase trung tâm. Các usecase mở rộng gồm: 'Thiết lập cây tài khoản', 'Quản lý trạng thái tài khoản' và 'Xem danh sách tài khoản' với quan hệ <<extend>>. Usecase 'Xác thực người dùng' là điều kiện bắt buộc (<<include>>).

Bảng 3.15. Đặc tả Usecase Quản lý Hệ thống Tài khoản kế toán

Đặc tả Usecase: UC-ACCT-01 – Quản lý Hệ thống Tài khoản kế toán	
Tên Usecase	Quản lý Hệ thống Tài khoản kế toán
Mã Usecase	UC-ACCT-01
Tác nhân	Kế toán viên
Mô tả	Kế toán viên quản lý hệ thống tài khoản kế toán (Chart of Accounts) của công ty. Hỗ trợ thiết lập cây tài khoản phân cấp, quản lý trạng thái và xem danh sách tài khoản theo tiêu chí lọc.
Điều kiện tiên quyết	Kế toán viên đã đăng nhập với quyền account:manage. Dữ liệu company_id hợp lệ được inject từ API Gateway.
Điều kiện sau	Thông tin tài khoản được lưu vào bảng accounts với các trường: id (UUID), company_id, code, name, account_type, parent_id, is_active. Audit log ghi nhận thay đổi.
Luồng sự kiện chính	1. Kế toán viên truy cập module Hệ thống Tài khoản kế toán. 2. Hệ thống hiển thị danh sách tài khoản theo cấu trúc cây phân cấp. 3. Kế toán viên thực hiện một trong các hành động: thêm mới, chỉnh sửa, thay đổi trạng thái hoặc xem danh sách. 4. Hệ thống kiểm tra tính hợp lệ (mã tài khoản không trùng, parent_id tồn tại). 5. Hệ thống cập nhật CSDL và ghi audit log. 6. Hệ thống trả về kết quả theo cấu trúc ApiResponse<T>.
Luồng thay thế	4a. Mã tài khoản đã tồn tại → HTTP 409 Conflict. 4b. parent_id không hợp lệ → HTTP 400. 3a. Không có tài khoản nào → Hệ thống hiển thị danh sách rỗng.
Ngoại lệ	Không có quyền account:manage → HTTP 403. Lỗi CSDL → HTTP 500.

Bảng 3.16. Đặc tả Usecase Thiết lập cây tài khoản

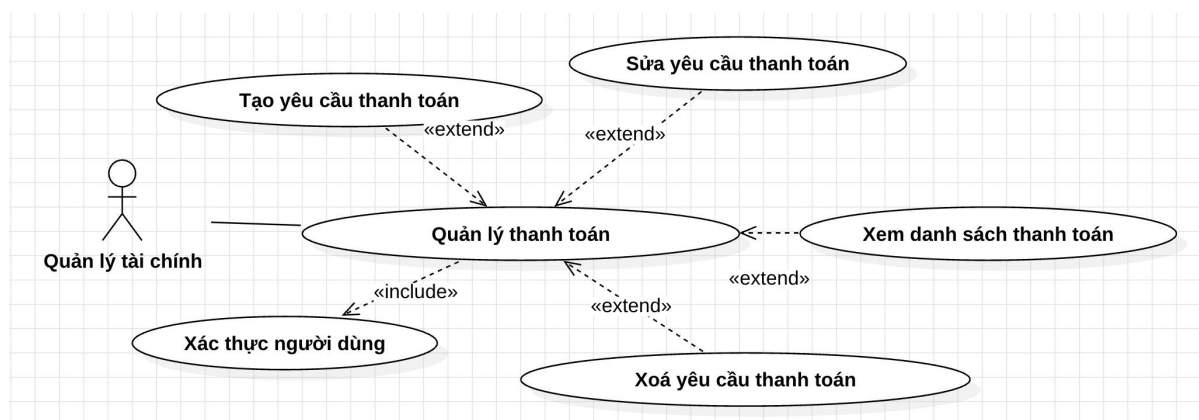
Đặc tả Usecase: UC-ACCT-02 – Thiết lập cây tài khoản	
Tên Usecase	Thiết lập cây tài khoản
Mã Usecase	UC-ACCT-02
Tác nhân	Kế toán viên
Mô tả	Kế toán viên tạo mới và tổ chức hệ thống tài khoản kế toán theo cấu trúc phân cấp cha-con (parent-child). Mỗi tài khoản có mã (code), tên, loại tài khoản và có thể liên kết với tài khoản cha.
Điều kiện tiên quyết	Kế toán viên có quyền account:create. Tài khoản cha (nếu có) đã tồn tại trong hệ thống.
Điều kiện sau	Tài khoản mới được tạo với parent_id trỏ đến tài khoản cha. Cấu trúc cây tài khoản được cập nhật và hiển thị đúng thứ tự phân cấp.
Luồng sự kiện chính	1. Kế toán viên gửi POST /api/core/v1/accounts với {code, name, accountType, parentId}. 2. Hệ thống kiểm tra mã tài khoản chưa tồn tại trong công ty. 3. Hệ thống kiểm tra parentId hợp lệ (nếu có). 4. Hệ thống tạo bản ghi tài khoản mới. 5. Hệ thống ghi audit log. 6. Hệ thống trả về thông tin tài khoản vừa tạo.
Luồng thay thế	2a. Mã tài khoản đã tồn tại → HTTP 409. 3a. parentId không tồn tại → HTTP 404.
Ngoại lệ	Không có quyền → HTTP 403. Lỗi CSDL → HTTP 500.

Bảng 3.17. Đặc tả Usecase Quản lý trạng thái tài khoản

Đặc tả Usecase: UC-ACCT-03 – Quản lý trạng thái tài khoản	
Tên Usecase	Quản lý trạng thái tài khoản
Mã Usecase	UC-ACCT-03

Đặc tả Usecase: UC-ACCT-03 – Quản lý trạng thái tài khoản	
Tác nhân	Kế toán viên
Mô tả	Kế toán viên kích hoạt hoặc vô hiệu hóa tài khoản kế toán. Tài khoản bị vô hiệu hóa (is_active = false) sẽ không thể sử dụng trong các bút toán mới.
Điều kiện tiên quyết	Tài khoản cần thay đổi trạng thái đã tồn tại. Kế toán viên có quyền account:update.
Điều kiện sau	Trường is_active của tài khoản được cập nhật. Audit log ghi nhận thay đổi trạng thái.
Luồng sự kiện chính	1. Kế toán viên gửi PUT /api/core/v1/accounts/{id} với {isActive: false/true}. 2. Hệ thống kiểm tra tài khoản tồn tại và thuộc company_id hiện tại. 3. Hệ thống kiểm tra không có bút toán đang chờ xử lý liên kết với tài khoản. 4. Hệ thống cập nhật trạng thái. 5. Hệ thống ghi audit log và trả về kết quả.
Luồng thay thế	3a. Tài khoản đang được dùng trong bút toán chưa hoàn thành → HTTP 400 với thông báo cảnh báo.
Ngoại lệ	Tài khoản không tồn tại → HTTP 404. Không có quyền → HTTP 403.

2. Usecase Quản lý thanh toán



Biểu đồ usecase Quản lý thanh toán thể hiện tác nhân 'Quản lý tài chính' tương tác với usecase trung tâm. Các usecase mở rộng gồm: 'Tạo yêu cầu thanh toán', 'Sửa yêu cầu thanh toán', 'Xem danh sách thanh toán' và 'Xoá yêu cầu thanh toán'. Usecase 'Xác thực người dùng' là điều kiện bắt buộc (<<include>>).

Bảng 3.18. Đặc tả Usecase Quản lý thanh toán

Đặc tả Usecase: UC-PAY-01 – Quản lý thanh toán	
Tên Usecase	Quản lý thanh toán
Mã Usecase	UC-PAY-01
Tác nhân	Quản lý tài chính
Mô tả	Quản lý tài chính quản lý toàn bộ vòng đời của các khoản thanh toán trong hệ thống, bao gồm tạo mới, chỉnh sửa, xem danh sách và xoá yêu cầu thanh toán. Thanh toán có thể liên kết với hóa đơn qua bảng invoice_payments.
Điều kiện tiên quyết	Quản lý tài chính đã đăng nhập với quyền payment:manage. Kỳ kế toán (fiscal_period) liên quan đang ở trạng thái mở (open).
Điều kiện sau	Bản ghi thanh toán được lưu vào bảng payments với các trường: id (UUID), company_id, partner_id, amount (NUMERIC 18,2), payment_date, status, bank_account_id. Audit log ghi nhận thay đổi.
Luồng sự kiện chính	1. Quản lý tài chính truy cập module Quản lý thanh toán. 2. Hệ thống hiển thị danh sách thanh toán với bộ lọc (trạng thái, ngày, đối tác). 3. Người dùng chọn hành động cần thực hiện. 4. Hệ thống kiểm tra tính hợp lệ và thực thi hành động tương ứng. 5. Hệ thống ghi audit log và trả về kết quả.
Luồng thay thế	4a. Kỳ kế toán đóng → HTTP 400 'Fiscal period is closed'. 4b. Thanh toán đã được xác nhận (posted) → Không thể sửa/xóa, trả về HTTP 400.
Ngoại lệ	Không có quyền payment:manage → HTTP 403. Lỗi CSDL → HTTP 500.

Bảng 3.19. Đặc tả Usecase Tạo yêu cầu thanh toán

Đặc tả Usecase: UC-PAY-02 – Tạo yêu cầu thanh toán	
Tên Usecase	Tạo yêu cầu thanh toán
Mã Usecase	UC-PAY-02
Tác nhân	Quản lý tài chính
Mô tả	Quản lý tài chính tạo mới một khoản thanh toán, chỉ định đối tác, tài khoản ngân hàng, số tiền và ngày thanh toán. Thanh toán được tạo ở trạng thái draft trước khi được xác nhận (post).
Điều kiện tiên quyết	Đối tác (partner) tồn tại trong hệ thống. Tài khoản ngân hàng hợp lệ. Kỳ kế toán đang mở.
Điều kiện sau	Bản ghi thanh toán được tạo với status = 'draft'. Có thể được xác nhận (POST /v1/payments/{id}/post) để sinh bút toán kế toán.
Luồng sự kiện chính	1. Quản lý tài chính gửi POST /api/core/v1/payments với {partnerId, amount, paymentDate, bankAccountId, note}. 2. Hệ thống kiểm tra fiscal period đang mở. 3. Hệ thống kiểm tra partnerId và bankAccountId hợp lệ. 4. Hệ thống tạo bản ghi thanh toán với status = 'draft'. 5. Hệ thống ghi audit log. 6. Hệ thống trả về thông tin thanh toán đã tạo.
Luồng thay thế	2a. Kỳ kế toán đóng → HTTP 400. 3a. partnerId không tồn tại → HTTP 404.
Ngoại lệ	amount <= 0 → HTTP 400 validation error. Lỗi CSDL → HTTP 500.

Bảng 3.20. Đặc tả Usecase Sửa yêu cầu thanh toán

Đặc tả Usecase: UC-PAY-03 – Sửa yêu cầu thanh toán	
Tên Usecase	Sửa yêu cầu thanh toán
Mã Usecase	UC-PAY-03

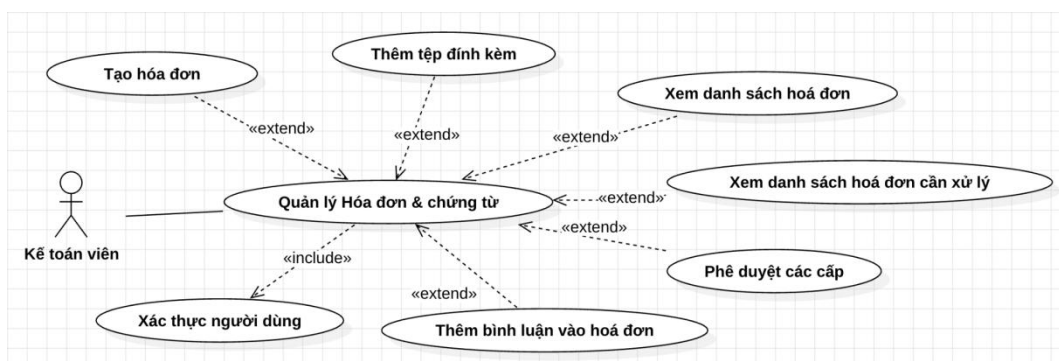
Đặc tả Usecase: UC-PAY-03 – Sửa yêu cầu thanh toán	
Tác nhân	Quản lý tài chính
Mô tả	Quản lý tài chính chỉnh sửa thông tin của một thanh toán đang ở trạng thái draft. Chỉ các trường như số tiền, ngày thanh toán, ghi chú có thể được chỉnh sửa.
Điều kiện tiên quyết	Thanh toán tồn tại và đang ở trạng thái 'draft'. Kỳ kế toán liên quan đang mở.
Điều kiện sau	Thông tin thanh toán được cập nhật. Audit log ghi nhận dữ liệu cũ và mới (old_data, new_data dạng JSONB).
Luồng sự kiện chính	1. Quản lý tài chính gửi PUT /api/core/v1/payments/{id} với dữ liệu cập nhật. 2. Hệ thống kiểm tra thanh toán tồn tại và status = 'draft'. 3. Hệ thống kiểm tra dữ liệu hợp lệ. 4. Hệ thống cập nhật bản ghi. 5. Hệ thống ghi audit log và trả về kết quả.
Luồng thay thế	2a. Thanh toán đã ở trạng thái 'posted' hoặc 'cancelled' → HTTP 400 'Cannot edit non-draft payment'.
Ngoại lệ	Thanh toán không tồn tại → HTTP 404. Không có quyền → HTTP 403.

Bảng 3.21. Đặc tả Usecase Xóa yêu cầu thanh toán

Đặc tả Usecase: UC-PAY-04 – Xóa yêu cầu thanh toán	
Tên Usecase	Xóa yêu cầu thanh toán
Mã Usecase	UC-PAY-04
Tác nhân	Quản lý tài chính
Mô tả	Quản lý tài chính xóa một thanh toán đang ở trạng thái draft khỏi hệ thống. Chỉ cho phép xóa khi thanh toán chưa được xác nhận.
Điều kiện tiên quyết	Thanh toán tồn tại, thuộc company_id hiện tại và status = 'draft'.
Điều kiện sau	Bản ghi thanh toán bị xóa khỏi CSDL. Audit log ghi

Đặc tả Usecase: UC-PAY-04 – Xoá yêu cầu thanh toán	
	nhận hành động xoá.
Luồng sự kiện chính	1. Quản lý tài chính gửi DELETE /api/core/v1/payments/{id}. 2. Hệ thống kiểm tra thanh toán tồn tại và status = 'draft'. 3. Hệ thống kiểm tra không có liên kết với invoice_payments. 4. Hệ thống xoá bản ghi và ghi audit log. 5. Hệ thống trả về HTTP 200 xác nhận thành công.
Luồng thay thế	2a. status != 'draft' → HTTP 400 'Cannot delete posted/cancelled payment'. 3a. Thanh toán đã liên kết với hóa đơn → HTTP 400, yêu cầu hủy liên kết trước.
Ngoại lệ	Không tồn tại → HTTP 404. Không có quyền → HTTP 403.

3. Usecase Quản lý Hóa đơn & Chứng từ



Biểu đồ usecase Quản lý Hóa đơn & Chứng từ thể hiện tác nhân 'Kế toán viên' tương tác với usecase trung tâm. Các usecase mở rộng gồm: 'Tạo hóa đơn', 'Thêm tệp đính kèm', 'Xem danh sách hoá đơn', 'Xem danh sách hoá đơn cần xử lý', 'Phê duyệt các cấp' và 'Thêm bình luận vào hoá đơn'. Usecase 'Xác thực người dùng' là điều kiện bắt buộc (<<include>>).

Bảng 3.22. Đặc tả Usecase Quản lý Hóa đơn & Chứng từ

Đặc tả Usecase: UC-INV-01 – Quản lý Hóa đơn & Chứng từ	
Tên Usecase	Quản lý Hóa đơn & Chứng từ

Đặc tả Usecase: UC-INV-01 – Quản lý Hóa đơn & Chứng từ	
Mã Usecase	UC-INV-01
Tác nhân	Kế toán viên
Mô tả	Kế toán viên quản lý toàn bộ vòng đời hóa đơn bán hàng (AR) và hóa đơn mua hàng (AP), từ tạo mới (draft), xác nhận (posted), phê duyệt nhiều cấp đến hoàn tất. Hỗ trợ đính kèm tệp và bình luận nội bộ trên từng hóa đơn.
Điều kiện tiên quyết	Kế toán viên đã đăng nhập với quyền invoice:manage. Kỳ kế toán (fiscal_period) liên quan đang mở. Đối tác (partner) đã tồn tại.
Điều kiện sau	Hóa đơn được lưu vào bảng invoices với các trường: id (UUID), company_id, invoice_type (AR/AP), partner_id, invoice_date, due_date, total_amount (NUMERIC 18,2), status. Các dòng hóa đơn lưu vào invoice_lines.
Luồng sự kiện chính	1. Kế toán viên truy cập module Hóa đơn & Chứng từ. 2. Hệ thống hiển thị danh sách hóa đơn với bộ lọc (loại, trạng thái, ngày, đối tác). 3. Kế toán viên thực hiện hành động phù hợp (tạo mới, xem, đính kèm, bình luận, phê duyệt). 4. Hệ thống kiểm tra hợp lệ và thực thi hành động. 5. Hệ thống ghi audit log và trả về kết quả.
Luồng thay thế	4a. Kỳ kế toán đóng → HTTP 400. 4b. Hóa đơn đã được xác nhận → Không cho phép sửa/xóa trực tiếp.
Ngoại lệ	Không có quyền → HTTP 403. Lỗi CSDL → HTTP 500.

Bảng 3.23. Đặc tả Usecase Tạo hóa đơn

Đặc tả Usecase: UC-INV-02 – Tạo hóa đơn	
Tên Usecase	Tạo hóa đơn
Mã Usecase	UC-INV-02
Tác nhân	Kế toán viên

Đặc tả Usecase: UC-INV-02 – Tạo hóa đơn	
Mô tả	Kế toán viên tạo mới hóa đơn bán hàng (AR) hoặc hóa đơn mua hàng (AP) với các dòng chi tiết (invoice_lines) bao gồm mô tả, số lượng, đơn giá và tài khoản kế toán tương ứng.
Điều kiện tiên quyết	Kế toán viên có quyền invoice:create. Kỳ kế toán đang mở. Đối tác đã tồn tại. Tài khoản kế toán tham chiếu đang hoạt động.
Điều kiện sau	Hóa đơn được tạo với status = 'draft'. Các dòng hóa đơn được lưu vào invoice_lines. Tổng tiền được tính tự động.
Luồng sự kiện chính	1. Kế toán viên gửi POST /api/core/v1/invoices với header hóa đơn và mảng lines. 2. Hệ thống kiểm tra fiscal_period mở. 3. Hệ thống kiểm tra partnerId và accountId trong mỗi dòng hợp lệ. 4. Hệ thống tính tổng tiền từ các dòng chi tiết. 5. Hệ thống tạo bản ghi invoices và invoice_lines. 6. Hệ thống ghi audit log và trả về hóa đơn vừa tạo.
Luồng thay thế	2a. Fiscal period đóng → HTTP 400. 3a. accountId không tồn tại hoặc đã vô hiệu hóa → HTTP 400.
Ngoại lệ	Không có dòng hóa đơn nào → HTTP 400 validation. Lỗi CSDL → HTTP 500.

Bảng 3.24. Đặc tả Usecase Phê duyệt các cấp

Đặc tả Usecase: UC-INV-03 – Phê duyệt các cấp	
Tên Usecase	Phê duyệt các cấp
Mã Usecase	UC-INV-03
Tác nhân	Kế toán viên
Mô tả	Kế toán viên xem danh sách hóa đơn cần xử lý (đang chờ phê duyệt) và thực hiện phê duyệt theo quy trình nhiều cấp. Sau khi đủ cấp phê duyệt, hóa đơn được xác nhận (confirmed) và sinh bút toán kế toán tự động thông qua

Đặc tả Usecase: UC-INV-03 – Phê duyệt các cấp	
	Camunda workflow engine.
Điều kiện tiên quyết	Kế toán viên có quyền invoice:approve. Hóa đơn đang ở trạng thái chờ phê duyệt (pending_approval). Camunda workflow đang hoạt động.
Điều kiện sau	Trạng thái hóa đơn được cập nhật theo bước phê duyệt. Khi đủ cấp → status = 'posted', bút toán kế toán được sinh tự động vào journal_entries và journal_lines.
Luồng sự kiện chính	1. Kế toán viên truy cập GET /api/core/v1/invoice-tasks/tasks để xem danh sách cần xử lý. 2. Kế toán viên chọn hóa đơn và xem chi tiết. 3. Kế toán viên thực hiện phê duyệt hoặc từ chối. 4. Hệ thống ghi audit log.
Luồng thay thế	3a. Kế toán viên từ chối → Hóa đơn chuyển về status = 'rejected', thông báo gửi đến người tạo. 5a. Chưa đủ cấp phê duyệt → Hóa đơn chuyển sang người phê duyệt cấp tiếp theo.
Ngoại lệ	Camunda service không khả dụng → HTTP 503. Không có quyền phê duyệt → HTTP 403.

Bảng 3.25. Đặc tả Usecase Thêm tệp đính kèm vào hóa đơn

Đặc tả Usecase: UC-INV-04 – Thêm tệp đính kèm vào hóa đơn	
Tên Usecase	Thêm tệp đính kèm vào hóa đơn
Mã Usecase	UC-INV-04
Tác nhân	Kế toán viên
Mô tả	Kế toán viên đính kèm tệp (PDF, hình ảnh, Excel...) vào hóa đơn để lưu trữ chứng từ gốc. Chức năng này được cung cấp bởi EFMS Common Service thông qua bảng attachments và entity_links.
Điều kiện tiên quyết	Hóa đơn đã tồn tại. Kế toán viên có quyền truy cập. Common Service đang hoạt động.
Điều kiện sau	Metadata tệp được lưu vào bảng attachments. Liên kết giữa hóa đơn và tệp được lưu vào bảng entity_links (polymorphic).

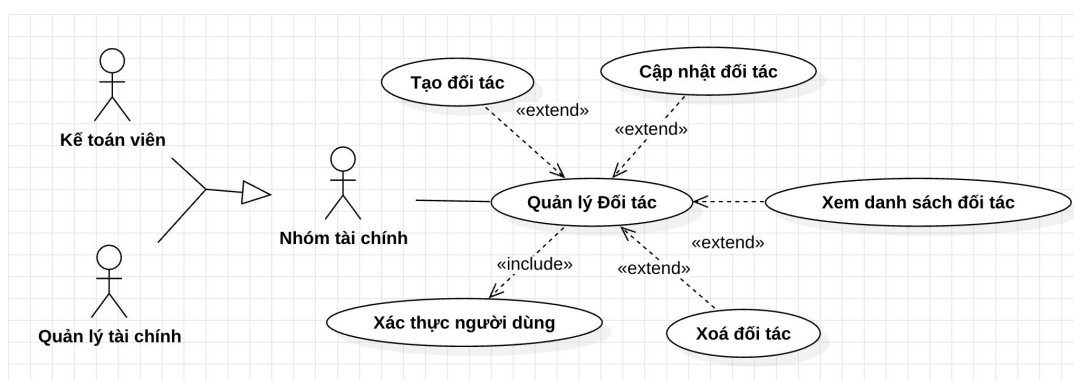
Đặc tả Usecase: UC-INV-04 – Thêm tệp đính kèm vào hóa đơn	
	Không có foreign key cứng giữa Core và Common Service.
Luồng sự kiện chính	1. Kế toán viên chọn hóa đơn và chọn chức năng Thêm tệp đính kèm. 2. Kế toán viên tải lên tệp qua POST /api/common/v1/attachments với {entityType: 'invoice', entityId}. 3. Common Service lưu metadata tệp vào bảng attachments. 4. Common Service tạo liên kết trong entity_links. 5. Hệ thống trả về thông tin tệp đính kèm.
Luồng thay thế	2a. Kích thước tệp vượt giới hạn → HTTP 400 'File too large'. 2b. Định dạng tệp không hỗ trợ → HTTP 400 'Unsupported file type'.
Ngoại lệ	Common Service không khả dụng → HTTP 503. Lỗi lưu trữ tệp → HTTP 500.

Bảng 3.26. Đặc tả Usecase Thêm bình luận vào hoá đơn

Đặc tả Usecase: UC-INV-05 – Thêm bình luận vào hoá đơn	
Tên Usecase	Thêm bình luận vào hoá đơn
Mã Usecase	UC-INV-05
Tác nhân	Kế toán viên
Mô tả	Kế toán viên thêm bình luận nội bộ vào hóa đơn để trao đổi thông tin trong quá trình xử lý. Chức năng được cung cấp bởi EFMS Common Service thông qua bảng comments và entity_links.
Điều kiện tiên quyết	Hóa đơn đã tồn tại. Người dùng đã đăng nhập và có quyền truy cập hóa đơn.
Điều kiện sau	Bình luận được lưu vào bảng comments. Liên kết được tạo trong entity_links với entityType = 'invoice'.
Luồng sự kiện chính	1. Kế toán viên gửi POST /api/common/v1/comments với {content, entityType: 'invoice', entityId}. 2. Common Service kiểm tra nội dung không rỗng.

Đặc tả Usecase: UC-INV-05 – Thêm bình luận vào hoá đơn	
	3. Common Service lưu bình luận vào bảng comments. 4. Common Service tạo liên kết trong entity_links. 5. Hệ thống trả về thông tin bình luận vừa tạo.
Luồng thay thế	2a. Nội dung bình luận rỗng → HTTP 400 validation error.
Ngoại lệ	Common Service không khả dụng → HTTP 503.

4. Usecase Quản lý Đối tác



Biểu đồ usecase Quản lý Đối tác thể hiện tác nhân 'Nhóm tài chính' (bao gồm cả Kế toán viên và Quản lý tài chính) tương tác với usecase trung tâm. Các usecase mở rộng gồm: 'Tạo đối tác', 'Cập nhật đối tác', 'Xem danh sách đối tác' và 'Xoá đối tác'. Usecase 'Xác thực người dùng' là điều kiện bắt buộc (<<include>>).

Bảng 3.27. Đặc tả Usecase Quản lý Đối tác

Đặc tả Usecase: UC-PARTNER-01 – Quản lý Đối tác	
Tên Usecase	Quản lý Đối tác
Mã Usecase	UC-PARTNER-01
Tác nhân	Kế toán viên, Quản lý tài chính (Nhóm tài chính)
Mô tả	Nhóm tài chính quản lý danh mục đối tác (khách hàng và nhà cung cấp) của công ty. Đối tác được sử dụng trong hóa đơn, thanh toán và báo cáo công nợ. Mỗi đối tác được cô lập theo company_id.
Điều kiện tiên quyết	Người dùng đã đăng nhập với quyền partner:manage. Dữ liệu company_id hợp lệ.

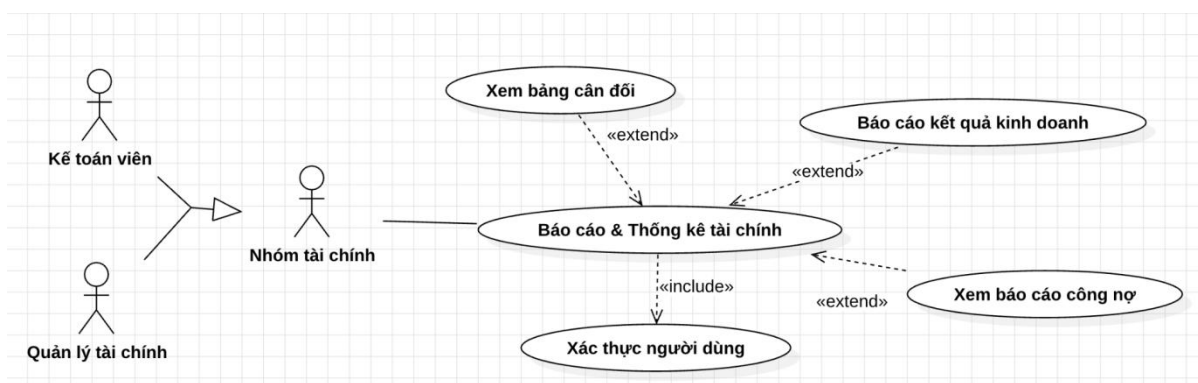
Đặc tả Usecase: UC-PARTNER-01 – Quản lý Đối tác	
Điều kiện sau	Thông tin đối tác được lưu vào bảng partners với các trường: id (UUID), company_id, name, partner_type (customer/vendor), tax_code, address, email, phone.
Luồng sự kiện chính	1. Người dùng truy cập module Quản lý Đối tác. 2. Hệ thống hiển thị danh sách đối tác với bộ lọc (loại, tên, mã số thuế). 3. Người dùng thực hiện hành động: tạo mới, cập nhật, xem hoặc xóa. 4. Hệ thống kiểm tra tính hợp lệ và thực thi. 5. Hệ thống ghi audit log và trả về kết quả.
Luồng thay thế	3a. Xóa đối tác đang có hóa đơn chưa hoàn tất → HTTP 400, yêu cầu xử lý hóa đơn trước.
Ngoại lệ	Không có quyền → HTTP 403. Lỗi CSDL → HTTP 500.

Bảng 3.28. Đặc tả Usecase Tạo đối tác

Đặc tả Usecase: UC-PARTNER-02 – Tạo đối tác	
Tên Usecase	Tạo đối tác
Mã Usecase	UC-PARTNER-02
Tác nhân	Kế toán viên, Quản lý tài chính
Mô tả	Người dùng tạo mới một đối tác (khách hàng hoặc nhà cung cấp) với đầy đủ thông tin liên hệ và phân loại. Đối tác mới được liên kết với company_id của người tạo.
Điều kiện tiên quyết	Người dùng có quyền partner:create. Mã số thuế (nếu có) chưa tồn tại trong công ty.
Điều kiện sau	Bản ghi đối tác mới được tạo trong bảng partners. Sẵn sàng được sử dụng trong hóa đơn và thanh toán.
Luồng sự kiện chính	1. Người dùng gửi POST /api/core/v1/partners với {name, partnerType, taxCode, address, email, phone}. 2. Hệ thống kiểm tra taxCode chưa trùng trong cùng công ty. 3. Hệ thống tạo bản ghi đối tác mới.

Đặc tả Usecase: UC-PARTNER-02 – Tạo đối tác	
	4. Hệ thống ghi audit log và trả về thông tin đối tác.
Luồng thay thế	2a. taxCode đã tồn tại → HTTP 409 Conflict.
Ngoại lệ	Trường name rỗng → HTTP 400. Lỗi CSDL → HTTP 500.

5. Usecase Báo cáo & Thống kê tài chính



Biểu đồ usecase Báo cáo & Thống kê tài chính thể hiện tác nhân 'Nhóm tài chính' (Kế toán viên và Quản lý tài chính) tương tác với usecase trung tâm. Các usecase mở rộng gồm: 'Xem bảng cân đối', 'Báo cáo kết quả kinh doanh' và 'Xem báo cáo công nợ'. Usecase 'Xác thực người dùng' là điều kiện bắt buộc (<<include>>).

Bảng 3.29. Đặc tả Usecase Báo cáo & Thống kê tài chính

Đặc tả Usecase: UC-REPORT-01 – Báo cáo & Thống kê tài chính	
Tên Usecase	Báo cáo & Thống kê tài chính
Mã Usecase	UC-REPORT-01
Tác nhân	Kế toán viên, Quản lý tài chính (Nhóm tài chính)
Mô tả	Nhóm tài chính xem các loại báo cáo tài chính tổng hợp của công ty: bảng cân đối kế toán (trial balance), báo cáo kết quả kinh doanh, và báo cáo công nợ (aging report). Dữ liệu được tổng hợp từ journal_entries, journal_lines và invoices theo kỳ kế toán và company_id.
Điều kiện tiên quyết	Người dùng đã đăng nhập với quyền report:view. Kỳ kế toán (fiscal_period) tham chiếu đã tồn tại.
Điều kiện sau	Báo cáo được tính toán và trả về theo thời gian thực. Không

Đặc tả Usecase: UC-REPORT-01 – Báo cáo & Thống kê tài chính	
	có thay đổi dữ liệu trong CSDL (chỉ đọc).
Luồng sự kiện chính	1. Người dùng chọn loại báo cáo và tham số (fiscalPeriodId, asOfDate, loại công nợ). 2. Hệ thống nhận yêu cầu tại endpoint tương ứng. 3. Hệ thống truy vấn dữ liệu từ journal_lines, invoices theo company_id. 4. Hệ thống tổng hợp và tính toán số liệu. 5. Hệ thống trả về báo cáo theo cấu trúc ApiResponse<T>.
Luồng thay thế	3a. Kỳ kế toán không tồn tại → HTTP 404. 3b. Không có dữ liệu trong kỳ → Trả về báo cáo với số liệu bằng 0.
Ngoại lệ	Không có quyền report:view → HTTP 403. Query timeout → HTTP 504.

Bảng 3.30. Đặc tả Usecase Xem bảng cân đối

Đặc tả Usecase: UC-REPORT-02 – Xem bảng cân đối	
Tên Usecase	Xem bảng cân đối (Trial Balance)
Mã Usecase	UC-REPORT-02
Tác nhân	Kế toán viên, Quản lý tài chính
Mô tả	Người dùng xem bảng cân đối số phát sinh (trial balance) theo kỳ kế toán. Báo cáo thể hiện số dư đầu kỳ, phát sinh nợ, phát sinh có và số dư cuối kỳ của từng tài khoản kế toán.
Điều kiện tiên quyết	Kỳ kế toán tham chiếu đã tồn tại và có dữ liệu bút toán.
Điều kiện sau	Báo cáo trial balance được trả về với đầy đủ dữ liệu theo từng tài khoản. Tổng phát sinh nợ = Tổng phát sinh có (bất biến kế toán kép).
Luồng sự kiện chính	1. Người dùng gửi GET /api/core/v1/reports/trial-balance?fiscalPeriodId={id}. 2. Hệ thống truy vấn journal_lines theo fiscal_period_id và company_id.

Đặc tả Usecase: UC-REPORT-02 – Xem bảng cân đối	
	3. Hệ thống tính tổng debit/credit cho mỗi account. 4. Hệ thống sắp xếp theo mã tài khoản và trả về kết quả. 5. Hệ thống xác nhận tổng debit = tổng credit (kiểm tra tính cân đối).
Luồng thay thế	5a. Tổng debit $\neq$ tổng credit $\rightarrow$ Hệ thống cảnh báo dữ liệu bất thường trong response.
Ngoại lệ	fiscalPeriodId không tồn tại $\rightarrow$ HTTP 404. Không có quyền $\rightarrow$ HTTP 403.

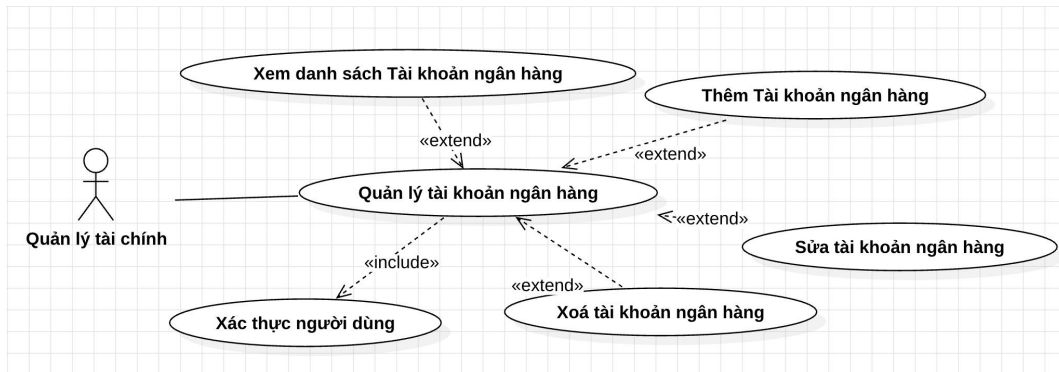
Bảng 3.31. Đặc tả Usecase Xem báo cáo công nợ

Đặc tả Usecase: UC-REPORT-03 – Xem báo cáo công nợ	
Tên Usecase	Xem báo cáo công nợ (Aging Report)
Mã Usecase	UC-REPORT-03
Tác nhân	Kế toán viên, Quản lý tài chính
Mô tả	Người dùng xem báo cáo công nợ phải thu (AR) hoặc phải trả (AP) theo phân nhóm thời gian quá hạn (0-30 ngày, 31-60 ngày, 61-90 ngày, >90 ngày). Giúp theo dõi và quản lý rủi ro tín dụng.
Điều kiện tiên quyết	Hệ thống có dữ liệu hóa đơn chưa thanh toán hết.
Điều kiện sau	Báo cáo aging được trả về theo từng đối tác và nhóm thời gian quá hạn. Không thay đổi dữ liệu CSDL.
Luồng sự kiện chính	1. Người dùng gửi GET /api/core/v1/reports/aging?type=AR&asOfDate={date}. 2. Hệ thống truy vấn các hóa đơn chưa thanh toán đầy đủ (remaining_amount > 0). 3. Hệ thống tính số ngày quá hạn so với due_date. 4. Hệ thống phân nhóm theo khoảng thời gian quá hạn. 5. Hệ thống tổng hợp theo từng đối tác và trả về kết quả.
Luồng thay thế	2a. Không có hóa đơn chưa thanh toán $\rightarrow$ Trả về danh sách rỗng.

Đặc tả Usecase: UC-REPORT-03 – Xem báo cáo công nợ

Ngoại lệ	Không có quyền report:view → HTTP 403. Query timeout → HTTP 504.
----------	--

6. Usecase Quản lý Tài khoản ngân hàng



Biểu đồ usecase Quản lý Tài khoản ngân hàng thể hiện tác nhân 'Quản lý tài chính' tương tác với usecase trung tâm. Các usecase mở rộng gồm: 'Xem danh sách Tài khoản ngân hàng', 'Thêm Tài khoản ngân hàng', 'Sửa tài khoản ngân hàng' và 'Xoá tài khoản ngân hàng'. Usecase 'Xác thực người dùng' là điều kiện bắt buộc (<<include>>).

Bảng 3.32. Đặc tả Usecase Quản lý Tài khoản ngân hàng

Đặc tả Usecase: UC-BANK-01 – Quản lý Tài khoản ngân hàng	
Tên Usecase	Quản lý Tài khoản ngân hàng
Mã Usecase	UC-BANK-01
Tác nhân	Quản lý tài chính
Mô tả	Quản lý tài chính quản lý danh mục tài khoản ngân hàng và tiền mặt của công ty. Tài khoản ngân hàng được liên kết với tài khoản kế toán tương ứng trong Chart of Accounts và được sử dụng trong các giao dịch thanh toán.
Điều kiện tiên quyết	Quản lý tài chính đã đăng nhập với quyền bank_account:manage. Tài khoản kế toán tham chiếu đã tồn tại và đang hoạt động.
Điều kiện sau	Thông tin tài khoản ngân hàng được lưu vào bảng bank_accounts với các trường: id (UUID), company_id, bank_name, account_number, account_name, currency, gl_account_id (FK đến accounts), is_active.

Đặc tả Usecase: UC-BANK-01 – Quản lý Tài khoản ngân hàng	
Luồng sự kiện chính	1. Quản lý tài chính truy cập module Quản lý Tài khoản ngân hàng. 2. Hệ thống hiển thị danh sách tài khoản ngân hàng của công ty. 3. Người dùng thực hiện hành động: thêm mới, sửa, xoá hoặc xem chi tiết. 4. Hệ thống kiểm tra hợp lệ và thực thi. 5. Hệ thống ghi audit log và trả về kết quả.
Luồng thay thế	3a. Xoá tài khoản đang có giao dịch thanh toán chưa hoàn tất → HTTP 400.
Ngoại lệ	Không có quyền → HTTP 403. Lỗi CSDL → HTTP 500.

Bảng 3.33. Đặc tả Usecase Thêm Tài khoản ngân hàng

Đặc tả Usecase: UC-BANK-02 – Thêm Tài khoản ngân hàng	
Tên Usecase	Thêm Tài khoản ngân hàng
Mã Usecase	UC-BANK-02
Tác nhân	Quản lý tài chính
Mô tả	Quản lý tài chính thêm mới tài khoản ngân hàng hoặc tài khoản tiền mặt và liên kết với tài khoản kế toán GL tương ứng để phản ánh số dư chính xác trên sổ sách.
Điều kiện tiên quyết	Quản lý tài chính có quyền bank_account:create. Tài khoản kế toán GL tham chiếu tồn tại và đang hoạt động.
Điều kiện sau	Bản ghi tài khoản ngân hàng mới được tạo với is_active = true. Sẵn sàng được chọn trong giao dịch thanh toán.
Luồng sự kiện chính	1. Quản lý tài chính gửi POST /api/core/v1/bank-accounts với {bankName, accountNumber, accountName, currency, glAccountId}. 2. Hệ thống kiểm tra glAccountId tồn tại và đang hoạt động. 3. Hệ thống kiểm tra accountNumber chưa trùng trong công ty. 4. Hệ thống tạo bản ghi tài khoản ngân hàng.

Đặc tả Usecase: UC-BANK-02 – Thêm Tài khoản ngân hàng	
	5. Hệ thống ghi audit log và trả về thông tin tài khoản.
Luồng thay thế	2a. glAccountId không tồn tại → HTTP 404. 3a. accountNumber đã tồn tại → HTTP 409 Conflict.
Ngoại lệ	Không có quyền → HTTP 403. Lỗi CSDL → HTTP 500.

Bảng 3.34. Đặc tả Usecase Sửa & Xoá Tài khoản ngân hàng

Đặc tả Usecase: UC-BANK-03 – Sửa & Xoá Tài khoản ngân hàng	
Tên Usecase	Sửa & Xoá Tài khoản ngân hàng
Mã Usecase	UC-BANK-03
Tác nhân	Quản lý tài chính
Mô tả	Quản lý tài chính cập nhật thông tin tài khoản ngân hàng (tên, ngân hàng, liên kết GL) hoặc xoá tài khoản không còn sử dụng. Chỉ cho phép xoá khi tài khoản chưa có giao dịch.
Điều kiện tiên quyết	Tài khoản ngân hàng tồn tại và thuộc company_id hiện tại. Quản lý tài chính có quyền bank_account:update và bank_account:delete.
Điều kiện sau	Thông tin tài khoản được cập nhật, hoặc bản ghi bị xoá khỏi CSDL. Audit log ghi nhận thay đổi.
Luồng sự kiện chính (Sửa)	1. Quản lý tài chính gửi PUT /api/core/v1/bank-accounts/{id} với dữ liệu cập nhật. 2. Hệ thống kiểm tra tài khoản tồn tại. 3. Hệ thống kiểm tra glAccountId mới (nếu thay đổi) còn hoạt động. 4. Hệ thống cập nhật và ghi audit log.
Luồng sự kiện chính (Xoá)	1. Quản lý tài chính gửi DELETE /api/core/v1/bank-accounts/{id}. 2. Hệ thống kiểm tra không có thanh toán nào liên kết với tài khoản. 3. Hệ thống xoá bản ghi và ghi audit log.
Luồng thay thế	Xoá: Tài khoản đang có giao dịch → HTTP 400 'Bank

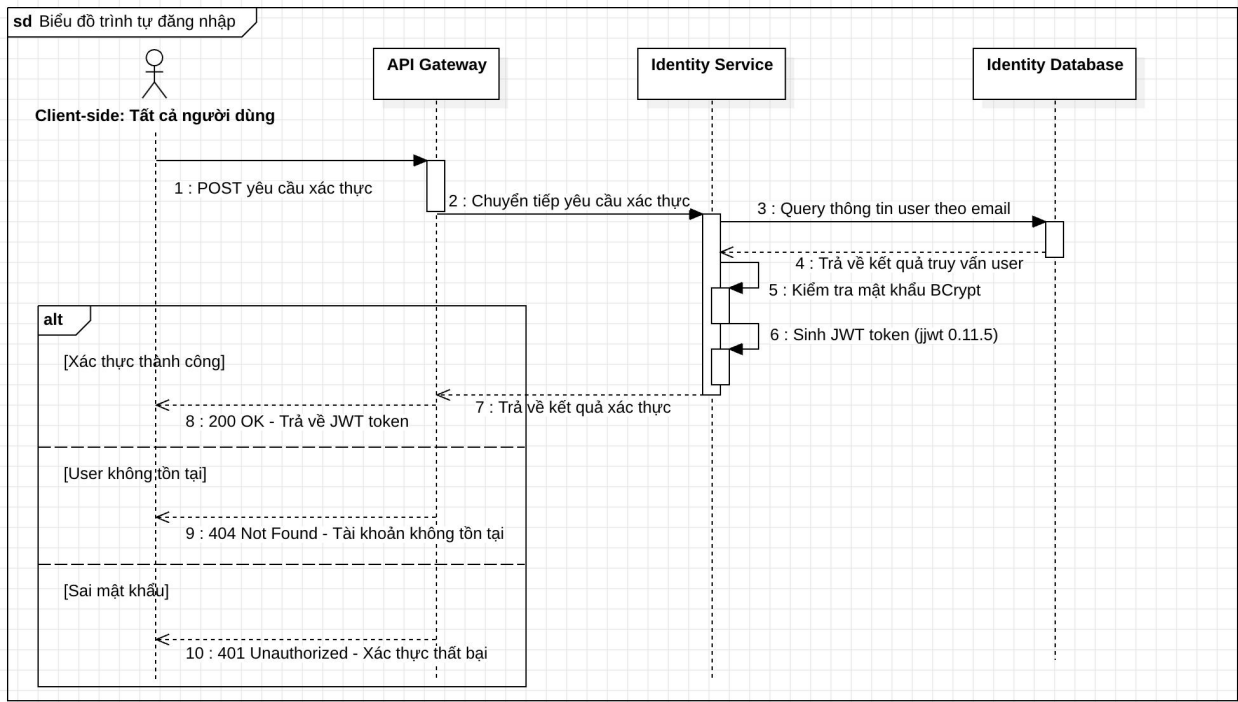
Đặc tả Usecase: UC-BANK-03 – Sửa & Xoá Tài khoản ngân hàng

	account in use, cannot delete'.
Ngoại lệ	Không tồn tại → HTTP 404. Không có quyền → HTTP 403.

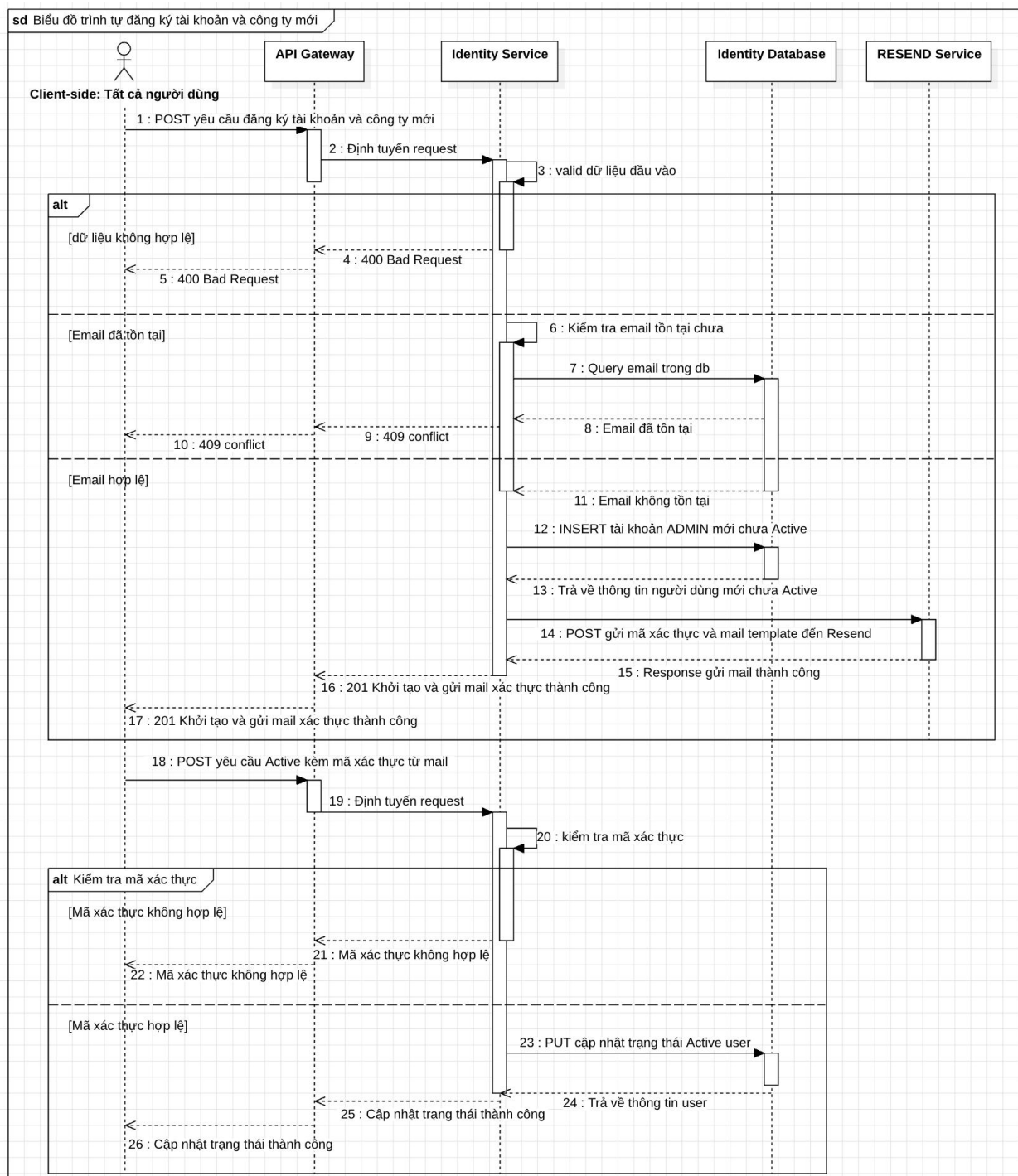
3.2.3. Biểu đồ Trình tự (Sequence Diagram)

3.2.3.1 Biểu đồ Trình tự module Quản lý xác thực, phân quyền (EFMS Identity Service)

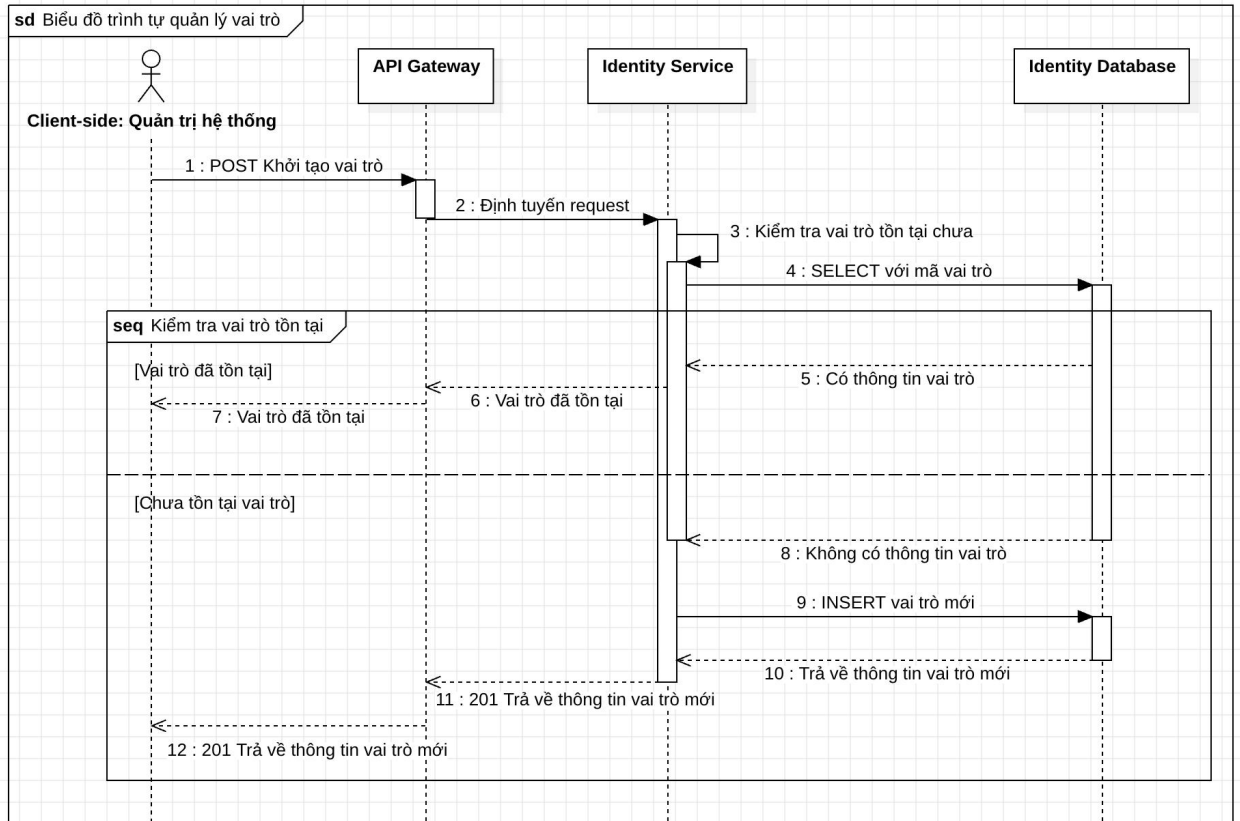
1. Biểu đồ trình tự chức năng đăng nhập:



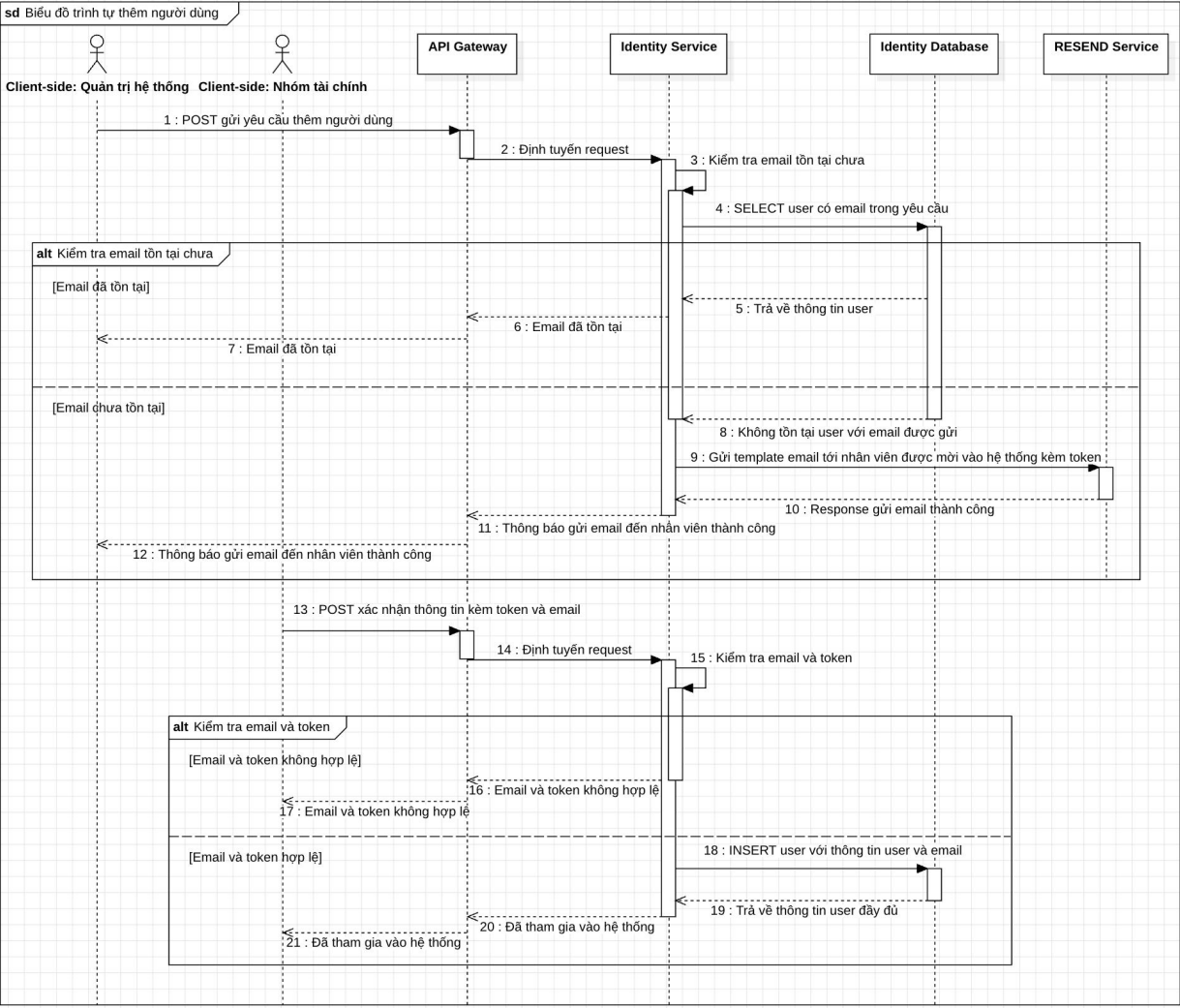
2. Biểu đồ trình tự chức năng đăng ký tài khoản công ty mới:



3. Biểu đồ trình tự chức năng quản lý vai trò người dùng:

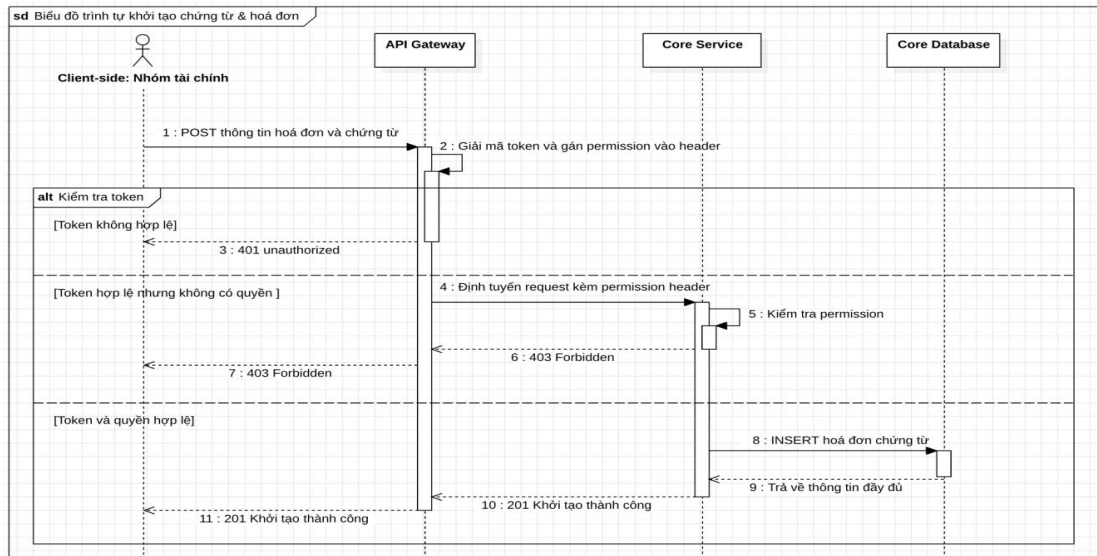


4. Biểu đồ trình tự chức năng thêm người dùng vào hệ thống công ty:

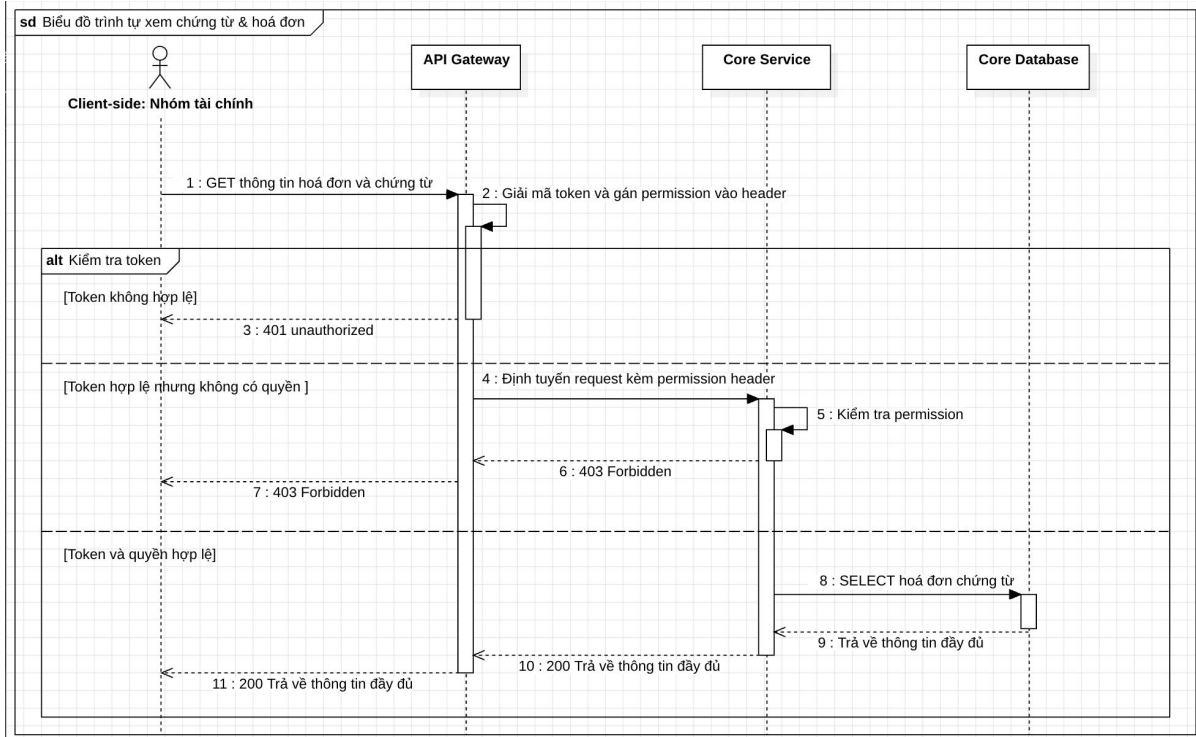


3.2.3.2 Biểu đồ Trình tự module tài chính kế toán (EFMS Core Service)

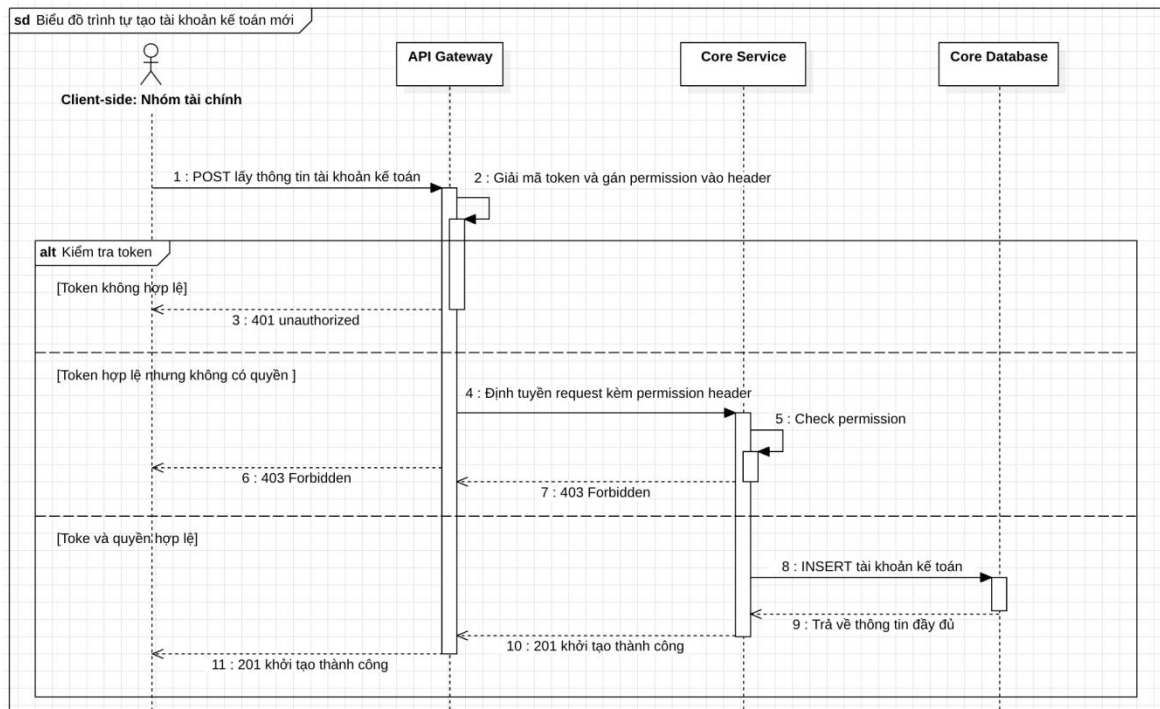
1. Biểu đồ trình tự chức năng khởi tạo chứng từ & hoá đơn:



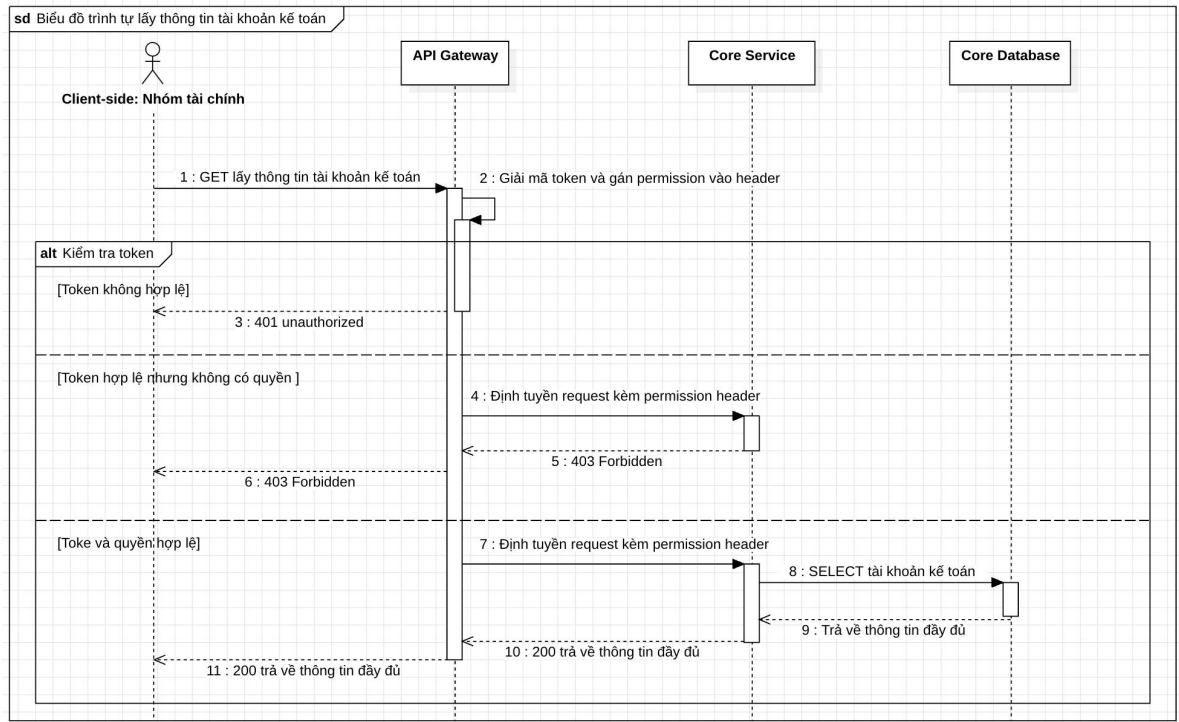
2. Biểu đồ trình tự chức năng xem chứng từ & hoá đơn:



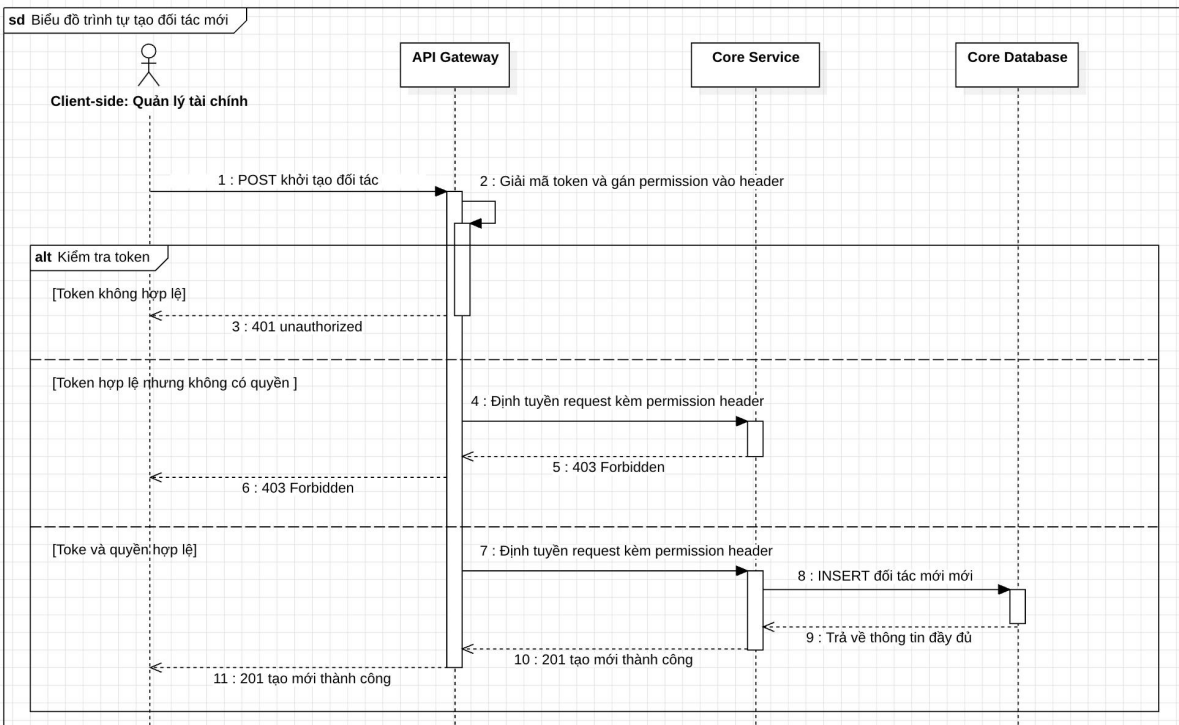
3. Biểu đồ trình tự chức năng tạo tài khoản kế toán:



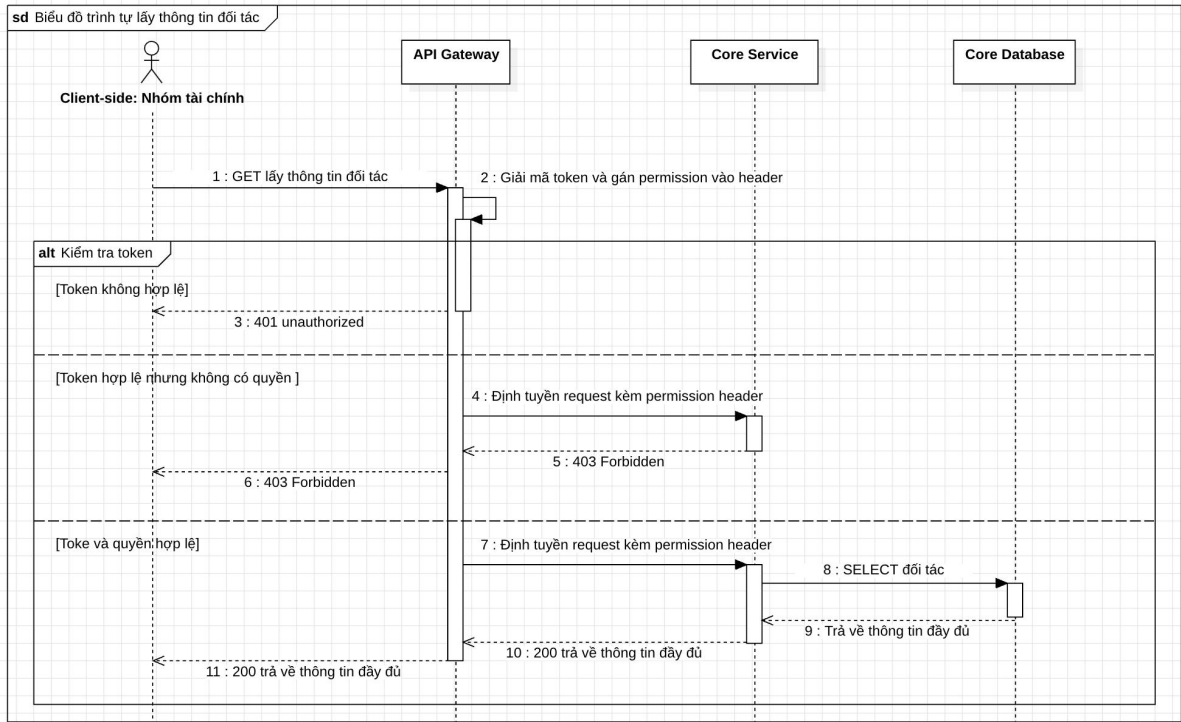
4. Biểu đồ trình tự chức năng lấy thông tin tài khoản kế toán:



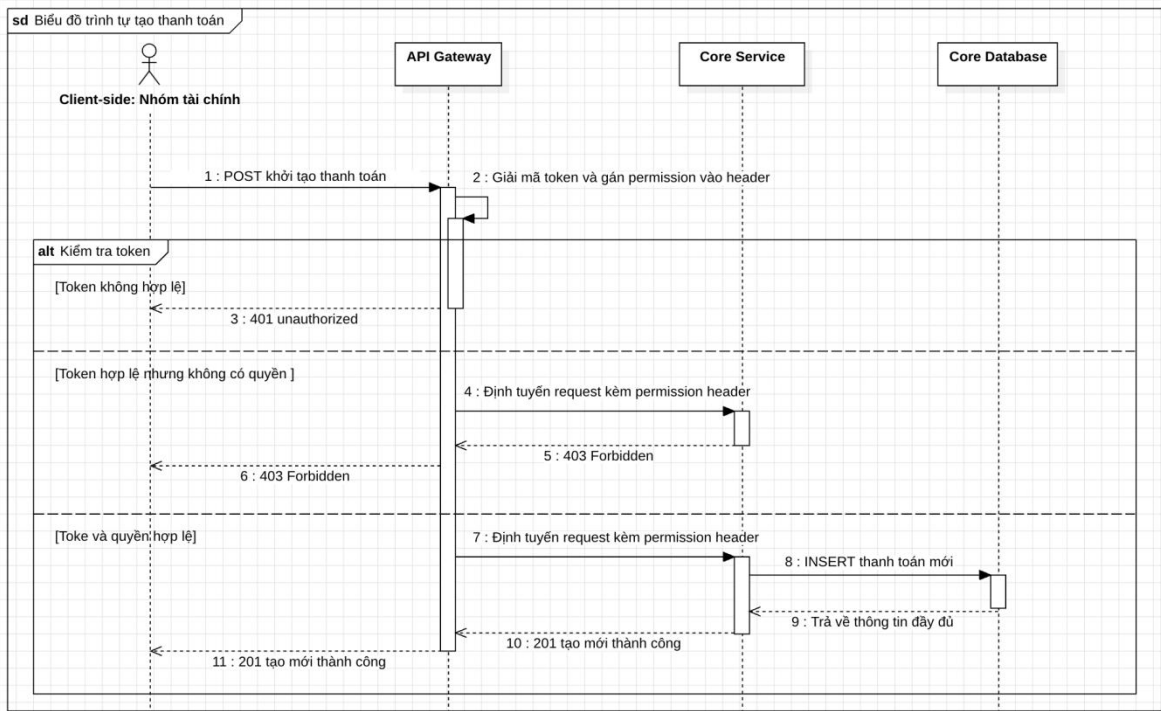
5. Biểu đồ trình tự chức năng tạo đối tác:



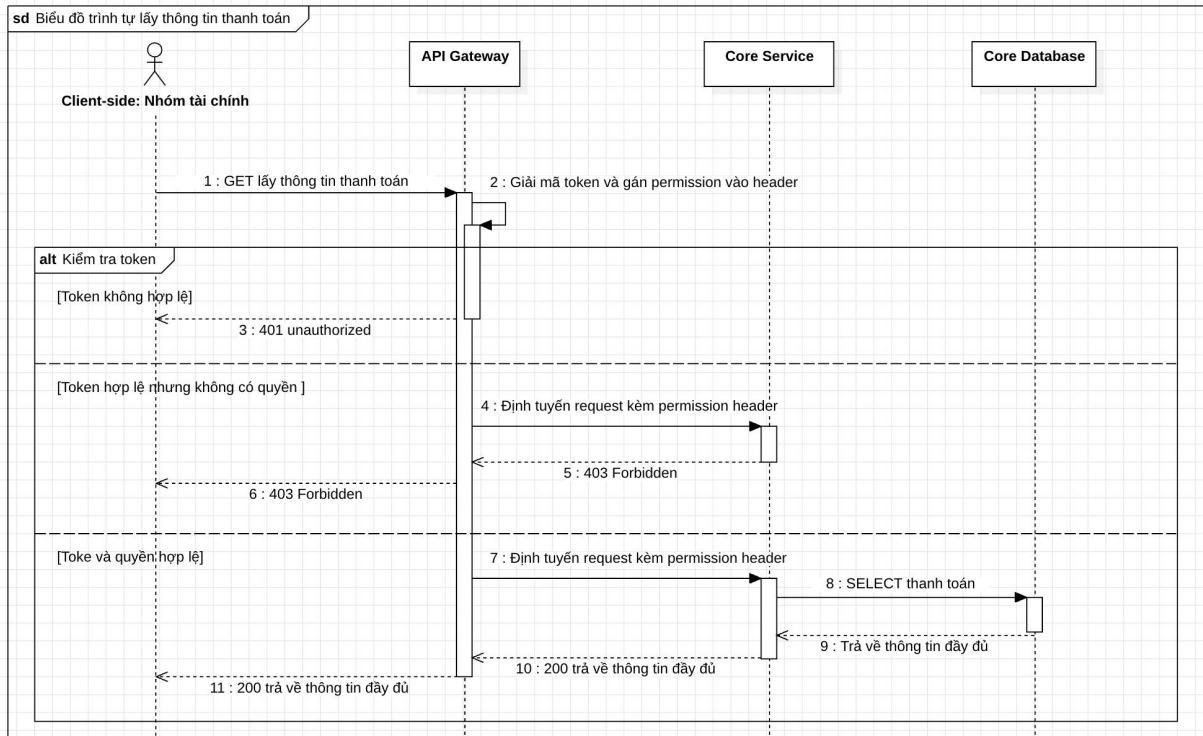
6. Biểu đồ trình tự chức năng xem đối tác:



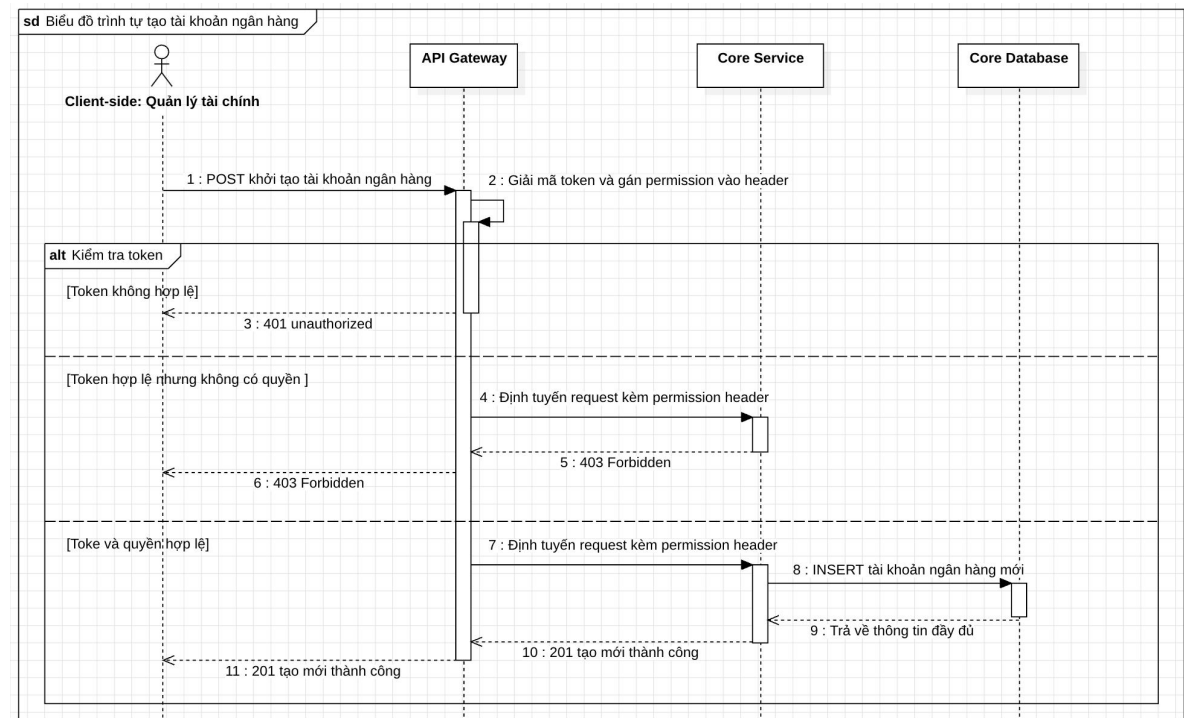
7. Biểu đồ trình tự chức năng tạo thanh toán:



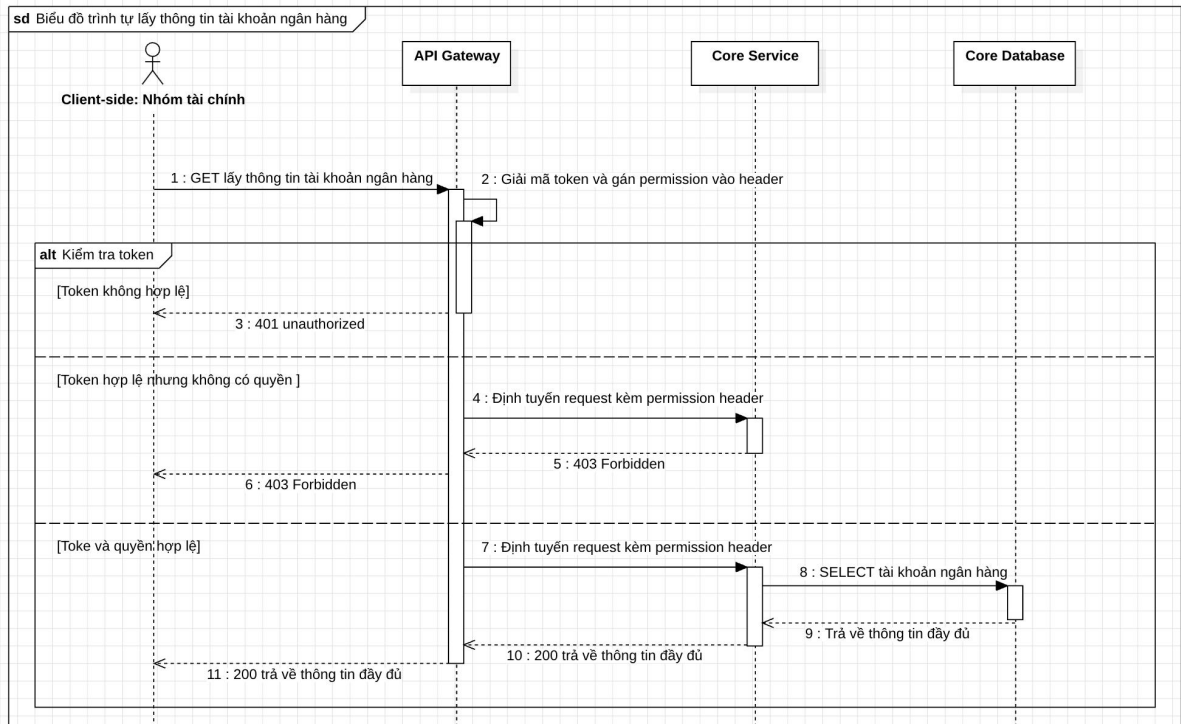
8. Biểu đồ trình tự chức năng xem thanh toán:



9. Biểu đồ trình tự chức năng tạo tài khoản ngân hàng:

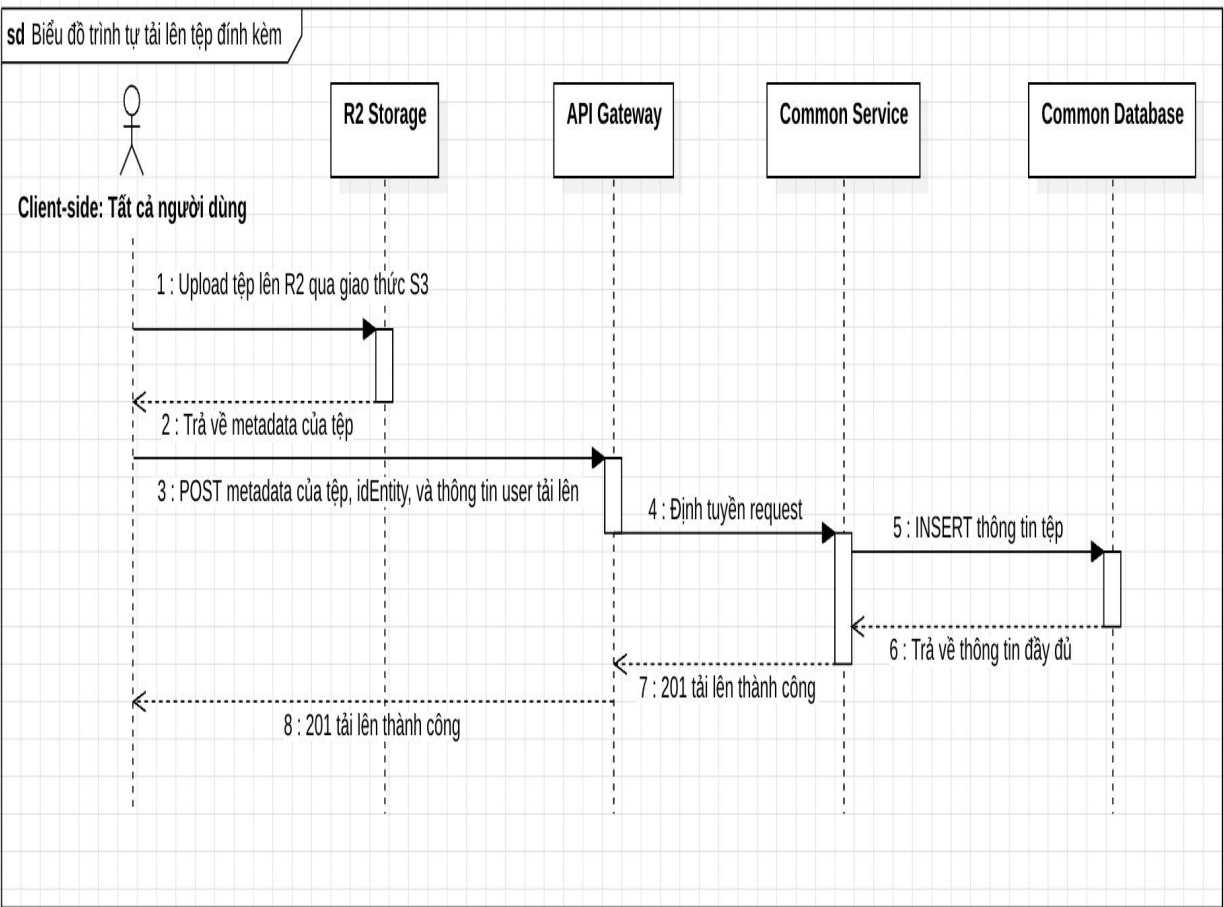


10. Biểu đồ trình tự chức năng xem tài khoản ngân hàng:

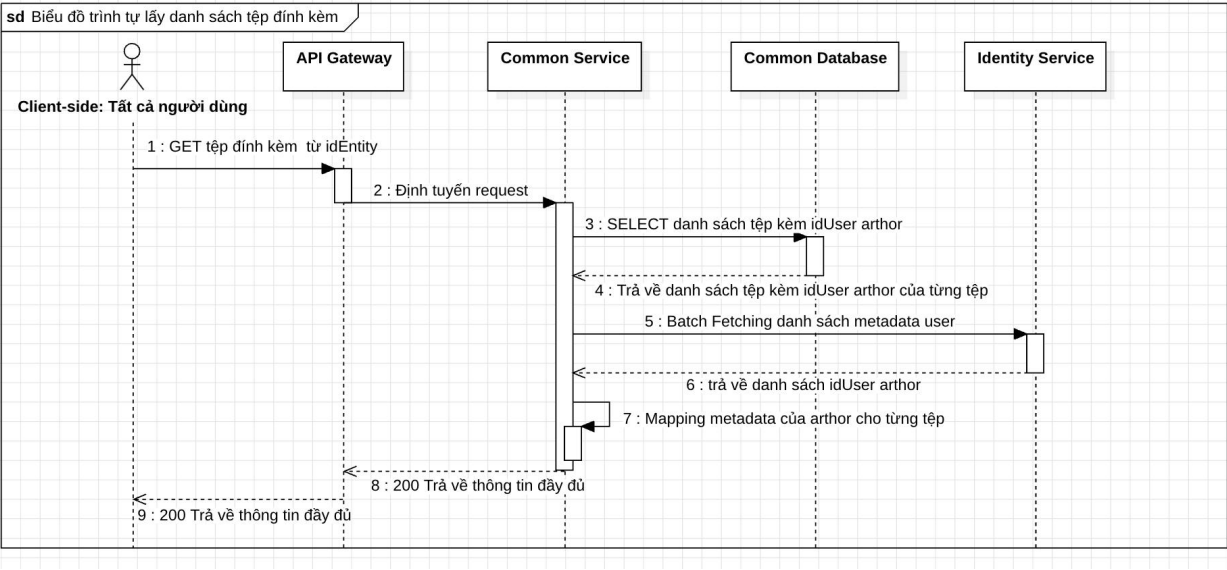


3.2.3.3 Biểu đồ Trình tự module dịch vụ dùng chung (EFMS Common Service)

1. Biểu đồ trình tự chức năng tải lên tệp đính kèm:

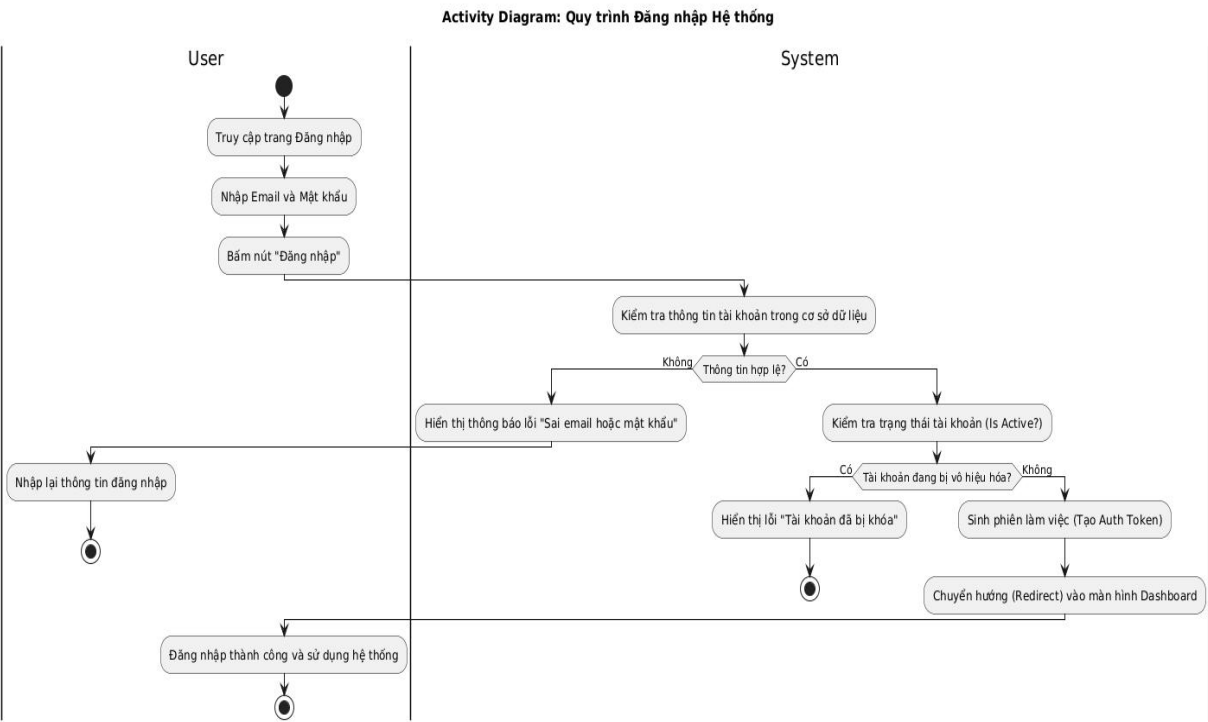


2. Biểu đồ trình tự chức năng lấy danh sách tệp đính kèm theo entity:

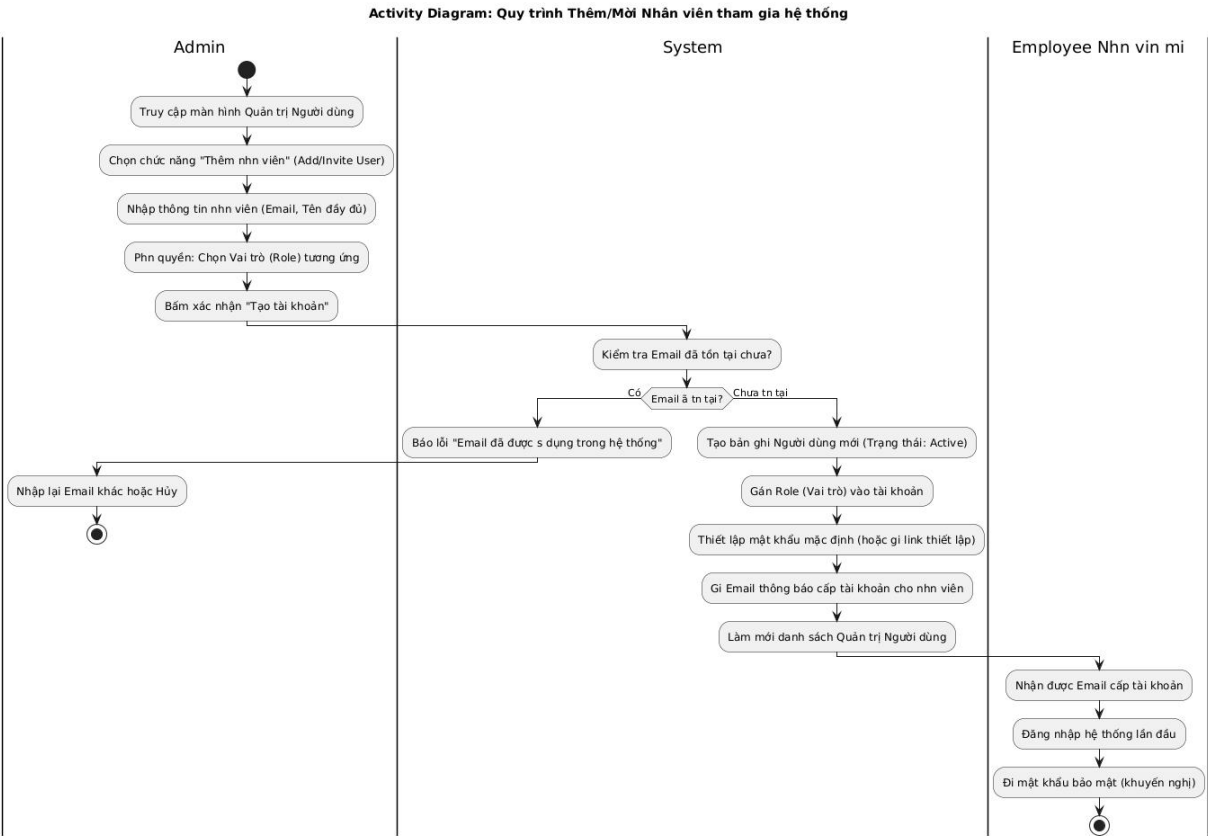


3.2.4. Biểu đồ hoạt động (Activity Diagram)

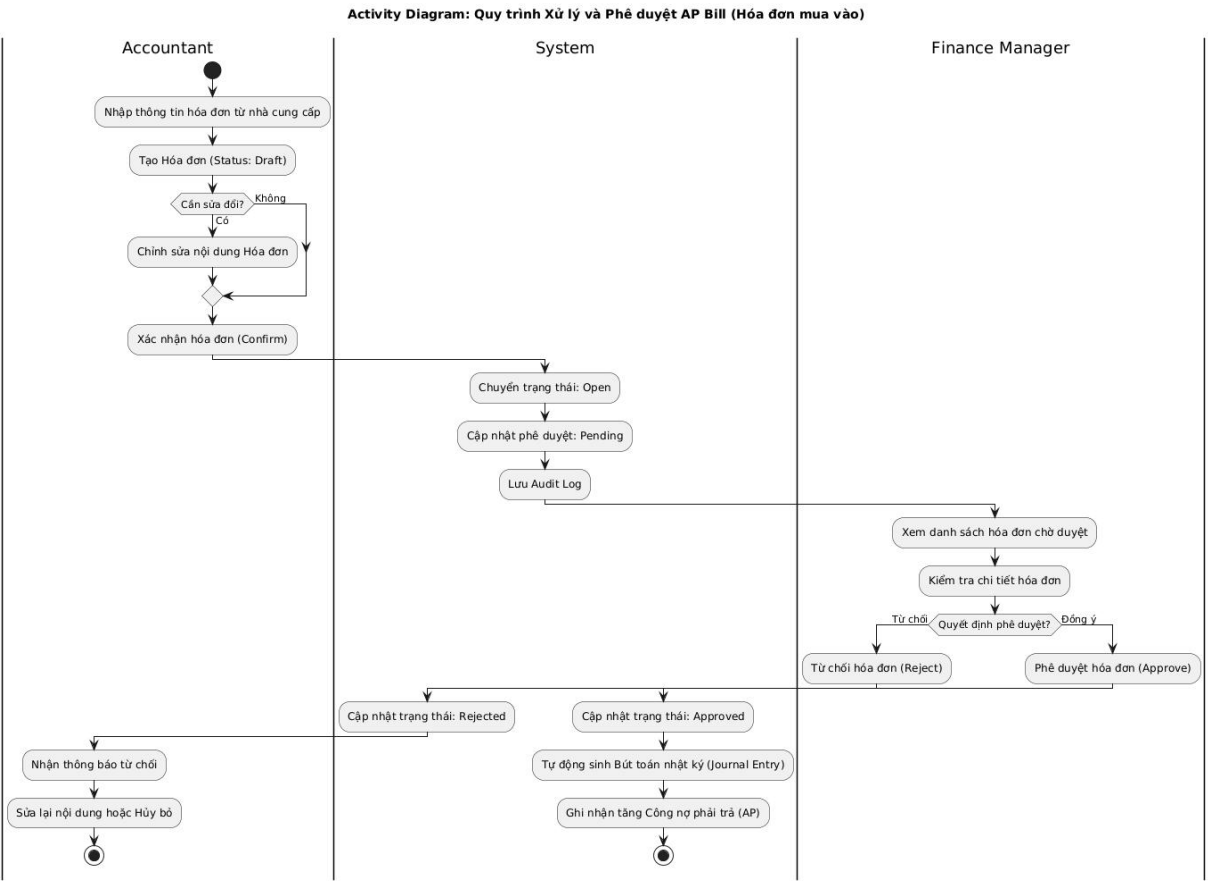
3.2.4.1. Luồng Đăng nhập (Login Workflow)



3.2.4.2. Luồng Mời/Thêm Nhân viên mới (Invite Employee)

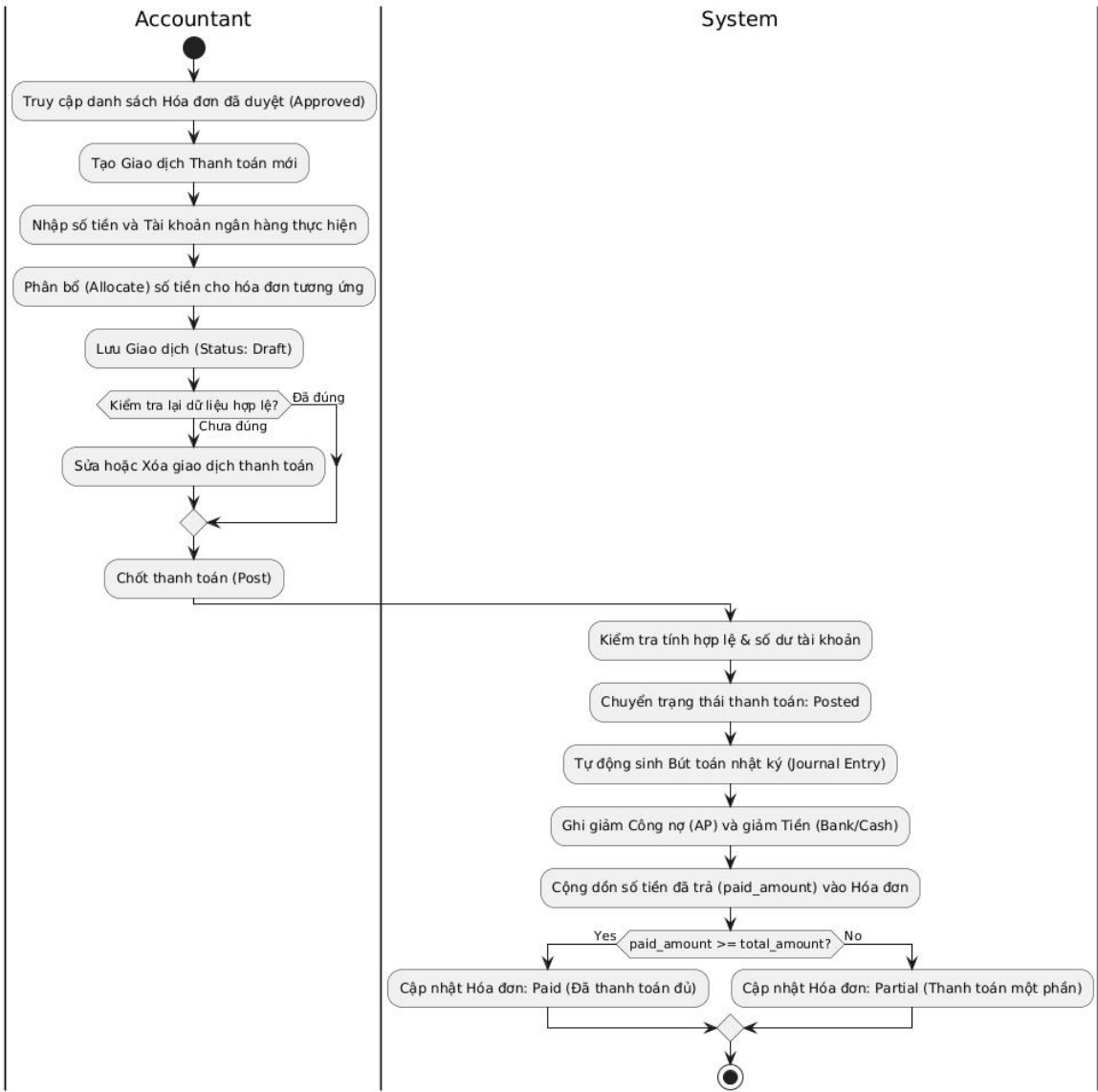


3.2.4.3. Luồng xử lý và phê duyệt Hóa đơn mua vào (AP Bill)



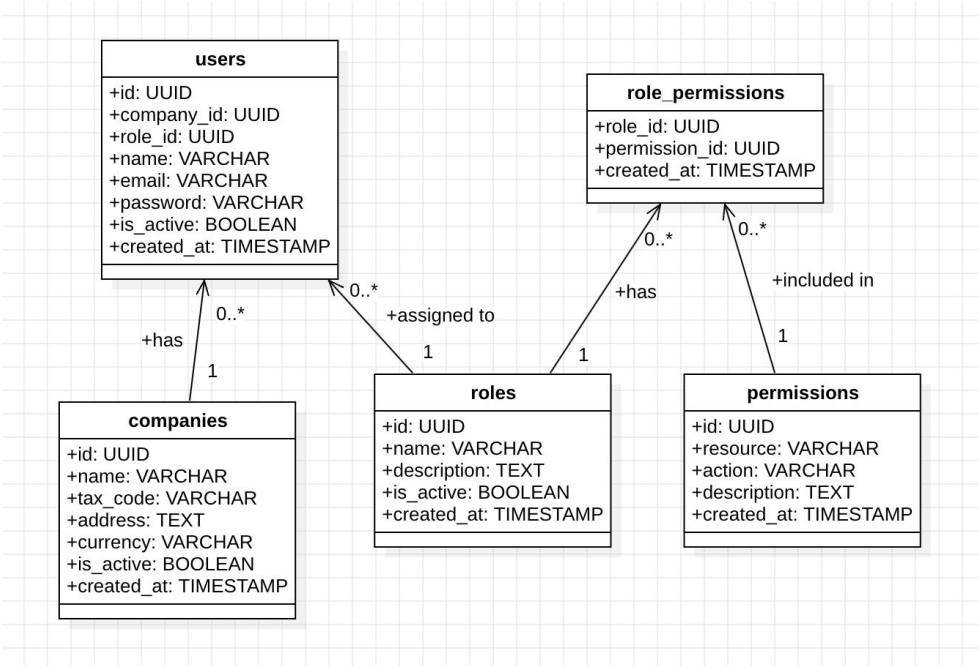
3.2.4.4. Luồng xử lý và phê duyệt Hóa đơn thu tiền(AR Bill)

Activity Diagram: Quy trình Ghi nhận và Chốt Thanh toán (Payment)



3.2.5. Biểu đồ lớp (Class Diagram)

3.2.5.1. EFMS Identity Service (Schema: identity)



Schema quản lý thông tin tổ chức, tài khoản người dùng, phân quyền bảo mật và nhật ký hệ thống.

Bảng 1.1: identity.companies (Danh sách công ty)

Tên trường	Kiểu dữ liệu	Độ rộng / Mặc định	Thuộc tính / Giải thích
id	UUID	gen_rando m_uuid()	Primary Key (PK)
name	VARCHAR	255	Tên công ty/doanh nghiệp
tax_code	VARCHAR	50	Mã số thuế
address	TEXT	-	Địa chỉ trụ sở
currency	VARCHAR	3 / Mặc định 'VND'	Đồng tiền hạch toán gốc
is_active	BOOLEAN	Default TRUE	Trạng thái hoạt động của công ty
created_at	TIMESTAMP	-	Thời điểm tạo bản ghi

Bảng 1.2: identity.roles (Vai trò/Nhóm quyền)

Tên trường	Kiểu dữ liệu	Độ rộng / Mặc định	Thuộc tính / Giải thích
id	UUID	-	Primary Key (PK)
name	VARCHAR	100	Tên vai trò (Ví dụ: Admin, Kế toán trưởng)
description	TEXT	-	Mô tả chức năng vai trò
is_active	BOOLEAN	Default TRUE	Trạng thái kích hoạt vai trò
created_at	TIMESTAMP	-	Thời điểm tạo bản ghi

Bảng 1.3: identity.permissions (Quyền chi tiết)

Tên trường	Kiểu dữ liệu	Độ rộng / Mặc định	Thuộc tính / Giải thích
id	UUID	-	Primary Key (PK)
resource	VARCHAR	100	Tên tài nguyên chịu tác động (Ví dụ: INVOICES, USERS)
action	VARCHAR	50	Hành động thao tác (Ví dụ: READ, CREATE, POST)
description	TEXT	-	Mô tả chi tiết quyền hạn
created_at	TIMESTAMP	-	Thời điểm tạo bản ghi

Bảng 1.4: identity.role_permissions (Bảng trung gian Vai trò - Quyền)

Tên trường	Kiểu dữ liệu	Độ rộng / Mặc định	Thuộc tính / Giải thích
role_id	UUID	-	Composite Primary Key, Foreign Key -> identity.roles(id)

Tên trường	Kiểu dữ liệu	Độ rộng / Mặc định	Thuộc tính / Giải thích
permission_id	UUID	-	Composite Primary Key, Foreign Key -> identity.permissions(id)
created_at	TIMESTAMP	-	Thời điểm gán quyền vào vai trò

Bảng 1.5: identity.users (Tài khoản người dùng)

Tên trường	Kiểu dữ liệu	Độ rộng / Mặc định	Thuộc tính / Giải thích
id	UUID	-	Primary Key (PK)
company_id	UUID	-	Foreign Key -> identity.companies(id)
name	VARCHAR	255	Họ tên người dùng
email	VARCHAR	255	Email đăng nhập, thuộc tính UNIQUE
password	VARCHAR	255	Mật khẩu (đã hash)
is_active	BOOLEAN	Default TRUE	Trạng thái tài khoản người dùng
created_at	TIMESTAMP	-	Thời điểm tạo người dùng

Bảng 1.6: identity.audit_logs (Nhật ký thay đổi phân quyền & hệ thống)

Tên trường	Kiểu dữ liệu	Độ rộng / Mặc định	Thuộc tính / Giải thích
id	UUID	-	Primary Key (PK)
table_name	VARCHAR	100	Tên bảng phát sinh thay đổi dữ liệu
record_id	UUID	-	ID của bản ghi bị tác động

bỏ ở v4).

Bảng 2.1: core.accounts (Hệ thống tài khoản kế toán - Chart of Accounts)

Tên trường	Kiểu dữ liệu	Độ rộng / Mặc định	Thuộc tính / Giải thích
id	UUID	-	Primary Key (PK)
company_id	UUID	-	Khóa tham chiếu công ty (<i>Indexed, Hợp thành UNIQUE chung với cột code</i>)
code	VARCHAR	20	Số hiệu tài khoản kế toán (Ví dụ: 1111, 1121)
name	VARCHAR	255	Tên gọi tài khoản
type	VARCHAR	50	Loại tài khoản (Asset, Liability, Equity, Revenue, Expense)
balance_type	VARCHAR	10	Tính chất số dư (Debit hoặc Credit)
parent_id	UUID	-	Foreign Key -> Định quy chính bảng core.accounts(id) (Tài khoản mẹ)
is_active	BOOLEAN	Default TRUE	Trạng thái sử dụng của tài khoản
created_at	TIMESTAMP	-	Thời điểm khởi tạo

Bảng 2.2: core.bank_accounts (Tài khoản ngân hàng của doanh nghiệp)

Tên trường	Kiểu dữ liệu	Độ rộng / Mặc định	Thuộc tính / Giải thích
id	UUID	-	Primary Key (PK)
company_id	UUID	-	ID công ty sở hữu tài khoản ngân hàng này

Tên trường	Kiểu dữ liệu	Độ rộng / Mặc định	Thuộc tính / Giải thích
name	VARCHAR	255	Tên gọi nhớ tài khoản ngân hàng
bank_name	VARCHAR	255	Tên ngân hàng thương mại phát hành
account_number	VARCHAR	100	Số tài khoản ngân hàng
type	VARCHAR	20	Loại tài khoản ngân hàng
currency_code	VARCHAR	3 / Mặc định 'VND'	Mã loại tiền tệ lưu trữ
opening_balance	NUMERIC	18,2	Số dư ban đầu lúc khai báo hệ thống
gl_account_id	UUID	-	Foreign Key -> Định danh Sổ cái kế toán core.accounts(id)
is_active	BOOLEAN	Default TRUE	Trạng thái hoạt động tài khoản ngân hàng
created_at	TIMESTAMP	-	Thời gian tạo bản ghi

Bảng 2.3: core.partners (Đối tác Khách hàng / Nhà cung cấp)

Tên trường	Kiểu dữ liệu	Độ rộng / Mặc định	Thuộc tính / Giải thích
id	UUID	-	Primary Key (PK)
company_id	UUID	-	ID doanh nghiệp sở hữu tệp khách hàng này (<i>Indexed</i>)
name	VARCHAR	255	Tên đầy đủ của đối tác
type	VARCHAR	20	Phân loại đối tác (Ví dụ: CUSTOMER, VENDOR, BOTH)

Tên trường	Kiểu dữ liệu	Độ rộng / Mặc định	Thuộc tính / Giải thích
tax_code	VARCHAR	50	Mã số thuế của công ty đối tác
phone	VARCHAR	50	Số điện thoại liên lạc
email	VARCHAR	255	Địa chỉ hòm thư điện tử
address	TEXT	-	Địa chỉ giao dịch đăng ký
ar_account_id	UUID	-	Foreign Key -> core.accounts(id) (Tài khoản Phải thu khách hàng - AR)
ap_account_id	UUID	-	Foreign Key -> core.accounts(id) (Tài khoản Phải trả người bán - AP)
is_active	BOOLEAN	Default TRUE	Trạng thái hoạt động
created_at	TIMESTAMP	-	Thời điểm lưu hồ sơ

Bảng 2.4: core.journal_entries (Chứng từ kế toán / Định khoản tổng hợp)

Tên trường	Kiểu dữ liệu	Độ rộng / Mặc định	Thuộc tính / Giải thích
id	UUID	-	Primary Key (PK)
company_id	UUID	-	ID công ty (<i>Indexed</i>)
period_id	UUID	-	ID kỳ kế toán quản trị ứng dụng (<i>Indexed</i>)
entry_date	DATE	-	Ngày hạch toán chứng từ trên sổ sách (<i>Indexed</i>)
reference	VARCHAR	255	Mã tham chiếu chứng từ gốc (Số hóa đơn, số phiếu thu/chi)

Tên trường	Kiểu dữ liệu	Độ rộng / Mặc định	Thuộc tính / Giải thích
description	TEXT	-	Diễn giải chung chứng từ nghiệp vụ
status	VARCHAR	20 / Mặc định 'draft'	Trạng thái bút toán (draft, posted, voided)
source	VARCHAR	50	Nguồn sinh chứng từ (Ví dụ: INVOICE, PAYMENT, MANUAL)
source_ref_id	UUID	-	ID tham chiếu trực tiếp đến đối tượng nguồn sinh
created_by	UUID	-	Người lập chứng từ (App Level logic)
posted_by	UUID	-	Người duyệt/ghi sổ cái chính thức
posted_at	TIMESTAMP	-	Thời điểm ghi sổ cái thành công
created_at	TIMESTAMP	-	Thời điểm tạo bản ghi nhập

Bảng 2.5: core.journal_lines (Bút toán chi tiết Nợ/Có - Kế toán kép)

Tên trường	Kiểu dữ liệu	Độ rộng / Mặc định	Thuộc tính / Giải thích
id	UUID	-	Primary Key (PK)
journal_entry_id	UUID	ON DELETE CASCADE	Foreign Key -> core.journal_entries(id)
account_id	UUID	-	Foreign Key -> Tài khoản kế toán chi tiết core.accounts(id)
partner_id	UUID	-	Foreign Key -> core.partners(id) (Theo dõi chi tiết công nợ đối tác)

Tên trường	Kiểu dữ liệu	Độ rộng / Mặc định	Thuộc tính / Giải thích
			nếu có)
debit	NUMERIC	18,2 / Mặc định 0	Số tiền phát sinh Nợ (<i>Ràng buộc: debit >= 0</i>)
credit	NUMERIC	18,2 / Mặc định 0	Số tiền phát sinh Có (<i>Ràng buộc: credit >= 0, không cho phép đồng thời cả debit và credit cùng > 0 trên một dòng</i>)
currency_code	VARCHAR	3 / Mặc định 'VND'	Đơn vị tiền tệ giao dịch
amount_currency	NUMERIC	18,2	Số tiền quy theo nguyên tệ giao dịch thực tế
exchange_rate	NUMERIC	18,6 / Mặc định 1	Tỷ giá hối đoái tại thời điểm hạch toán
description	TEXT	-	Diễn giải nội dung chi tiết dòng định khoản
created_at	TIMESTAMP	-	Thời điểm tạo bản ghi

Bảng 2.6: core.invoices (Hóa đơn Đầu vào / Đầu ra)

Tên trường	Kiểu dữ liệu	Độ rộng / Mặc định	Thuộc tính / Giải thích
id	UUID	-	Primary Key (PK)
company_id	UUID	-	ID công ty quản lý hóa đơn (<i>Indexed</i>)
partner_id	UUID	-	Foreign Key -> Đối tác liên quan core.partners(id) (<i>Indexed</i>)
invoice_type	VARCHAR	5	Loại hóa đơn (Ví dụ: AP - Mua vào, AR - Bán ra) (<i>Indexed</i>)

Tên trường	Kiểu dữ liệu	Độ rộng / Mặc định	Thuộc tính / Giải thích
			<i>chung kèm status)</i>
invoice_number	VARCHAR	100	Số hóa đơn tài chính chính thức
invoice_date	DATE	-	Ngày lập hóa đơn
due_date	DATE	-	Hạn thanh toán hóa đơn (<i>Indexed</i>)
currency_code	VARCHAR	3 / Mặc định 'VND'	Đồng tiền giao dịch trên hóa đơn
exchange_rate	NUMERIC	18,6 / Mặc định 1	Tỷ giá áp dụng hóa đơn
subtotal	NUMERIC	18,2 / Mặc định 0	Tổng tiền hàng trước thuế
tax_amount	NUMERIC	18,2 / Mặc định 0	Tổng tiền thuế giá trị gia tăng (VAT)
total_amount	NUMERIC	18,2 / Mặc định 0	Tổng số tiền thanh toán (Gồm thuế)
paid_amount	NUMERIC	18,2 / Mặc định 0	Số tiền lũy kế thực tế đã thanh toán
status	VARCHAR	20 / Mặc định 'draft'	Trạng thái thanh toán (draft, unpaid, partially_paid, paid)
approval_status	VARCHAR	20	Trạng thái phê duyệt nội bộ (pending, approved, rejected)
approval_comment	TEXT	-	Ý kiến phê duyệt hoặc lý do từ chối hóa đơn
journal_entry_id	UUID	-	Foreign Key -> Chứng từ sinh ra tự động core.journal_entries(id)

Tên trường	Kiểu dữ liệu	Độ rộng / Mặc định	Thuộc tính / Giải thích
created_by	UUID	-	ID người nhập liệu (App Level)
created_at	TIMESTAMP	-	Thời điểm số hóa hóa đơn

Bảng 2.7: core.invoice_lines (Chi tiết mặt hàng trên hóa đơn)

Tên trường	Kiểu dữ liệu	Độ rộng / Mặc định	Thuộc tính / Giải thích
id	UUID	-	Primary Key (PK)
invoice_id	UUID	ON DELETE CASCADE	Foreign Key -> Tập hóa đơn chính core.invoices(id)
account_id	UUID	-	Foreign Key -> Tài khoản kế toán đối ứng core.accounts(id)
description	TEXT	-	Tên hàng hóa, dịch vụ diễn giải chi tiết
quantity	NUMERIC	10,2 / Mặc định 1	Số lượng mặt hàng mua/bán
unit_price	NUMERIC	18,2 / Mặc định 0	Đơn giá chưa thuế
tax_rate	NUMERIC	5,2 / Mặc định 0	Phần trăm thuế suất áp dụng (Ví dụ: 8.00, 10.00)
tax_amount	NUMERIC	18,2 / Mặc định 0	Tiền thuế VAT của dòng hàng này
amount	NUMERIC	18,2 / Mặc định 0	Thành tiền trước thuế của dòng mặt hàng (quantity * unit_price)

Bảng 2.8: core.payments (Phiếu thanh toán Trả tiền / Thu tiền)

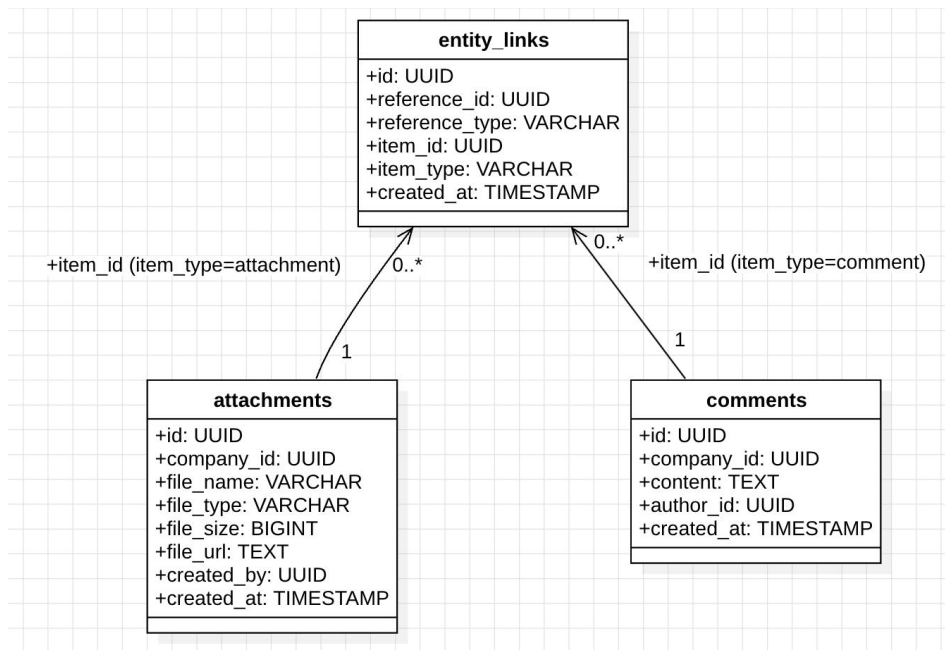
Tên trường	Kiểu dữ liệu	Độ rộng / Mặc định	Thuộc tính / Giải thích
id	UUID	-	Primary Key (PK)
company_id	UUID	-	ID doanh nghiệp phát sinh dòng tiền (<i>Indexed</i>)
partner_id	UUID	-	Foreign Key -> core.partners(id)
invoice_id	UUID	-	Foreign Key -> core.invoices(id) (Hóa đơn cần thanh toán bốc dỡ - <i>Indexed</i>)
payment_type	VARCHAR	10	Dạng thanh toán (Ví dụ: INBOUND - Thu tiền, OUTBOUND - Chi tiền)
payment_date	DATE	-	Ngày lập phiếu chi/thu tiền mặt hoặc báo Có/Nợ
currency_code	VARCHAR	3 / Mặc định 'VND'	Loại tiền giao dịch thanh toán
exchange_rate	NUMERIC	18,6 / Mặc định 1	Tỷ giá quy đổi ngoại tệ
amount	NUMERIC	18,2	Số tiền thực chi/thu
payment_method	VARCHAR	50	Phương thức thanh toán (CASH, BANK_TRANSFER, CREDIT_CARD)
bank_account_id	UUID	-	Foreign Key -> Tài khoản ngân hàng công ty core.bank_accounts(id) (nếu đi qua BANK)
reference	VARCHAR	255	Mã giao dịch ngân hàng / Ủy nhiệm chi / Số phiếu thu

Tên trường	Kiểu dữ liệu	Độ rộng / Mặc định	Thuộc tính / Giải thích
journal_entry_id	UUID	-	Foreign Key -> Định khoản kế toán phát sinh tự động core.journal_entries(id)
created_by	UUID	-	ID thủ quỹ / kế toán thanh toán thực hiện (App Level)
created_at	TIMESTAMP	-	Thời điểm tạo bản ghi

Bảng 2.9: core.audit_logs (Nhật ký thay đổi dữ liệu tài chính kế toán)

Tên trường	Kiểu dữ liệu	Độ rộng / Mặc định	Thuộc tính / Giải thích
id	UUID	-	Primary Key (PK)
table_name	VARCHAR	100	Tên bảng phát sinh tác động (<i>Tổ hợp Index cùng cột record_id</i>)
record_id	UUID	-	ID của bản ghi kế toán bị sửa đổi (<i>Tổ hợp Index cùng cột table_name</i>)
action	VARCHAR	20	Hành động tác động (INSERT, UPDATE, DELETE)
changed_by	UUID	-	ID người sửa đổi dữ liệu cốt lõi (App Level)
changed_at	TIMESTAMP	-	Thời gian thực hiện hành động
old_data	JSONB	-	Giá trị cấu trúc cũ (Lưu JSON)
new_data	JSONB	-	Giá trị cấu trúc mới cập nhật (Lưu JSON)

3.2.5.3. EFMS Common Service (Schema: common)



Schema tiện ích chung, quản lý file đính kèm, bình luận hệ thống và liên kết động giữa các thực thể mà không cần sử dụng ràng buộc khóa ngoại cứng (Loose Coupling Architecture).

Bảng 3.1: common.attachments (Tập tin đính kèm chứng từ/hóa đơn)

Tên trường	Kiểu dữ liệu	Độ rộng / Mặc định	Thuộc tính / Giải thích
id	UUID	-	Primary Key (PK)
company_id	UUID	-	ID doanh nghiệp lưu trữ file
file_name	VARCHAR	255	Tên file đính kèm hiển thị
file_type	VARCHAR	100	Định dạng file (Ví dụ: application/pdf, image/png)
file_size	BIGINT	-	Dung lượng file tính bằng đơn vị Bytes
file_url	TEXT	-	Đường dẫn lưu trữ vật lý hoặc URL Cloud Storage (S3/MinIO)
created_by	UUID	-	Mã người dùng upload tệp (App Level)
created_at	TIMESTAMP	-	Thời điểm tải lên thành công

Bảng 3.2: common.comments (Thảo luận / Ghi chú nội bộ trên chứng từ)

Tên trường	Kiểu dữ liệu	Độ rộng / Mặc định	Thuộc tính / Giải thích
id	UUID	-	Primary Key (PK)
company_id	UUID	-	ID tổ chức áp dụng
content	TEXT	-	Nội dung bình luận / Lý do xử lý văn bản nghiệp vụ
author_id	UUID	-	ID người thảo luận (App Level)
created_at	TIMESTAMP	-	Thời gian gửi phản hồi

Bảng 3.3: common.entity_links (Bảng ánh xạ đa năng / Liên kết động phân hệ)

Tên trường	Kiểu dữ liệu	Độ rộng / Mặc định	Thuộc tính / Giải thích
id	UUID	-	Primary Key (PK)
reference_id	UUID	-	ID thực thể nguồn (<i>Hợp thành Index phức hợp cùng reference_type và tạo khóa UNIQUE toàn cục</i>)
reference_type	VARCHAR	50	Loại thực thể gốc (Ví dụ: INVOICES, JOURNAL_ENTRIES)
item_id	UUID	-	ID của thực thể đích được liên kết tới như tệp tin/bình luận (<i>Hợp thành Index cùng item_type</i>)
item_type	VARCHAR	50	Loại đối tượng liên kết mục tiêu (Ví dụ: ATTACHMENTS, COMMENTS)
created_at	TIMESTAMP	-	Thời điểm ánh xạ quan hệ thành công

Bảng 3.4: common.audit_logs (Nhật ký thay đổi tiện ích dùng chung)

Tên trường	Kiểu dữ liệu	Độ rộng / Mặc định	Thuộc tính / Giải thích
id	UUID	-	Primary Key (PK)
table_name	VARCHAR	100	Tên bảng trong schema common phát sinh tác động
record_id	UUID	-	Mã định danh bản ghi bị thay đổi
action	VARCHAR	20	Hành động thao tác dữ liệu (INSERT, UPDATE, DELETE)
changed_by	UUID	-	ID người dùng thao tác dữ liệu (App Level)
changed_at	TIMESTAMP	-	Thời gian lưu nhật ký tệp
old_data	JSONB	-	Dữ liệu JSON cũ trước chỉnh sửa
new_data	JSONB	-	Dữ liệu JSON mới sau khi cập nhật

CHƯƠNG 4. XÂY DỰNG CHƯƠNG TRÌNH

4.1. Xây dựng hệ thống

4.1.1. Mô tả hệ thống

Hệ thống EFMS được xây dựng trên nền tảng kiến trúc Microservices hiện đại, nhằm đảm bảo sự phân tách rõ ràng giữa các ranh giới nghiệp vụ (Microservices Boundary). Hệ thống vận hành với ngôn ngữ Java 21, framework Spring Boot 3.3.x và hệ quản trị cơ sở dữ liệu PostgreSQL. Kiến trúc tổng thể được chia thành 04 dịch vụ độc lập, giao tiếp với nhau qua chuẩn REST API:

- EFMS API Gateway (Port 8080): Đóng vai trò là điểm truy cập trung tâm (Entry point) cho toàn bộ hệ thống. Dịch vụ này đảm nhận việc định tuyến động (Dynamic Routing) đến các dịch vụ hạ nguồn, kiểm soát lỗi toàn cục, xử lý CORS và đặc biệt là xác thực mã thông báo JWT từ người dùng.
- EFMS Identity Service (Port 8081): Quản lý cấu trúc tổ chức đa doanh nghiệp (Multi-tenancy), hồ sơ người dùng, xác thực (Authentication) và phân quyền kiểm soát truy cập dựa trên vai trò (RBAC).
- EFMS Core Service (Port 8082): Trái tim của hệ thống, xử lý toàn bộ các nghiệp vụ tài chính cốt lõi (Procure-to-Pay) bao gồm: Quản lý danh mục tài khoản (COA), đối tác (Partners), hóa đơn (Invoices), thanh toán (Payments) và tự động hóa sổ cái kế toán (General Ledger).
- EFMS Common Service (Port 8083): Cung cấp các tiện ích dùng chung xuyên suốt hệ thống như quản lý siêu dữ liệu tệp đính kèm (Attachments), bình luận (Comments) và hệ thống lưu vết lịch sử thay đổi (Audit Logs).

4.1.2. Lý do chọn công nghệ

Việc quyết định stack công nghệ cho EFMS được đánh giá dựa trên yêu cầu khắt khe về tính chính xác tài chính, bảo mật và hiệu năng:

- Java 21 & Spring Boot 3.3.x: Java 21 mang lại hiệu suất cao với các luồng ảo (Virtual Threads), tối ưu cho các hệ thống xử lý đồng thời. Hệ sinh thái Spring Boot/Spring Cloud cung cấp bộ công cụ toàn diện (Gateway, Security) giúp rút ngắn thời gian phát triển và đảm bảo tính ổn định cấp doanh nghiệp.
- PostgreSQL: Là hệ quản trị cơ sở dữ liệu quan hệ mạnh mẽ, không chỉ hỗ trợ tốt các giao dịch ACID (bắt buộc cho hệ thống kế toán) mà còn hỗ trợ kiểu dữ liệu JSONB, rất tối ưu để lưu trữ dữ liệu cũ/mới (old_data, new_data) trong bảng Audit Logs.
- Kiến trúc Microservices: Thay vì kiến trúc Monolithic truyền thống, Microservices giúp cô lập lỗi (Fault Isolation). Nếu dịch vụ tiện ích (Common) gặp sự cố, luồng thanh toán hóa đơn tại dịch vụ cốt lõi (Core) vẫn có thể hoạt động độc lập.
- Mô hình Polymorphic (Đa hình) ở cấp Database: Trong dịch vụ Common, hệ thống sử dụng bảng trung gian entity_links (reference_id, reference_type) thay vì dùng khóa ngoại (Foreign Keys) cứng nhắc. Điều này cho phép đính kèm tài liệu vào bất kỳ thực thể nào (Hóa đơn, Thanh toán) mà không làm rườm rà cấu trúc Database.

4.1.3. Khả năng mở rộng (Scalability)

Hệ thống được thiết kế với tư duy "Mở rộng từ trong ra ngoài" (Scale from within), đáp ứng nhu cầu tăng trưởng của doanh nghiệp ở nhiều khía cạnh:

Phương diện	Chi tiết thiết kế mở rộng
-------------	---------------------------

Phương diện	Chi tiết thiết kế mở rộng
Mở rộng theo chiều ngang (Horizontal)	Các dịch vụ được đóng gói độc lập, cho phép dễ dàng tăng số lượng bản sao (instances) cho các dịch vụ chịu tải cao (như Core Service) qua công nghệ Container mà không ảnh hưởng đến các dịch vụ khác.
Mở rộng Đa doanh nghiệp (Multi-tenancy)	EFMS sử dụng cơ chế cô lập dữ liệu logic bằng trường <code>company_id</code> ở tầng ứng dụng. Thiết kế này giúp hệ thống phục vụ vô số khách hàng/doanh nghiệp trên cùng một hạ tầng Database vật lý mà dữ liệu vẫn được bảo mật tuyệt đối.
Tính toàn vẹn dữ liệu phân tán	Việc loại bỏ hoàn toàn các khóa ngoại (Foreign Keys) liên kết chéo giữa các microservices ở cấp độ cơ sở dữ liệu giúp các module không bị phụ thuộc cứng. Việc tham chiếu được thực hiện qua chuỗi UUID chung toàn cầu.
Chuẩn hóa Giao thức Giao tiếp	Mọi API đều trả về cấu trúc bọc ApiResponse thống nhất và bắt buộc sử dụng MapStruct để ánh xạ Entity sang DTO. Điều này tạo ra một "hợp đồng giao tiếp" chuẩn, giúp các dịch vụ bên thứ ba hoặc tính năng mới tích hợp dễ dàng.

4.1.4. Chi tiết nghiệp vụ và Vận hành hệ thống

4.1.4.1. Cơ chế Bảo mật 4 Lớp (4-Phase Security Architecture)

Hệ thống không lưu trữ trạng thái phiên đăng nhập (Stateless) mà sử dụng cơ chế JWT kết hợp luồng xác thực 4 bước chặt chẽ:

- Phase 1 (Gateway): API Gateway chặn mọi Request, xác thực chữ ký và thời hạn của JWT.
- Phase 2 (Injection): Trích xuất thông tin người dùng (claims) từ token và cấy (inject) vào các Header tùy chỉnh (X-User-Id, X-User-Company-Id, X-User-Permission).
- Phase 3 (Local Context): Các dịch vụ hạ nguồn dùng GatewayHeaderFilter đọc Header này và nạp vào SecurityContextHolder cục bộ.
- Phase 4 (Access Control): Áp dụng RBAC tại cấp độ hàm thông qua annotation `@PreAuthorize("hasAuthority('...')` để đối chiếu quyền thao tác (Ví dụ: INVOICE:CREATE).

4.1.4.2. Luồng Nghiệp vụ Tài chính Cốt lõi (Procure-to-Pay)

- Vòng đời chứng từ qua DB State Machine: Thay vì dùng các Engine phức tạp, quy trình duyệt Hóa đơn (AP Bill) được quản lý qua State Machine tại Database. Hóa đơn đi từ Draft → Open/Pending (Chờ Quản lý/Admin duyệt) → Approved → Thanh toán.
- Tự động hóa Bút toán kép (Double-Entry Automation): Hệ thống tuân thủ nghiêm ngặt nguyên tắc kế toán chuẩn. Việc hạch toán (Journal Entries) không cho phép nhập tay. Bút toán chỉ được tự động sinh ra bởi hệ thống (thông qua

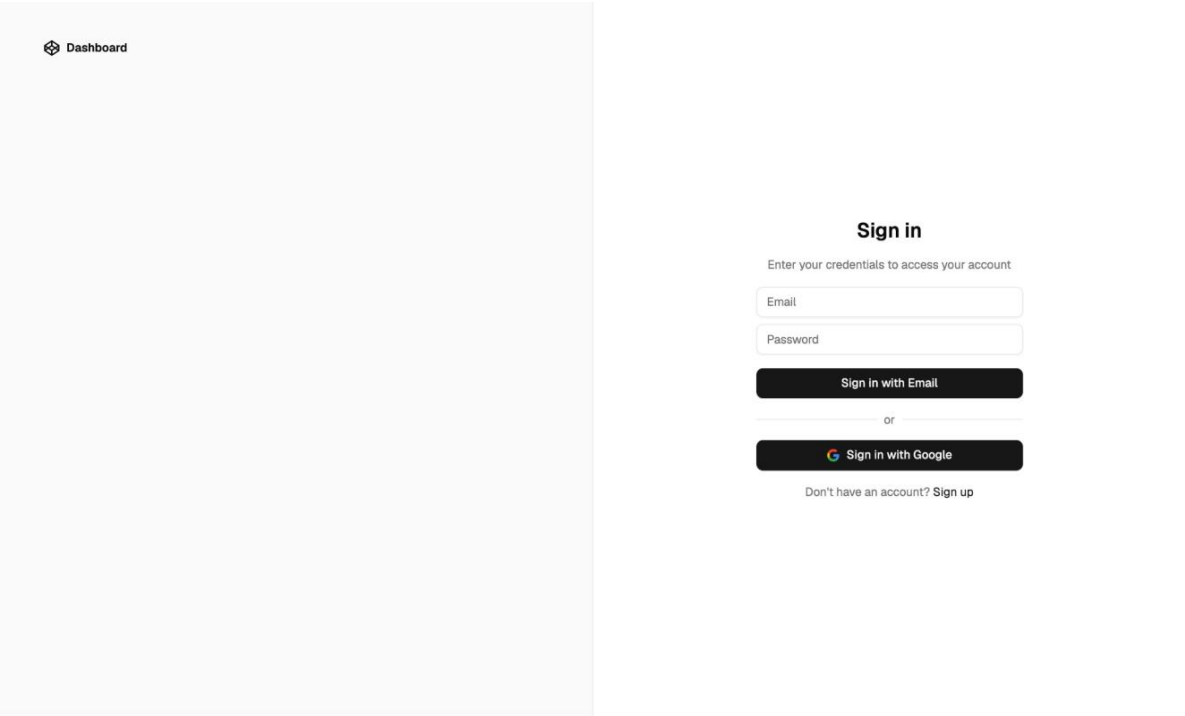
CreateJournalEntryWorker) khi Hóa đơn được duyệt hoặc khi Thanh toán được ghi sổ (Posted) nhằm đảm bảo cân đối Nợ/Có (Debit/Credit).

4.1.4.3. Tích hợp Trợ lý ảo Trí tuệ Nhân tạo (AI Integration)

- Điểm nhấn công nghệ của EFMS là việc nghiên cứu và ứng dụng giao thức MCP (Model Context Protocol) Server.
- Hệ thống xây dựng một máy chủ trung gian bảo mật (chạy qua HTTP + SSE Transport với luồng OAuth 2.1) để kết nối hệ thống dữ liệu cốt lõi với mô hình ngôn ngữ lớn (Claude AI).
- Việc này cho phép người quản lý tài chính thực hiện các thao tác tra cứu phức tạp (như tính toán số dư, báo cáo tuổi nợ, phân tích trạng thái hóa đơn) bằng ngôn ngữ tự nhiên, nâng cao đáng kể trải nghiệm và hiệu quả quản trị trong kỷ nguyên chuyển đổi số.

4.2. Giao diện người dùng

4.2.1. Giao diện đăng nhập.



4.2.2. Giao diện đăng ký tài khoản Admin và công ty mới

Dashboard

Đăng ký công ty

Tạo tài khoản và công ty mới để sử dụng EFMS

Email

Gửi mã

Họ và Tên

Mã xác thực (OTP)

Mật khẩu

Tên công ty

Đăng ký

Đã có tài khoản? Đăng nhập

4.2.3. Giao diện bảng điều khiển (Dashboard)

Dashboard

Hoá đơn & Giao dịch

Hóa đơn & Chứng từ

Thanh toán

Bút toán nhật ký

Tài khoản

Tài khoản ngân hàng

Tài khoản kế toán

Đối tác

Đối tác

Cấu hình

Cài đặt cá nhân

Thông tin công ty

Kết nối AI (MCP)

Tổng quan hệ thống

Dữ liệu thời gian thực được tổng hợp từ Core Service

Tổng Phải Thu (AR)

11.478.256.908,81 đ

Tổng Phải Trả (AP)

3.749.054.325 đ

Thanh toán tháng này

3.137.000.000 đ

Hóa đơn AP chờ duyệt

0

Doanh thu & Chi phí

Tổng hợp theo tháng (triệu VND)

Th12

Th01

Th02

Th03

Th04

Th05

Trạng thái Hóa đơn

Phân bố tổng số lượng hóa đơn

Chờ duyệt

Từ chối

Đã duyệt

Hóa đơn chờ xử lý

Các hóa đơn cần phê duyệt hoặc xử lý

Xem tất cả →

Zenith Logistics Inc

13421 · 24/05/2026

AR

11.000.000.000 đ

open

Thanh toán gần đây

Các giao dịch thanh toán mới nhất

Xem tất cả →

NovaTech Innovation...

112.000.000 đ

Nguyen Van A

ROLE: FINANCE MANAGER

4.2.4. Giao diện danh sách Hoá đơn chứng từ

Dashboard

Hoá đơn & Giao dịch

Hóa đơn & Chứng từ

Thanh toán

Bút toán nhật ký

Tài khoản

Tài khoản ngân hàng

Tài khoản kế toán

Đối tác

Đối tác

Cấu hình

Cài đặt cá nhân

Thông tin công ty

Kết nối AI (MCP)

NV
Nguyen Van A
ROLE_FINANCE_MANAGER

🏠 🔄 🔍 Search... Searching...

Hóa đơn & Chứng từ

Quản lý hóa đơn bán hàng (AR), hóa đơn mua hàng (AP) và phê duyệt thanh toán.

Công việc cần xử lý Tắt cả hóa đơn

🔍

🔄 + Thêm hóa đơn

☐	Số HĐ	Loại	Ngày HĐ	Đối tác	Tổng tiền	Còn nợ	Trạng thái	Phê duyệt	
☐	DRAFT	HĐ Mua (AP)	25/5/2026	SkyHigh Construction Ltd	135.000.000 đ	135.000.000 đ	Open	Đã duyệt	...
☐	1231	HĐ Mua (AP)	24/5/2026	NovaTech Innovations LLC	112.000.000,00 US\$	0,00 US\$	Paid	Đã duyệt	...
☐	DRAFT	HĐ Mua (AP)	24/5/2026	Apex Global Trading Co.	120.000.000 đ	120.000.000 đ	Open	Đã duyệt	...
☐	13421	HĐ Bán (AR)	24/5/2026	Zenith Logistics Inc	11.000.000.000 đ	11.000.000.000 đ	Open	—	...
☐	DRAFT	HĐ Mua (AP)	24/5/2026	SkyHigh Construction Ltd	120.000.000 đ	120.000.000 đ	Open	Đã duyệt	...
☐	143214	HĐ Mua (AP)	23/5/2026	BrightFuture Energy Co.	112.000.000 đ	112.000.000 đ	Open	Đã duyệt	...
☐	23142314	HĐ Mua (AP)	23/5/2026	NovaTech Innovations LLC	212.800.000,00 US\$	212.800.000,00 US\$	Open	Từ chối	...
☐	2313	HĐ Bán (AR)	23/5/2026	NovaTech Innovations LLC	125.440.000 đ	125.440.000 đ	Open	—	...
☐	1321	HĐ Mua (AP)	23/5/2026	Apex Global Trading Co.	212.800.000,00 US\$	212.800.000,00 US\$	Open	Đã duyệt	...
☐	4321	HĐ Mua (AP)	23/5/2026	NovaTech Innovations LLC	526.400.000,00 US\$	526.400.000,00 US\$	Open	Đã duyệt	...

0 of 20 row(s) selected. Rows per page 10 Page 1 of 2

4.2.5. Giao diện tạo Hoá đơn chứng từ

Dashboard

Hoá đơn & Giao dịch

Hóa đơn & Chứng từ

Thanh toán

Bút toán nhật ký

Tài khoản

Tài khoản ngân hàng

Tài khoản kế toán

Đối tác

Đối tác

Cấu hình

Cài đặt cá nhân

Thông tin công ty

Kết nối AI (MCP)

NV
Nguyen Van A
ROLE_FINANCE_MANAGER

🏠 🔄 🔍 Search... Searching...

← Tạo hóa đơn

Thông tin chính

Nhập các thông tin chính của hóa đơn

Loại

AR Invoice

Đối tác

Chọn

Ngày hóa đơn

27/05/2026

Số HĐ

Tiền tệ

VND

Trạng thái

Trạng thái của hóa đơn

DRAFT

Lưu

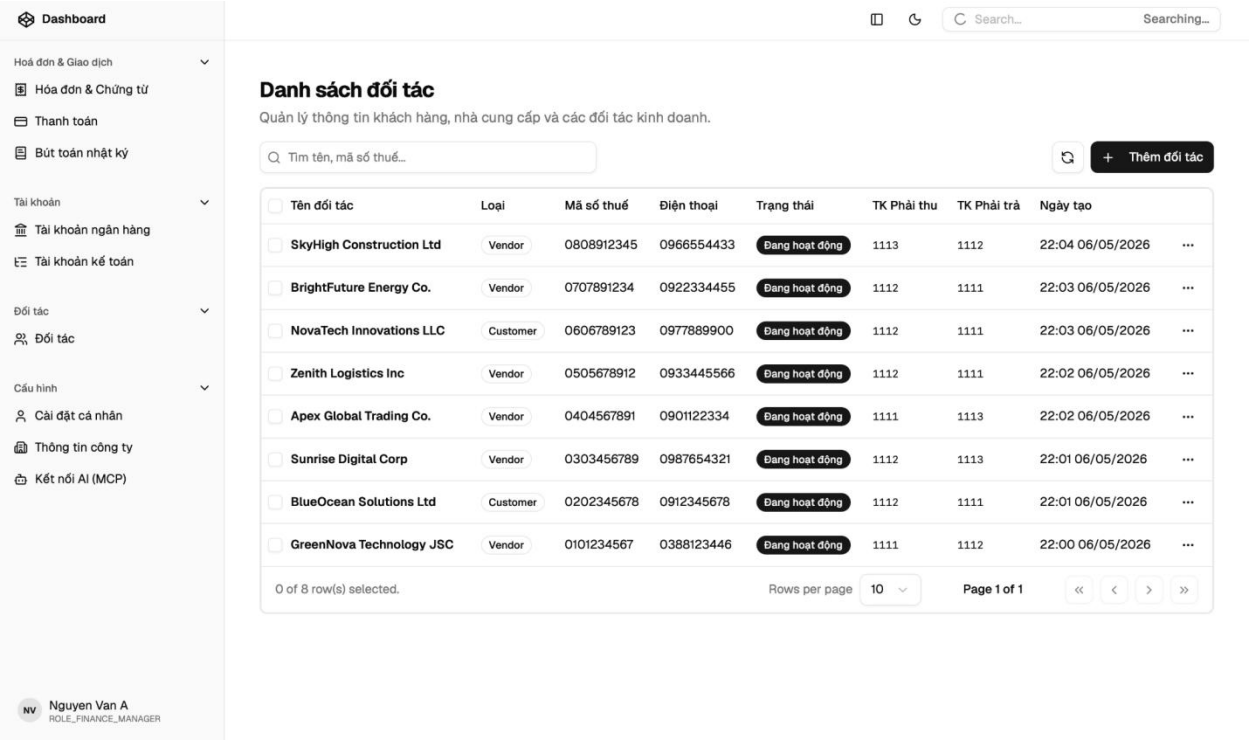
Chi tiết hóa đơn

Dòng hóa đơn

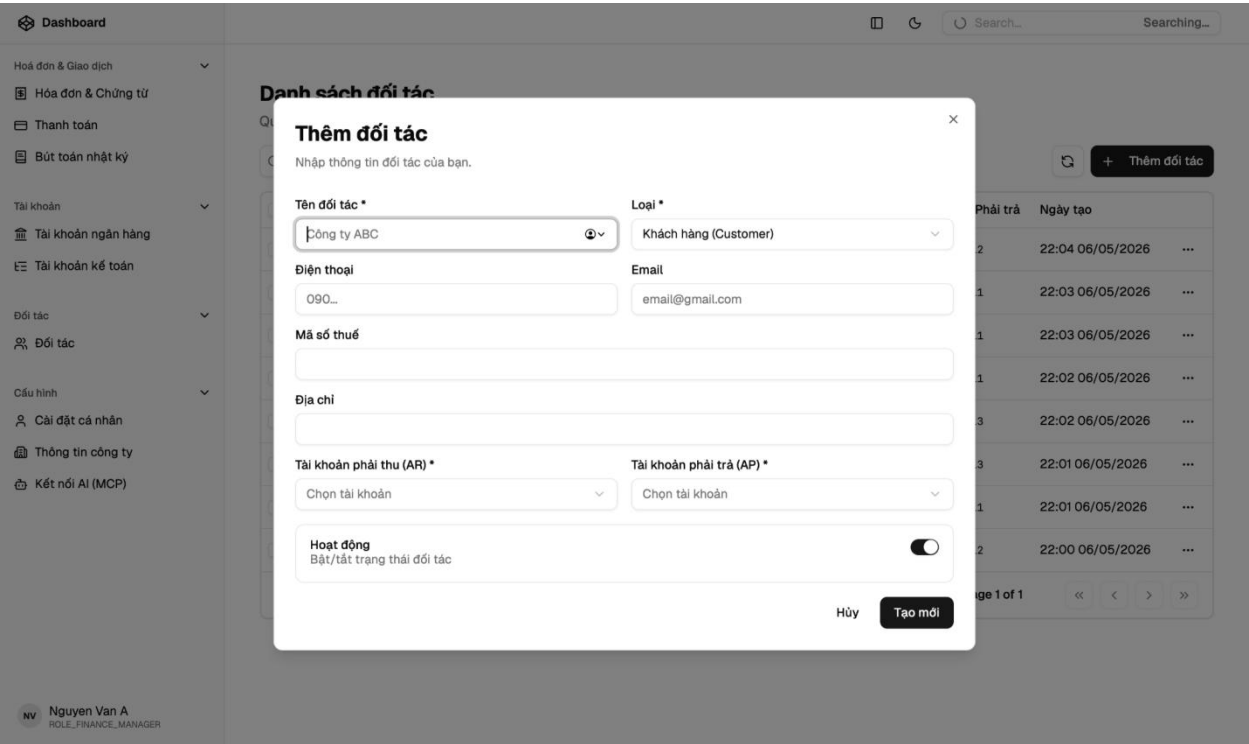
+ Thêm

Mô tả	TK	SL	Giá	Thuế%	Thuế	Tiền
Nhập mô tả...		1			0	...
Subtotal						0
Tax						0
Total						0

4.2.6. Giao diện danh sách đối tác.



4.2.7. Giao diện form thêm đối tác.



4.2.8. Giao diện cập nhật thông tin cá nhân.

Dashboard

Hoá đơn & Giao dịch

Hóa đơn & Chứng từ

Thanh toán

Bút toán nhật ký

Tài khoản

Tài khoản ngân hàng

Tài khoản kế toán

Đối tác

Đối tác

Cấu hình

Cài đặt cá nhân

Thông tin công ty

Kết nối AI (MCP)

NV

Nguyen Van A

ROLE: FINANCE_MANAGER

Search...

Searching...

Cài đặt cá nhân

Thông tin tài khoản

Cập nhật thông tin cá nhân của bạn

Họ và tên

Nguyen Van A

Email

denafoi664@homvela.com

Lưu

4.2.9. Giao diện thông tin doanh nghiệp.

Dashboard

Hoá đơn & Giao dịch

Hóa đơn & Chứng từ

Thanh toán

Bút toán nhật ký

Tài khoản

Tài khoản ngân hàng

Tài khoản kế toán

Đối tác

Đối tác

Cấu hình

Cài đặt cá nhân

Thông tin công ty

Kết nối AI (MCP)

NV Nguyen Van A
ROLE: FINANCE_MANAGER

Search...

Searching...

Cài đặt doanh nghiệp

Thông tin doanh nghiệp

Quản lý các thông tin cơ bản của doanh nghiệp

Tên doanh nghiệp

ACC

Mã số thuế

3215423

Tiền tệ mặc định

VND - Việt Nam Đồng

Địa chỉ

Hanoi

Lưu

4.2.10. Giao diện thông tin hướng dẫn kết nối đến MCP Server bằng AI Agent.

Dashboard

Hoá đơn & Giao dịch

Hóa đơn & Chứng từ

Thanh toán

Bút toán nhật ký

Tài khoản

Tài khoản ngân hàng

Tài khoản kế toán

Đối tác

Đối tác

Cấu hình

Cài đặt cá nhân

Thông tin công ty

Kết nối AI (MCP)

NV Nguyen Van A
ROLE: FINANCE_MANAGER

Search...

Searching...

Kết nối AI (MCP) Beta

Tích hợp EFMS với Claude Desktop thông qua Model Context Protocol (MCP) để AI có thể truy cập dữ liệu tài chính của bạn.

OAuth 2.1 Endpoints

Các endpoint xác thực dùng để cấu hình OAuth flow với Claude Desktop

Client ID — Định danh OAuth client

claude-connector

Client Secret Key — Mã bảo mật OAuth

scXL1fmeZSXICXu8nbWbwzq

MCP Server

Endpoint MCP server dùng HTTP/SSE transport

MCP URL — Endpoint chính cho Claude Desktop

https://mcp.hnhdecor.com/mcp

Transport Type — Streaming HTTP (StreamableHTTP)

http

MCP server yêu cầu Bearer token hợp lệ được cấp qua OAuth flow. Claude Desktop sẽ tự động xử lý luồng xác thực khi bạn cấu hình đúng.

4.2.11. Giao diện danh sách người dùng dành cho admin.

Dashboard

Quản trị hệ thống

Người dùng

Vai trò & Quyền người dùng

Cấu hình

Cài đặt cá nhân

Thông tin công ty

Kết nối AI (MCP)

VL

Viet Linh

ROLE_ADMIN

Quản lý người dùng

Xem danh sách, phân quyền và quản lý trạng thái tài khoản người dùng trong hệ thống.

Q

Tim theo tên, email...

Thêm người dùng

Họ và tên	Vai trò	Công ty	Trạng thái	Ngày tạo	Thao tác
Viet Linh linhtx2004@gmail.com	ROLE_ADMIN	ACC	Hoạt động	---	...
acc gagov49515@itquoted.com	ROLE_ACCOUNTANT	ACC	Hoạt động	---	...
Auditor rekexaj936@itquoted.com	ROLE_AUDITOR	ACC	Hoạt động	---	...
admin latijek873@ameady.com	ROLE_FINANCE_MANAGER	ACC	Hoạt động	---	...
Nguyen Van A denafol664@homvela.com	ROLE_FINANCE_MANAGER	ACC	Hoạt động	---	...

0 of 5 row(s) selected.

Rows per page10

Page 1 of 1

<<<>>>

4.2.12. Giao diện thêm nhân viên.

Dashboard

Quản trị hệ thống

Người dùng

Vai trò & Quyền người dùng

Cấu hình

Cài đặt cá nhân

Thông tin công ty

Kết nối AI (MCP)

VL

Viet Linh

ROLE_ADMIN

Quản lý người dùng

Xem danh sách, phân quyền và quản lý trạng thái tài khoản người dùng trong hệ thống.

Q

Tim theo tên, email...

Thêm người dùng

Họ và tên	Vai trò	Công ty	Trạng thái	Ngày tạo	Thao tác
Viet Linh linhtx2004@gmail.com			Hoạt động	---	...
acc gagov49515@itquoted.com			Hoạt động	---	...
Auditor rekexaj936@itquoted.com			Hoạt động	---	...
admin latijek873@ameady.com			Hoạt động	---	...
Nguyen Van A denafol664@homvela.com			Hoạt động	---	...

0 of 5 row(s) selected.

ar page10

Page 1 of 1

<<<>>>

Mời nhân viên

Email nhân viên mới

vd: employee@company.com

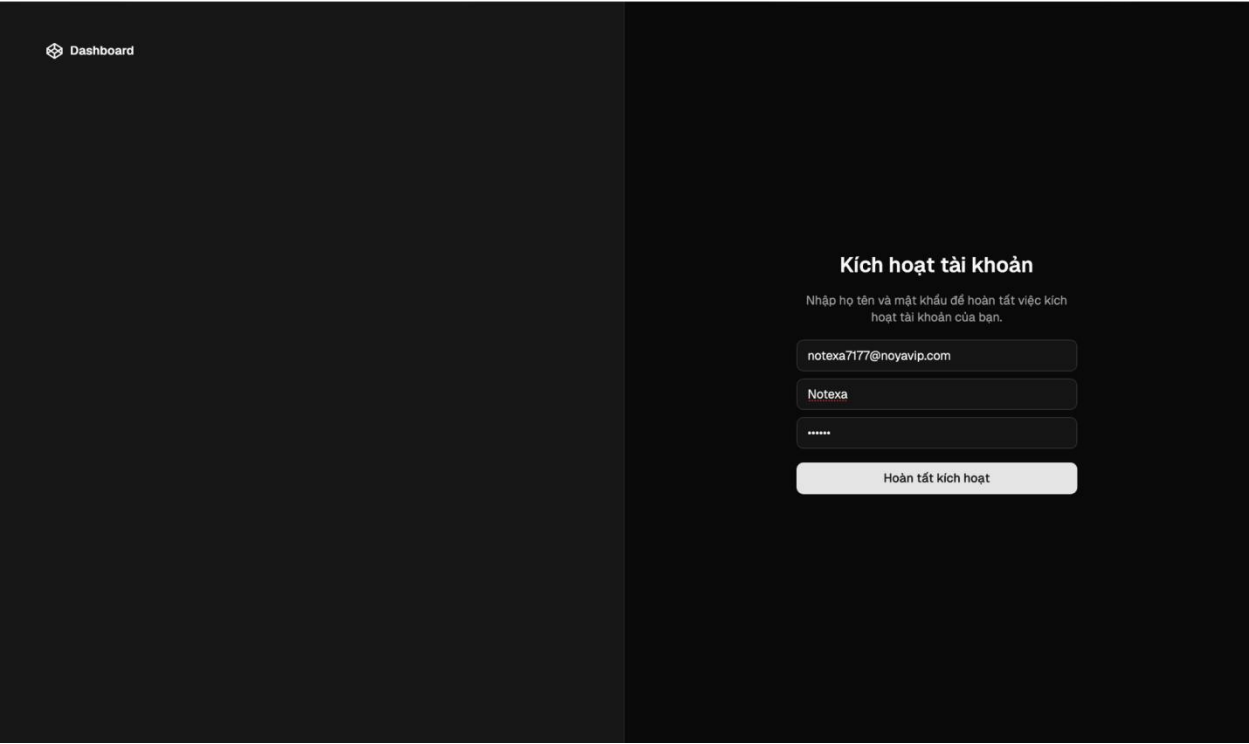
Phân quyền ban đầu

ROLE_ACCOUNTANT

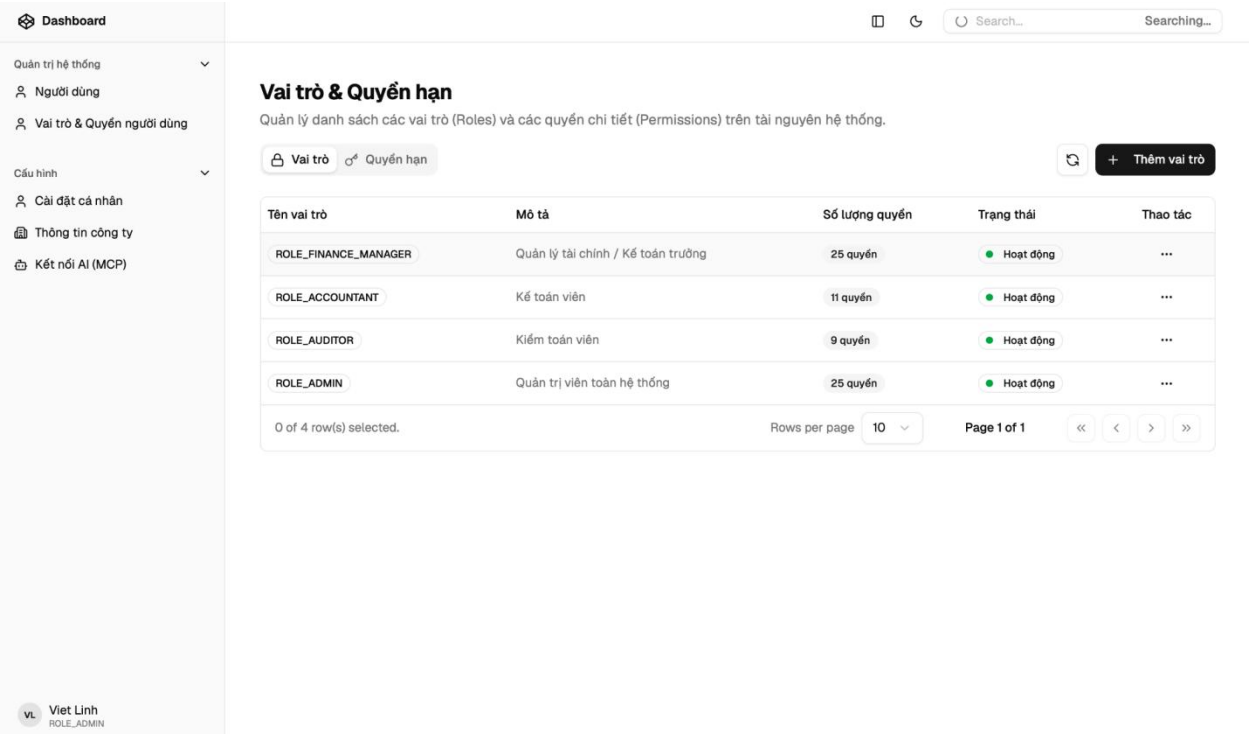
Hủy

Gửi lời mời

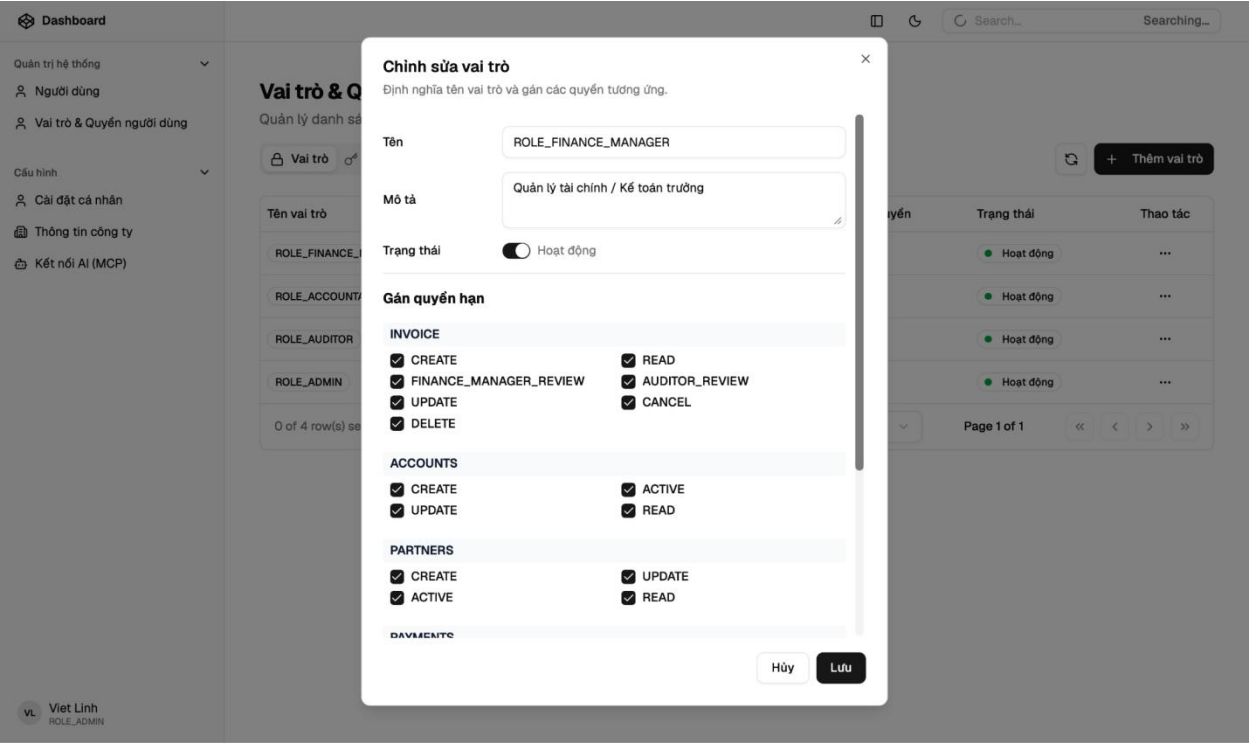
4.2.13. Giao diện form kích hoạt tài khoản dành cho nhân viên được mời.



4.2.14. Giao diện quản lý Vai trò & Quyền người dùng.



4.2.15. Giao diện form thiết lập Quyền người dùng.



4.3. Kiểm thử

4.3.1. Kiểm thử chức năng Đăng nhập & Xác thực (Identity Service)

Hệ thống EFMS sử dụng cơ chế bảo mật không trạng thái, yêu cầu trả về mã thông báo JWT để phân quyền khi đăng nhập thành công.

Bảng 4.1. Kiểm thử chức năng đăng nhập

TC	Các bước thực hiện	Kết quả dự kiến	Kết quả đạt được
TC1	1. Nhập email không tồn tại trên hệ thống 2. Nhập mật khẩu bất kỳ 3. Bấm đăng nhập	Thông báo tài khoản không tồn tại / Thông tin đăng nhập không chính xác.	Đạt
TC2	1. Nhập đúng email của người dùng 2. Nhập sai mật khẩu 3. Bấm đăng nhập	Thông báo mật khẩu không chính xác.	Đạt
TC3	1. Bỏ trống trường email 2. Bỏ trống mật khẩu 3. Bấm đăng nhập	Tại view hiển thị, hệ thống chặn và yêu cầu người dùng nhập đầy đủ email, mật khẩu.	Đạt
TC4	1. Nhập đúng email 2. Nhập đúng mật khẩu 3. Bấm đăng nhập	Đăng nhập thành công. Hệ thống sinh và trả về mã JWT token hợp lệ. Chuyển hướng người dùng đến trang Dashboard tương ứng với quyền hạn.	Đạt

4.3.2. Kiểm thử chức năng Quản lý Người dùng (Multi-tenancy & RBAC)

Thay vì chức năng "Đăng ký" tự do như các website bán hàng, hệ thống EFMS là phần mềm doanh nghiệp nên Admin sẽ chịu trách nhiệm thêm người dùng mới và phân quyền (Role) cho họ trong cùng một công ty.

Bảng 4.2. Kiểm thử chức năng tạo người dùng mới

TC	Các bước thực hiện	Kết quả dự kiến	Kết quả đạt được
TC1	1. Nhập sai định dạng email 2. Nhập đầy đủ các thông tin khác 3. Bấm Lưu	Thông báo nhập sai định dạng email.	Đạt
TC2	1. Nhập đúng định dạng email 2. Bỏ trống việc chọn Vai trò (Role) 3. Bấm Lưu	Thông báo không được bỏ trống trường Vai trò.	Đạt
TC3	1. Nhập email hợp lệ nhưng đã tồn tại trong hệ thống 2. Bấm Lưu	Thông báo email này đã được sử dụng.	Đạt
TC4	1. Bỏ trống tên người dùng 2. Bấm Lưu	Thông báo không được bỏ trống các trường thông tin bắt buộc.	Đạt
TC5	1. Nhập đầy đủ thông tin hợp lệ 2. Chọn đúng Vai trò (VD: Accountant) 3. Bấm Lưu	Tạo tài khoản thành công. Tài khoản mới được tự động gán đúng mã định danh của doanh nghiệp (company_id) để cô lập dữ liệu.	Đạt

4.3.3. Kiểm thử luồng nghiệp vụ Hóa đơn mua hàng (AP Bill - Core Service)

Đây là nghiệp vụ cốt lõi, kiểm thử vòng đời của hóa đơn qua cơ chế kiểm soát trạng thái tại cơ sở dữ liệu (DB State Machine).

Bảng 4.3. Kiểm thử luồng quản lý và phê duyệt Hóa đơn

TC	Tên	Các bước thực hiện	Kết quả dự kiến	Kết quả đạt được
TC1	Thêm mới	1. Bỏ trống thông	Thông báo lỗi không được	Đạt

TC	Tên	Các bước thực hiện	Kết quả dự kiến	Kết quả đạt được
	hóa đơn	tin Đối tác (Nhà cung cấp) 2. Bấm Tạo hóa đơn	bỏ trống trường Đối tác.	
TC2	Thêm mới hóa đơn	1. Nhập đầy đủ thông tin hợp lệ (Đối tác, Số tiền...) 2. Bấm Tạo hóa đơn	Tạo thành công. Hóa đơn được lưu ở trạng thái bản nháp (Draft).	Đạt
TC3	Xóa hóa đơn	1. Chọn một hóa đơn đang ở trạng thái Draft 2. Bấm Xóa	Thông báo xóa thành công. Hóa đơn biến mất khỏi danh sách.	Đạt
TC4	Xác nhận hóa đơn	1. Chọn hóa đơn đang ở trạng thái Draft 2. Bấm Xác nhận (Confirm)	Hóa đơn chuyển trạng thái sang Chờ duyệt (approval_status=pending). Hệ thống khóa, không cho phép xóa hóa đơn nữa.	Đạt
TC5	Phê duyệt hóa đơn	1. Đăng nhập bằng tài khoản Quản lý tài chính 2. Chọn hóa đơn đang chờ duyệt 3. Bấm Phê duyệt (Approve)	Trạng thái hóa đơn chuyển thành Approved. Hệ thống tự động phát sinh Bút toán kép (Journal Entries) tương ứng vào Sổ cái.	Đạt

KẾT LUẬN

1. Kết quả đạt được

Sau quá trình nghiên cứu, phân tích, thiết kế và xây dựng, đồ án đã hoàn thành hệ thống EFMS (Enterprise Financial Management System) với các chức năng cơ bản đáp ứng nhu cầu quản lý tài chính và kế toán trong doanh nghiệp.

Hệ thống đã xây dựng thành công các chức năng quản lý khách hàng, nhà cung cấp, tài khoản kế toán, tài khoản ngân hàng, hóa đơn phải thu, hóa đơn phải trả và thanh toán. Người dùng có thể tạo, cập nhật, xác nhận hoặc hủy hóa đơn; ghi nhận thu, chi tiền; phân bổ thanh toán và theo dõi công nợ.

Đối với quản lý tài chính và kế toán trưởng, EFMS hỗ trợ kiểm tra, phê duyệt hoặc từ chối hóa đơn mua hàng, đồng thời cung cấp Dashboard theo dõi các khoản phải thu, phải trả, thanh toán trong kỳ và hóa đơn đang chờ duyệt. Hệ thống cũng tự động sinh các bút toán kế toán từ nghiệp vụ thanh toán.

Đối với quản trị viên, hệ thống hỗ trợ quản lý doanh nghiệp, người dùng, vai trò và quyền truy cập theo mô hình RBAC. Cơ chế đa doanh nghiệp giúp phân tách dữ liệu và bảo đảm người dùng chỉ được truy cập thông tin thuộc công ty của mình.

Bên cạnh đó, hệ thống đã tích hợp Claude AI thông qua MCP Server nhằm hỗ trợ tra cứu dữ liệu bằng ngôn ngữ tự nhiên, phân tích rủi ro hóa đơn và cung cấp gợi ý cho người phê duyệt. AI đóng vai trò hỗ trợ, còn quyết định nghiệp vụ cuối cùng vẫn thuộc về người dùng có thẩm quyền.

Hệ thống được xây dựng theo kiến trúc microservices với React, Java 21, Spring Boot, Spring Cloud và PostgreSQL. API Gateway thực hiện định tuyến và xác thực JWT, góp phần bảo đảm tính bảo mật, khả năng bảo trì và mở rộng hệ thống.

2. Hạn chế

Mặc dù đã hoàn thành các chức năng chính trong phạm vi đồ án, hệ thống vẫn còn một số hạn chế cần tiếp tục nghiên cứu và hoàn thiện:

- Giao diện chưa được tối ưu hoàn toàn cho thiết bị di động.
- Hệ thống chưa triển khai đầy đủ các nghiệp vụ kế toán chuyên sâu như kỳ kế toán, khóa sổ, báo cáo tài chính và đối soát ngân hàng.
- Chưa tích hợp được nền tảng quản lý quy trình nghiệp vụ Camunda 8.
- Chưa tích hợp xác thực OTP, xác thực đa yếu tố, ngân hàng, hóa đơn điện tử và các hệ thống ERP bên ngoài.
- Các giải pháp tối ưu hiệu suất, giám sát, sao lưu và bảo mật nâng cao cho môi trường triển khai quy mô lớn chưa được áp dụng đầy đủ.

Do thời gian thực hiện và kinh nghiệm triển khai thực tế còn hạn chế nên hệ thống vẫn còn một số thiếu sót cần tiếp tục hoàn thiện trong tương lai.

3. Hướng phát triển đồ án

Trong thời gian tới, hệ thống EFMS có thể được tiếp tục phát triển và mở rộng theo các hướng sau:

- Tối ưu giao diện responsive trên điện thoại và máy tính bảng.
- Bổ sung các nghiệp vụ kế toán chuyên sâu, báo cáo tài chính và đối soát ngân hàng.

- Bổ sung thêm hệ thống quản lý quy trình nghiệp vụ Camunda 8.
- Xây dựng quy trình phê duyệt nhiều cấp linh hoạt theo hạn mức và chính sách của từng doanh nghiệp.
- Hoàn thiện chức năng OTP, xác thực đa yếu tố và tăng cường bảo mật hệ thống.
- Tích hợp ngân hàng, hóa đơn điện tử, cổng thanh toán và các hệ thống ERP.
- Tối ưu hiệu suất, khả năng chịu tải, giám sát, sao lưu dữ liệu và phát triển ứng dụng mobile.

TÀI LIỆU THAM KHẢO

- [1] Nhóm phát triển EFMS, *Tài liệu kiến trúc hệ thống EFMS*, tài liệu nội bộ dự án.
- [2] Nhóm phát triển EFMS, *Tài liệu mô tả luồng nghiệp vụ và phân quyền hệ thống EFMS*, tài liệu nội bộ dự án.
- [3] Oracle, *Java Platform, Standard Edition 21 Documentation*. Địa chỉ: <https://docs.oracle.com/en/java/javase/21/> (truy cập ngày 12/06/2026).
- [4] Spring, *Spring Boot 3.3 Reference Documentation*. Địa chỉ: <https://docs.spring.io/spring-boot/3.3/index.html> (truy cập ngày 12/06/2026).
- [5] Spring, *Spring Cloud Gateway Reference Documentation*. Địa chỉ: <https://docs.spring.io/spring-cloud-gateway/reference/index.html> (truy cập ngày 12/06/2026).
- [6] Spring, *Spring Security Reference*. Địa chỉ: <https://docs.spring.io/spring-security/reference/index.html> (truy cập ngày 12/06/2026).
- [7] The PostgreSQL Global Development Group, *PostgreSQL Documentation*. Địa chỉ: <https://www.postgresql.org/docs/current/> (truy cập ngày 12/06/2026).
- [8] Meta Open Source, *React Reference*. Địa chỉ: <https://react.dev/reference/react> (truy cập ngày 12/06/2026).
- [9] Apache Software Foundation, *Apache Maven Documentation*. Địa chỉ: <https://maven.apache.org/guides/index.html> (truy cập ngày 12/06/2026).
- [10] M. Jones, J. Bradley và N. Sakimura, *RFC 7519: JSON Web Token (JWT)*, Internet Engineering Task Force, 2015. Địa chỉ: <https://www.rfc-editor.org/rfc/rfc7519.html> (truy cập ngày 12/06/2026).
- [11] D. Hardt, *RFC 6749: The OAuth 2.0 Authorization Framework*, Internet Engineering Task Force, 2012. Địa chỉ: <https://www.rfc-editor.org/rfc/rfc6749.html> (truy cập ngày 12/06/2026).
- [12] Model Context Protocol, *Model Context Protocol Specification*. Địa chỉ: <https://modelcontextprotocol.io/specification/2025-11-25> (truy cập ngày 12/06/2026).
- [13] OWASP Foundation, *OWASP Top 10: Web Application Security Risks*. Địa chỉ: <https://owasp.org/www-project-top-ten/> (truy cập ngày 12/06/2026).